

The Disposition Pattern: Recording Potential in IES Common

Table of Contents

1 Introduction.....	4
1.1 Purpose and Status	4
1.2 The Problem: Recording Potential in an Extensional Ontology	5
1.3 Two Aspects of Capability.....	5
1.4 Purpose and Approach	5
1.5 Scope.....	7
1.6 Conventions	7
2 Architectural Direction: IES Top and Core	7
2.1 The Collapse of Entity and State	7
2.2 The Disposition Hierarchy.....	8
2.3 Activities as States	8
2.4 Motivation for the State/ClassOfState Encoding.....	9
2.5 Migration Path	9
3 Requirements: What Information Needs to Be Recorded.....	10
3.1 Non-Specific Assets with Potential	11
3.2 Class-Level Potential	11
3.3 Individual-Level Potential	11
3.4 Permitted and Prohibited Uses.....	12
3.5 Whole-Part Dependencies.....	13
3.6 Cross-Entity Dependencies.....	14
3.7 Measured and Quantified Potentials	14
4 Background: Modelling Potential (“Can Do”)	15
4.1 The core challenge	15
4.2 Set theory and possible worlds	17
Individual-level disposition	17
Class-level disposition	18
4.3 The type–token distinction in IES.....	19
4.4 Where the two encodings diverge.....	21
5 The Disposition Pattern.....	22
5.1 The Current Encoding (Aspect 1 Requirements).....	22

5.1.1 The Meta-Class Infrastructure	22
5.1.2 Two Interpretations	25
5.1.3 Set-Theoretic Description	26
5.1.4 Narrowing to State/ClassOfState Semantics	27
5.1.5 Applying to the Requirements	27
5.1.6 Coverage Assessment	39
5.2 Part 2: Proposed Refinements for Aspect 1.....	40
5.2.1 Motivation.....	40
5.2.2 Two New Properties	41
5.2.3 Applying to the Requirements	41
5.2.4 The State-Bearing vs Event-Bearing Question	49
5.3 Aspect 2: The Dependency Gap	50
6 Addressing Reviewer Feedback.....	51
6.1 “Starting with a UML Conceptual Model would have helped”	52
6.2 “Whole-life Entities cannot have capacities”	52
6.3 “Better alignment with modal logic — unsubstantiated”	53
6.4 “Clearer spatio-temporal semantics — unsubstantiated”	53
6.5 “The proposal reduces expressivity”	54
6.6 “The original model supports non-specific assets, class-level, and individual-level; the proposal excludes this”	54
6.7 “That’s how extensional ontologies work. Fudging this won’t make it better.”	55
6.8 “Disposition and ‘capacity’ are not 4D problems, it’s set-theory and possible worlds”	55
7 Findings and Questions for the IES-TG.....	56
7.1 Summary of Findings.....	56
7.2 Overclaims Acknowledged.....	56
7.3 Questions for the IES-TG.....	57
Appendix A: The Semantic Locus of the Modal Commitment	58
A.1 The Encoding Properties as the Determining Factor	58
A.2 Effect on Member Definitions	59
A.3 Three Consequences	60
A.4 Could PossibleWorld Provide the Missing Modality?.....	60
A.5 Summary	62
Appendix B Disposition and Dependencies	63
B.1 Set Theoretic Description.....	66

Appendix C Degradation Spectra and Capability Enhancement	76
C.1 Introduction	76
C.2 The Degradation Spectrum as a Lattice of States	76
C.3 Worked Example: Eurofighter Propulsion Degradation	77
C.3.1 The scenario	77
C.3.2 State classes for the propulsion system	77
C.3.3 State classes for the fighter.....	78
C.3.4 Capability instances.....	79
C.3.5 Disposition and dependency assertions.....	79
C.3.6 Disposition inheritance through the subclass hierarchy	80
C.3.7 Instance data: a degradation event	81
C.4 Interaction with Measured Dispositions.....	82
C.5 Capability Enhancement	83
C.5.1 The scenario	83
C.5.2 Enhanced state classes.....	83
C.5.3 The new disposition	83
C.5.4 Disposition inheritance in the enhanced case.....	84
C.6 The Complete Spectrum.....	84
C.7 Summary	85
Appendix D Evacuation Route	86
D.1 Scenario.....	86
D.2 Prefixes and Namespaces.....	87
D.3 Domain Extension Classes.....	88
D.4 Network Structure	89
D.4a The Evacuation Flow	91
D.5 Class-Level Disposition (§3.2)	92
D.6 Individual-Level Disposition (§3.3).....	92
D.7 Permitted Uses During the Emergency (§3.4)	93
D.8 Whole-Part Dependency (§3.5).....	94
D.9 Cross-Entity Dependency (§3.6).....	95
D.10 Putting It Together	96
D.11 Summary	97
Appendix E: Sandown Water Treatment Works Degradation.....	98
E.1 Scenario	98

E.2 Prefixes and Namespaces	99
E.3 Domain Extension Classes	100
E.4 Instance Data: The Infrastructure	101
E.5 The Degradation Spectrum: State Class Hierarchy	102
E.5.1 State classes for the WTW	102
E.5.2 State classes for the Sandown WTW (individual level).....	103
E.5.3 State classes for the sub-station.....	103
E.5.4 State classes for the backup generator.....	104
E.6 Disposition Assertions.....	104
E.6.1 Class-level dispositions (§3.2).....	104
E.6.2 Individual-level dispositions (§3.3).....	105
E.6.3 Disposition inheritance through the subclass hierarchy	105
E.7 Dependency Assertions	106
E.7.1 Cross-entity dependency: full capacity requires mains power (§3.6)	106
E.7.2 Whole-part dependency: degraded mode requires the backup generator (§3.5)	106
E.7.3 The cascading dependency	107
E.8 The Actuality: Sub-Station Failure Event	107
E.8.1 The event	107
E.8.2 States of the sub-station.....	108
E.8.3 States of the WTW	108
E.8.4 Structural comparison with Appendix C	110
E.9 Queries.....	110
E.10 Summary.....	112

1 Introduction

1.1 Purpose and Status

This document is an **informational analysis** prepared for the IES Technical Group (IES-TG). It examines the requirements for recording what things *can* do — their potential — within IES Common, analyses how far the existing disposition mechanism meets those requirements, and identifies where complementary work is needed. It is not a formal change proposal. Its purpose is to establish a shared understanding of the problem space and to seek the IES-TG’s endorsement of the analysis, the direction of travel, and the areas requiring further development.

The document has been developed collaboratively. David Leal’s review has contributed materially to the analysis, particularly in identifying two distinct aspects of capability (discussed in §1.3) that inform the

document's structure and recommendations. Earlier community feedback has also shaped the requirements and the treatment of the existing encoding mechanism.

1.2 The Problem: Recording Potential in an Extensional Ontology

Organisations routinely need to record what things *can* do. A defence planner needs to know that a Eurofighter can fly at Mach 2; an emergency planner needs to know that a community centre could serve as a shelter; a logistics manager needs to know that a vehicle can carry ten tonnes. These are modal claims — claims about possibility, not about what is actually happening. The aircraft may never fly at Mach 2; the community centre may never be used as a shelter. Yet the information that these things are *possible* is essential for planning, procurement, allocation, and readiness assessment.

The IES Common ontology is a four-dimensional (4D) extensional ontology used for enterprise information exchange across UK Government and its partners. In IES, identity is determined by spatio-temporal extent: an Entity is a four-dimensional object extended through space and time, and changes are modelled as States — temporal parts during which some property holds. This extensional commitment creates a fundamental difficulty for recording potential. The statement “Fighter XYZ-123 can fly at Mach 2” does not describe any actual State that exists in the world. There may be no spatio-temporal extent corresponding to this aircraft flying at Mach 2 — yet the information plainly needs to be recorded and exchanged.

The IES already provides a mechanism for recording dispositions — what things can do or are inclined to do — through the meta-classes `ies:DispositionalClass`, `ies:Capability`, and `ies:Tendency`, together with the properties `ies:isDisposedTo` and `ies:allHaveDisposition`. This mechanism encodes dispositions through class membership: `isDisposedTo` is a sub-property of `rdf:type`, and `allHaveDisposition` is a sub-property of `rdfs:subClassOf`. This document examines that existing mechanism, identifies its strengths and limitations, and analyses where it meets the requirements and where it does not.

1.3 Two Aspects of Capability

Review of this document has highlighted a distinction that is central to its conclusions. The requirements for recording potential involve two conceptually distinct aspects:

Aspect 1: Possible worlds — what could be. A church can be emergency accommodation. A Eurofighter can fly at Mach 2. In either case we are making a modal claim about the potential of the thing: there exists some possible world in which the thing is in a State of the specified kind. The claim is grounded in the set-theoretic intersection of the thing's possible States with a `ClassOfStates` (§4.2). Crucially, it is non-committal about *how* that state comes about — it does not enumerate the conditions that would need to be satisfied for the state to be realised.

Aspect 2: Designs and plans — what must hold structurally for the possible to be realised. A church operating as emergency accommodation must contain beds and have functional sanitation. A Eurofighter flying at Mach 2 must have both EJ200 engines installed and operational. These are not further modal claims about possibility — they are *structural constraints on composition*: assertions about what parts must be present and in what condition for a potential to be exercisable. They describe the design or operational requirements that govern the potential, not the potential itself.

The two aspects are logically related but conceptually separate. An Aspect 1 assertion says *that* a potential exists; an Aspect 2 assertion says *what must be true of the entity's composition* for that potential to be realisable. You can record one without the other: a planner may know that a Eurofighter can fly at Mach 2 without having access to the engineering specification that explains why, and an engineer may specify the propulsion requirements without making any claim about overall capability.

Both aspects are well understood in the broader ontology literature, but only Aspect 1 is currently addressed by IES. This distinction — between the modal assertion and the compositional constraint — structures the analysis in this document. The existing IES disposition mechanism, and the proposed refinements to it, address Aspect 1 well. Aspect 2 requires complementary work that this document identifies and scopes but does not fully resolve.

1.4 Purpose and Approach

The document is structured as follows:

§2 (Architectural Direction) establishes the Top and Core architecture: the collapse of Entity and State into a unified `ies_top:State` class, the migration of Event to Activity, the retention of the disposition meta-class hierarchy, and the motivation for adopting `State/ClassOfState` semantics.

§3 (Requirements) defines seven information exchange requirements through progressive Eurofighter scenarios. Requirements §3.1–§3.4 concern the modal assertion of potential (Aspect 1). Requirements §3.5–§3.6 concern dependencies on composition (Aspect 2). Requirement §3.7 (measured dispositions) is identified for future work. The requirements are pattern-neutral.

§4 (Background) establishes the shared theoretical foundation: the set-theoretic and possible-worlds semantics, the IES type–token apparatus, and the point at which the existing encoding and a proposed refinement diverge.

§5 (The Disposition Pattern) presents the existing encoding (Part 1) and introduces two proposed refinement properties (Part 2), applying both to requirements §3.1–§3.4. The dependency requirements (§3.5–§3.6) are analysed to show the coverage gap, and one possible approach to addressing them is presented for discussion. This section also considers the State-bearing versus Event-bearing question raised during review.

§6 (Addressing Reviewer Feedback) responds to specific concerns raised during peer review, acknowledging overclaims in the original proposal documentation and explaining the design decisions made in response.

§7 (Findings and Questions for the IES-TG) summarises the analysis and presents the specific questions on which the IES-TG’s guidance is sought.

Appendices provide supplementary worked examples in the civil emergency domain (evacuation route, degradation spectra).

1.5 Scope

In scope. The seven requirements defined in §3; the Eurofighter scenario used throughout; the existing `isDisposedTo/allHaveDisposition` encoding mechanism and its interpretation; two proposed refinement properties with `State/ClassOfState` semantics; set-theoretic and possible-worlds analysis; the Top and Core migration path; response to reviewer feedback; identification of the designs-and-plans gap.

Out of scope. Integration with IES Assessment patterns; full resolution of prohibitions (acknowledged as an open area); implementation guidance for specific triple stores or SPARQL endpoints; measured and quantified dispositions (identified as future work); full resolution of the designs-and-plans aspect (identified as requiring further collaborative development).

Deferred to appendices. Emergency and provisional use of facilities; cascading dependencies based on infrastructure scenarios.

1.6 Conventions

Turtle examples use IES v5 syntax and namespaces — the `ies:` prefix, `ies:State`, `ies:ClassOfState`, `ies:Entity`, etc. Consistent prefixes are used: `ies:` for the IES Common namespace, `ex:` for domain-specific class definitions, and `data:` for instance data. Set-theoretic notation is introduced in §4 and used consistently in §5. The Eurofighter scenario — including Fighter XYZ-123, EJ200 engines, and a Voyager tanker — is used as the running example across all sections.

2 Architectural Direction: IES Top and Core

At the outset of this work, we were briefed that the next major version of IES would restructure the ontology into two layers — `ies_top` (a general-purpose 4D extensional top ontology) and `ies_core` (a mid-level ontology providing cross-domain concepts and patterns). Among the planned changes were a significant simplification of the Entity/State class hierarchy and a reworking of the Event concept. These changes have direct implications for how dispositions should be modelled and motivated the encoding choices made throughout this document.

This section summarises the relevant architectural changes and explains why they led us to adopt `State` and `ClassOfState` semantics for the proposed properties from the outset.

2.1 The Collapse of Entity and State

In IES v5, `Entity` and `State` are separate classes. An `Entity` is a whole-life, four-dimensional object; a `State` is a temporal part of an `Entity` during which some property holds. The relationship between them is mediated by `isStateOf`, and the class hierarchy arranges each `Entity` subclass (e.g. `ies:Aircraft`) as also being a subclass of the corresponding `State` class (e.g. `ies:VehicleState`), so that whole-life `Entities` can satisfy `State`-typed domain constraints.

In the Top and Core architecture, this dual structure is eliminated. `ies_top:State` is defined as “*a SpatiotemporalExtent that is universe-bound*” and serves as the single foundational class for all concrete, world-bound individuals. There is no `Entity` class, no `isStateOf` property, and no

ClassOfEntity. What were previously Entity subclasses are now direct subclasses of `ies_top:State`:

- `ies_core:LivingOrganism` `rdfs:subClassOf` `ies_top:State`
- `ies_core:Artefact` `rdfs:subClassOf` `ies_top:State`
- `ies_core:Location` `rdfs:subClassOf` `ies_top:State`
- `ies_core:FusionOfIntegralParts` `rdfs:subClassOf` `ies_top:State`

Domain classes such as `ies_core:Vehicle`, `ies_core:Person`, and `ies_core:Organization` inherit from these and are therefore States. The distinction between a whole-life individual and its temporal parts is no longer a class-hierarchy concern; it is expressed through the mereological properties `ies_top:isPartOf` and `ies_top:isTemporalPartOf`, both of which have `ies_top:State` as their domain and range.

2.2 The Disposition Hierarchy

The `ies_core` ontology retains the disposition meta-class hierarchy, anchored to a revised set-theoretic terminology that renames `ClassOfX` to `SetOfX`:

```
ies_top:SetOfStates
├─ ies_core:Disposition
│   ├─ ies_core:Capability
│   └─ ies_core:Tendency
```

`ies_core:Disposition` is a subclass of `ies_top:SetOfStates` — the powertype of `ies_top:State`. This is the direct counterpart of the v5 arrangement where `ies:DispositionalClass` subclasses `ies:ClassOfState`, using the Top and Core naming convention.

The existing encoding properties are also preserved:

- `ies_core:isDisposedTo` — a sub-property of `rdf:type`, with domain `ies_top:State` and range `ies_core:Disposition`
- `ies_core:allHaveDisposition` — a sub-property of `rdfs:subClassOf`, with domain `ies_top:SetOfStates` and range `ies_core:Disposition`

2.3 Activities as States

In IES v5, `ies:Event` is not a subclass of `ies:State`. The Top and Core architecture replaces Event with `ies_core:Activity`, which sits within the `ies_top:State` hierarchy:

```
ies_top:State
├─ ies_core:FusionOfIntegralParts
│   └─ ies_core:Activity
```

Activities are therefore States. Any property with an `ies_top:State` domain — including the disposition properties defined in this document — admits Activities as bearers. This means that Activity-bearing dispositions — assertions that a particular kind of activity can give rise to

certain outcomes, or that an activity requires the co-occurrence of particular states — become structurally possible.

Whether Activity-bearing dispositions should be encouraged as a modelling practice is a question for the IES community. The practical case is plausible: a logistics planner might record that a particular type of resupply operation requires the participation of a vehicle with a specific payload capacity, or that a training exercise can produce a qualified crew state. No such requirement has been identified in the current analysis, and no use cases for Event-bearing dispositions have been raised in v5. The architectural change in Top and Core resolves the question of structural admissibility; the question of modelling guidance remains open.

2.4 Motivation for the State/ClassOfState Encoding

The Top and Core architecture motivates the central encoding decision in this document: the use of `State` and `ClassOfState` (or `SetOfStates` in Top and Core terminology) as the domain and range of the proposed disposition properties.

Dispositions are fundamentally about what kinds of temporal state an entity can be in. The proposed encoding makes this explicit: `hasDisposition` relates a `State` to a `ClassOfState`; `eachHasDisposition` relates a `ClassOfState` to a `ClassOfState`. This is the natural encoding for an ontology in which every concrete individual — whether formerly an `Entity`, a `State`, or an `Activity` — is a `State`.

Under IES v5, this choice requires an explanatory note: whole-life `Entities` satisfy the `State` domain because every `Entity` subclass is also a `State` subclass through the `Entity-as-State` subclassing pattern. Under Top and Core, no such explanation is needed — every concrete individual is a `State` by definition.

The same reasoning applies to the four dependency properties (`requires`, `eachRequires`, `requiresPart`, `eachRequiresPart` - see §5.2 Extending the Pattern: Six New Properties). By targeting `State` and `ClassOfState`, these properties work uniformly across all the kinds of thing that Top and Core provides: persons, vehicles, locations, organisations, and activities.

Throughout the remainder of this document, Turtle examples use IES v5 syntax and namespaces (the `ies:` prefix, `ies:State`, `ies:ClassOfState`, `ies:Entity`, etc.), since the proposed properties are first being introduced into IES Common v5. The migration to Top and Core namespaces and terminology is straightforward and is summarised in §2.5 below.

2.5 Migration Path

The following table maps the v5 constructs used throughout this document to their Top and Core equivalents:

This document (IES v5)	Top and Core	Notes
<code>ies:State</code>	<code>ies_top:State</code>	Semantics similar

This document (IES v5)	Top and Core	Notes
ies:ClassOfState	ies_top:SetOfStates	Naming convention change, set semantics
ies:DispositionalClass	ies_core:Disposition	Renamed; same structural role
ies:Capability	ies_core:Capability	Sub-class of ies_core:Disposition
ies:Tendency	ies_core:Tendency	Sub-class-of ies_core:Disposition
ies:Event	ies_core:Activity	Subclass of ies_top:State
ies:relationship (super-property)	ies_top:couple	See below

The six proposed properties require no structural redesign. Their domains, ranges, and sub-property relationships map directly, with a single exception: the super-property.

In IES v5, the proposed properties are sub-properties of ies:relationship. In Top and Core, ies_top provides a more differentiated property hierarchy:

- ies_top:couple — domain/range ies_top:Thing
- ies_top:relationship — domain/range ies_top:SpatiotemporalExtent
- ies_top:relationshipBetweenStates — domain/range ies_top:State

The proposed properties relate a State (a SpatiotemporalExtent) to a SetOfStates (a Set, which subclasses ies_top:Thing but not ies_top:SpatiotemporalExtent). Since the range crosses the State/Set boundary, these properties cannot be sub-properties of ies_top:relationship or ies_top:relationshipBetweenStates. The appropriate super-property is ies_top:couple, whose domain and range are ies_top:Thing and which therefore accommodates both States and Sets. This is a more precise placement than the v5 arrangement and correctly reflects the nature of disposition assertions as relationships between world-bound individuals and the types that characterise their potential states.

3 Requirements: What Information Needs to Be Recorded

This section defines the information exchange requirements for recording what things *can* do — their potential — within a 4D extensional ontology. The requirements are expressed through a progressive series of scenarios drawn from the military aviation domain. Each requirement describes what information a sending party needs to express, independent of how any particular encoding mechanism realises it.

Following the two-aspects distinction from §1.3, the requirements are organised into two groups:

Aspect 1 requirements (§3.1–§3.4) concern the modal assertion of potential — what things *could* do in some possible world. These are addressed by the existing disposition pattern and the proposed refinements.

Aspect 2 requirements (§3.5–§3.6) concern dependencies on composition — what *must* hold for potential to be realised. These require complementary work beyond the disposition pattern.

Future requirement (§3.7) concerns measured and quantified potentials and is identified for future work.

3.1 Non-Specific Assets with Potential

[Aspect 1 – possible worlds]

In early-stage mission planning, planners often need to reason about types of asset without yet knowing which specific type or platform will be used. The requirement is to record that *some class of thing* exists with a particular potential, without identifying that class by name.

Scenario. An operational planner is assembling a strike package. The plan requires an airborne asset that can fly at Mach 2. At this stage, the planner has not determined whether a Eurofighter, an F-35, or some other platform will fill the role — only that the plan requires a type of thing possessing this potential.

Information to be exchanged: “There exists a class of asset that can fly at Mach 2.”

This is the most abstract level of requirement: associating a potential with a class of thing whose identity is not yet resolved. Any pattern must be able to express this without forcing the sender to commit to a specific asset class.

3.2 Class-Level Potential

[Aspect 1 – possible worlds]

Once a specific asset type has been selected, planners need to record that *every member* of that class possesses a given potential. This is a universal assertion over a class rather than a statement about any individual.

Scenario. The planning team has determined that the Eurofighter Typhoon meets the Mach 2 requirement. They need to record that all Eurofighters — as a class — can fly at Mach 2.

Information to be exchanged: “All Eurofighters can fly at Mach 2.”

This requirement demands a mechanism for asserting that a potential applies to every instance of a named class, such that any individual Eurofighter inherits that assertion.

3.3 Individual-Level Potential

[Aspect 1 – possible worlds]

At the point of asset allocation, planners deal with specific airframes. An individual aircraft may have been assessed, certified, or designated as possessing a particular potential. This may be the result of a maintenance check, an operational assessment, or an engineering evaluation.

Scenario. Fighter XYZ-123 has completed a post-maintenance flight test and has been assessed as capable of Mach 2 flight. The assessment team needs to record that *this specific aircraft* has this potential.

Information to be exchanged: “Fighter XYZ-123 can fly at Mach 2.”

The distinction from §3.2 is important: class-level assertions apply universally, while individual-level assertions record specific assessments or designations. Note that an individual retains the potential of its class even when its current operational condition prevents it from actually realising that potential — a Eurofighter with a failed engine still *has* the potential to fly at Mach 2, but cannot currently do so. The relationship between potential and the conditions under which it can be realised is addressed in §3.4.

3.4 Permitted and Prohibited Uses

[Aspect 1 – possible worlds]

Potentials are rarely unconditional. An asset’s ability to realise a potential may depend on its current condition, and its *permission* to exercise that potential may be subject to regulatory or operational constraints. The requirement is to record both the conditions under which a potential can be realised and the circumstances under which exercising it is permitted or prohibited.

Scenario. Three distinct modifiers apply to Mach 2 flight in a Eurofighter:

1. **Conditional availability.** Mach 2 flight is only achievable when the aircraft is in an operational condition. An aircraft undergoing maintenance or with degraded systems cannot realise this potential regardless of its design characteristics. The condition constrains *when* the potential can be realised.
2. **Permitted use.** Mach 2 flight is a permitted use for Eurofighters when operating within designated supersonic training areas. The permission constrains *where/when and under what authority* the potential may be exercised.
3. **Prohibited use.** Low-altitude supersonic flight over populated areas is prohibited by standing operational orders, even when the aircraft is fully capable of it and the pilot is qualified to perform it. The prohibition constrains the potential regardless of the asset’s physical condition or the operational context.

Information to be exchanged:

- “Eurofighters can fly at Mach 2 only when in an operational condition.”
- “Mach 2 flight is a permitted use within designated supersonic training areas.”
- “Low-altitude supersonic flight over populated areas is a prohibited use.”

These three modifiers are distinct: conditional availability is a physical or engineering constraint; permission is an authorisation; prohibition is a restriction. All three must be expressible at both the class level (all Eurofighters) and the individual level (Fighter XYZ-123 has a specific operational restriction imposed after an incident). The encoding of these modifiers need not use a single mechanism — §5 treats conditional availability differently from permission and prohibition, reflecting their fundamentally different natures.

3.5 Whole-Part Dependencies

[Aspect 2 – designs and plans]

Complex assets are composed of parts, and the potential of the whole frequently depends on the states of those parts. Because parts are mereologically contained within the whole — they occupy a subset of its spacetime — these are intra-entity dependencies: the dependency follows the part-whole structure. The requirement is to record that one entity's potential is dependent on the potential of its component entities.

Scenario. A Eurofighter's potential to fly at Mach 2 depends on its propulsion system, which comprises twin EJ200 jet engines. When both engines are fully operational, the aircraft can achieve Mach 2. If one engine fails, the aircraft can still fly but the Mach 2 potential is lost — the aircraft is in a degraded state. The detailed modelling of degradation spectra (partial capability, "limp modes", and graduated performance loss) is discussed in Appendix C Degradation Spectra; for the purposes of this requirement, the key point is that the potential of the whole depends on the states of its parts.

Importantly, these dependencies may span different types of part. A Eurofighter's potential to fly at Mach 2 depends not only on both EJ200 engines being operational, but also on the airframe being structurally sound and the flight control system being functional. More broadly, a complex asset's potential may depend on a combination of mechanical, electronic, and even human components — for example, a military vehicle requires an operational engine *and* a functioning transmission *and* a licensed, competent crew member.

Information to be exchanged:

- *"Fighter XYZ-123's potential to fly at Mach 2 requires both EJ200 engines (EJ200-001 and EJ200-002) to be in an installed, operational state."*
- *"Fighter XYZ-123's potential to fly at Mach 2 also requires the airframe and flight control system to be operational."*
- *"If only one EJ200 engine is operational, Fighter XYZ-123 can fly but not at Mach 2."*

The logical structure of these dependencies can take several forms: the whole may require all parts of a given type to be operational, at least m of n parts, or exactly n parts. Dependencies may also combine across different part types — for example, both engines operational *and* the flight control system functional.

3.6 Cross-Entity Dependencies

[Aspect 2 – designs and plans]

Dependencies can extend beyond a single asset's internal composition. An entity's potential may depend on the potential of an entirely separate, external entity — one that does not share its spacetime and is not one of its parts. Unlike the whole-part dependencies of §3.5, these are inter-entity dependencies: they link independent entities whose lifecycles and spatial extents are distinct. The potential at any node in such a chain depends on the potentials of the nodes it depends upon, and the loss of potential at any point propagates through to the dependent entities. The model must support chains of arbitrary depth.

Scenario. A Eurofighter's potential to conduct extended-range operations depends on the availability of a Voyager tanker aircraft with the potential to deliver fuel via a compatible boom or drogue system. If the Voyager's aerial refuelling system is unserviceable, the Eurofighter's extended-range potential is lost — even though the fighter itself is fully operational. The dependency chain runs: Eurofighter extended-range operations → Voyager tanker availability → Voyager refuelling system serviceability.

Information to be exchanged:

- *"Fighter XYZ-123's potential for extended-range operations depends on a Voyager tanker having the potential to deliver fuel via aerial refuelling."*
- *"The Voyager's aerial refuelling potential depends on its boom/drogue system being serviceable."*
- *"If the Voyager's refuelling system is unserviceable, Fighter XYZ-123's extended-range potential is unavailable, regardless of the fighter's own condition."*

The distinction between §3.5 and §3.6 is important for encoding. Whole-part dependencies (§3.5) follow the mereological structure: parts occupy a subset of the whole's spacetime, and the dependency can be grounded in the *isPartOf* relationship. Cross-entity dependencies link independent entities that do not share spacetime; they require a different mechanism that expresses temporal co-occurrence — the dependent entity can only exercise its potential during periods when the entity it depends upon is in the required state.

3.7 Measured and Quantified Potentials

This requirement is included for completeness; the encoding of measured and quantified potentials is not addressed in §5 and is identified as future work in §7.

The preceding requirements treat potentials as qualitative assertions: an entity either has a potential or it does not. In practice, many potentials have measurable characteristics — a maximum speed, a load capacity, a range, a throughput rate. The requirement is to record not just *that* an entity has a potential, but *how much* of it.

Scenario. A Eurofighter's potential to carry external stores is not simply "can carry stores" — it is quantified: the aircraft can carry a maximum external payload of 6,500 kg across its thirteen

hardpoints. Similarly, a Voyager tanker’s refuelling potential is not simply “can refuel other aircraft” — it can transfer fuel at a rate of up to 3,600 kg per minute via its boom system.

At the class level, the Eurofighter type has a design payload capacity of 6,500 kg. At the individual level, Fighter XYZ-123 may have a reduced capacity of 5,000 kg due to a structural limitation identified during inspection.

Information to be exchanged:

- *“Eurofighters can carry an external payload of up to 6,500 kg.”*
- *“Fighter XYZ-123 can carry an external payload of up to 5,000 kg (reduced due to structural limitation).”*
- *“The Voyager tanker can transfer fuel at up to 3,600 kg per minute via its boom system.”*

This requirement introduces the association of measurable quantities with potentials, including units of measure and the distinction between design-level (class) and assessed (individual) values. It also raises the question of how quantified potentials interact with the dependency structures described in §3.5 and §3.6 — for example, a degraded engine state might reduce the maximum speed potential from Mach 2 to Mach 1.2 rather than eliminating it entirely.

4 Background: Modelling Potential (“Can Do”)

4.1 The core challenge

The IES is a four-dimensional (4D) extensional ontology. An Entity in IES — a person, an aircraft, a building — is a four-dimensional object extended through both space and time. Identity is determined by spatio-temporal extent: two things that occupy exactly the same region of spacetime are, by definition, the same thing. Changes to an Entity are not modelled as updates to its properties but as distinct temporal parts called States. A State is a temporal slice of an Entity during which some property holds — “Fred while employed at Acme Ltd” or “Fighter XYZ-123 while flying at Mach 2.”

As §2 has described, the next major version of IES simplifies this hierarchy by collapsing Entity and State into a single foundational class (`ies_top:State`). The Turtle examples throughout this document use IES v5 syntax — in which Entity and State remain separate classes — but the reader should bear in mind that this distinction is being eliminated. §2.5 provides a possible migration mapping.

This extensional commitment creates a fundamental difficulty when we try to record what something *can* do, as opposed to what it *actually does*. The statement “Fighter XYZ-123 can fly at Mach 2” is not a claim about any actual State that exists in the world. It is a modal claim — a claim about *possibility*. Fighter XYZ-123 may never have flown at Mach 2, and may never do so.

There need be no spatio-temporal extent corresponding to “Fighter XYZ-123 flying at Mach 2” in the actual world.

Yet organisations plainly need to record such potential. As §3 illustrates, defence planners need to know what an aircraft *can* do; emergency planners need to know what a building *could* serve as. An extensional ontology that cannot express possibility would be unable to meet these requirements.

IES already contains one concept that touches on modal reasoning: `ies:PossibleWorld`, defined as “an Element that encompasses a number of Events, Entities and States that may occur / have occurred.” Its basic mechanism is straightforward: Elements are grouped into a `PossibleWorld` via `ies:isPartOf` to assert that they share the same hypothetical truth. A disposition that has not been actualised — such as a State in which a location is operating as an emergency water distribution point — can be represented as a State instance that is `partOf` a `PossibleWorld`. This records the modal claim without asserting that the State has actually occurred. `PossibleWorld` provides the building block of possibility within IES, though the IES specification acknowledges it as “a very simple placeholder” that is likely to grow in later versions. Both the current and proposed encoding mechanisms can work alongside `PossibleWorld`; neither depends on it, but both are compatible with its use for conditional or hypothetical scenarios.

The IES already has a mechanism for recording dispositions — the set of classes (`DispositionalClass`, `Capability`, `Tendency`) and the properties `isDisposedTo` and `allHaveDisposition` that together form the current encoding. This document examines that existing mechanism, identifies its limitations — particularly around dependency relationships — and proposes six new properties that extend and complement it. Both the existing encoding and the proposed extension use the same meta-classes and the same set-theoretic foundation. Where they differ is in *how* the relationship between an Element and its dispositions is expressed in RDF: the current encoding uses class membership (via `rdf:type` and `rdfs:subClassOf`), while the proposed extension uses dedicated properties (`hasDisposition` and `eachHasDisposition`) with `State/ClassOfState` semantics. The remainder of this section establishes the shared theoretical ground; §5 presents both the existing encoding and the proposed extension in detail.

There is a natural question about the relationship between the two aspects. If an Aspect 1 assertion says that $Sx \cap R \neq \emptyset$ — that there exists some possible world in which entity x has a state of type R — does that possible world necessarily contain the compositional conditions that Aspect 2 describes? In one sense, yes: any possible world that contains a Eurofighter flying at Mach 2 also contains that aircraft with its engines operational. The Aspect 2 constraints are, in the possible-worlds framing, part of the *content* of that possible world. But this does not mean that an Aspect 1 assertion *encodes* those constraints, or that they can be *read off* from it. The set-theoretic intersection $Sx \cap R \neq \emptyset$ asserts that the overlap is non-empty; it says nothing about what else must be true in the worlds where the overlap obtains. Aspect 2 is concerned precisely with making those structural conditions explicit — as machine-readable assertions

that a consuming system can evaluate, query, or reason over, independently of the modal claim.

This is why the two aspects require different mechanisms. Aspect 1 is naturally served by a property that asserts modal possibility (§5.1, §5.2). Aspect 2 requires a mechanism that asserts structural co-occurrence conditions — what parts must be present, in what states, for the disposition to be exercisable (§5.3). Encoding the former does not implicitly encode the latter, and conflating the two would overload the disposition properties with semantics they are not designed to carry.

4.2 Set theory and possible worlds

The bridge between extensional identity and modal possibility is built from two formal tools: possible-worlds semantics and set theory. Both encoding mechanisms rely on the same set-theoretic foundation, which this section presents in pattern-neutral terms.

Individual-level disposition

Consider an individual Element x — say, Fighter XYZ-123 in its current operational condition. In the actual world, x has a collection of States: periods of maintenance, periods of flight, periods on the ground. But we can also consider the States that x *could* have across all possible worlds — every State that is consistent with x 's nature, including States that may never be actualised. Call this set S_x :

$$S_x = \{ s \mid s \text{ isStateOf } x \text{ in some possible world } w \}$$

Now consider a ClassOfElement R — a class whose members are Elements sharing some property. For example, R might be “*performing a Mach 2 flight*”, a class of Elements that are in a state of flying at Mach 2. Every Element that is an actual instance of Mach 2 flight is a member of R , but so too are all the *possible* instances across all possible worlds.

To say that x has the disposition R is to say that S_x and R overlap — that there exists at least one possible world in which x has a State that is a member of R :

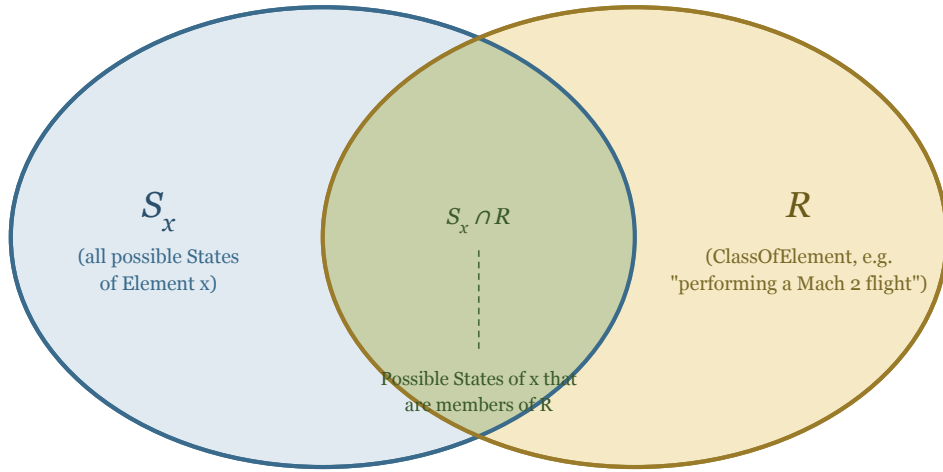
$$S_x \cap R \neq \emptyset$$

In plain English: “there is some possible State of x that would count as performing a Mach 2 flight.” Equivalently, using logical notation:

$$\exists s \mid (s \text{ isStateOf } x) \wedge (s \in R)$$

This does not assert that x has ever actually flown at Mach 2 — only that such a State is possible.

Individual-level disposition: $S_x \cap R \neq \emptyset$



The disposition holds iff this intersection is non-empty: $S_x \cap R \neq \emptyset$

Pattern-neutral: illustrates the shared set-theoretic foundation without showing how either encoding mechanism represents it in RDF.

Figure 1 Individual-level dispositions: $S_x \cap R \neq \emptyset$

Class-level disposition

The requirements in §3 also demand assertions at the class level: “every Eurofighter can fly at Mach 2.” Let X be a `ClassOfElement` whose members are Eurofighters in operational condition (e.g. “Eurofighter in operational condition”), and let R be the same `ClassOfElement` as above. The class-level assertion is a universally quantified version of the individual case:

$$\forall x \in X, Sx \cap R \neq \emptyset$$

That is: for every member of X , there exists at least one possible world in which that member has a State that is a member of R . In everyday language: “each Eurofighter in operational condition is capable of performing a Mach 2 flight.” Using a shorthand property for readability:

$$\forall x \in X (x \text{ hasDisposition } R)$$

A note on notation. The property name `hasDisposition` is used here for readability. The formal semantics — the non-empty intersection of possible States with a `ClassOfElement` — are common ground shared by both encoding mechanisms. Both assert this same set-theoretic relationship; they differ in *how* that assertion is expressed in RDF. The current mechanism expresses it through class membership (`rdf:type` and `rdfs:subClassOf`), while the proposed mechanism expresses it through a dedicated property. The name `hasDisposition` is also the proposed property name for the new mechanism (see §5.2), but in

this section it is used purely as a notation convenience — the formal semantics it stands for are pattern-neutral.

Note that this set-theoretic foundation uses `Element` and `ClassOfElement` generically — the formalism is independent of whether the encoding targets `Element` or `State`. The proposed extension in §5.2 narrows the domain and range to `State/ClassOfState` specifically, following the architectural motivation described in §2.4, but the underlying set-theoretic relationship is the same.

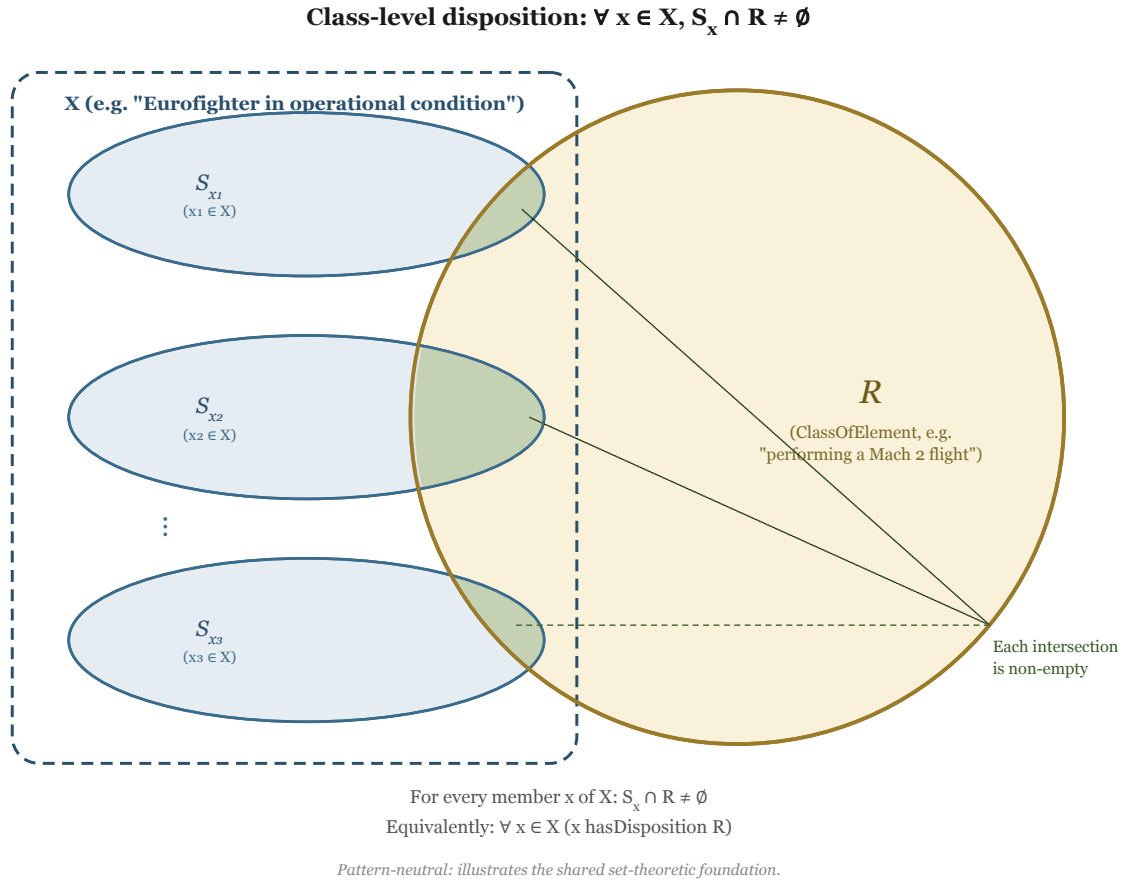


Figure 2 Class-level disposition $\forall x \in X, S_x \cap R \neq \emptyset$

4.3 The type–token distinction in IES

The requirements in §3 operate at two levels: individual assertions (“Fighter XYZ-123 can fly at Mach 2”) and class-level assertions (“every Eurofighter can fly at Mach 2”). IES has a well-developed apparatus for working at both levels, which both encoding mechanisms exploit.

At the individual level, IES models particular things as `Elements`. Both `Entity` and `State` are subclasses of `Element`. An `Entity` such as Fighter XYZ-123 represents the whole-life extent of a thing — its entire existence through space and time. A `State` such as “Fighter XYZ-123 during

2025” represents a temporal part of that Entity — a bounded period during which some property holds. The `isStateOf` relationship links a State to the Element it is a temporal part of. Note that while both Entity and State are subclasses of Element, Entity is *not* a subclass of State — the two are distinct branches of the Element hierarchy. Crucially, however, every subclass of Entity is also a subclass of the corresponding State class: `ies:Person` is a subclass of `ies:PersonState`, `ies:Vehicle` is a subclass of `ies:VehicleState`, and so on throughout the hierarchy. This reflects the IES 4D commitment that the whole-life extent of an Entity is itself a (maximal) temporal part — the State that spans the Entity’s entire existence. The consequence is that properties typically associated with States, including disposition assertions, can also be applied to whole-life Entities. (As noted in §2.1, the Entity/State distinction described here is a feature of the v5 hierarchy used in this document’s worked examples; it is eliminated in the Top and Core architecture.)

At the type level, IES uses `ClassOfElement` and its subclasses (`ClassOfEntity`, `ClassOfState`, and so on) to model classes of things. The powertype relationship connects these levels: `ClassOfEntity` is the powertype of Entity, meaning that every subclass of Entity is an instance of `ClassOfEntity`. This “pushing up” of type levels is central to IES’s ability to make statements about classes of things, not just individual instances. In the Top and Core architecture, `ClassOfX` becomes `SetOfX` (e.g. `SetOfStates` replaces `ClassOfState`), but the structural role — and the powertype mechanism — remain the same.

Both encoding mechanisms must provide assertions at both levels. An individual-level disposition assertion relates a specific Element (whether a temporal State or a whole-life Entity) to a `ClassOfElement`. A class-level disposition assertion relates a `ClassOfElement` to another `ClassOfElement`, asserting that every member of the first class has the specified disposition. The formal apparatus is the same in both cases — it is the set-theoretic relationship defined in §4.2 — but the granularity of the assertion differs.

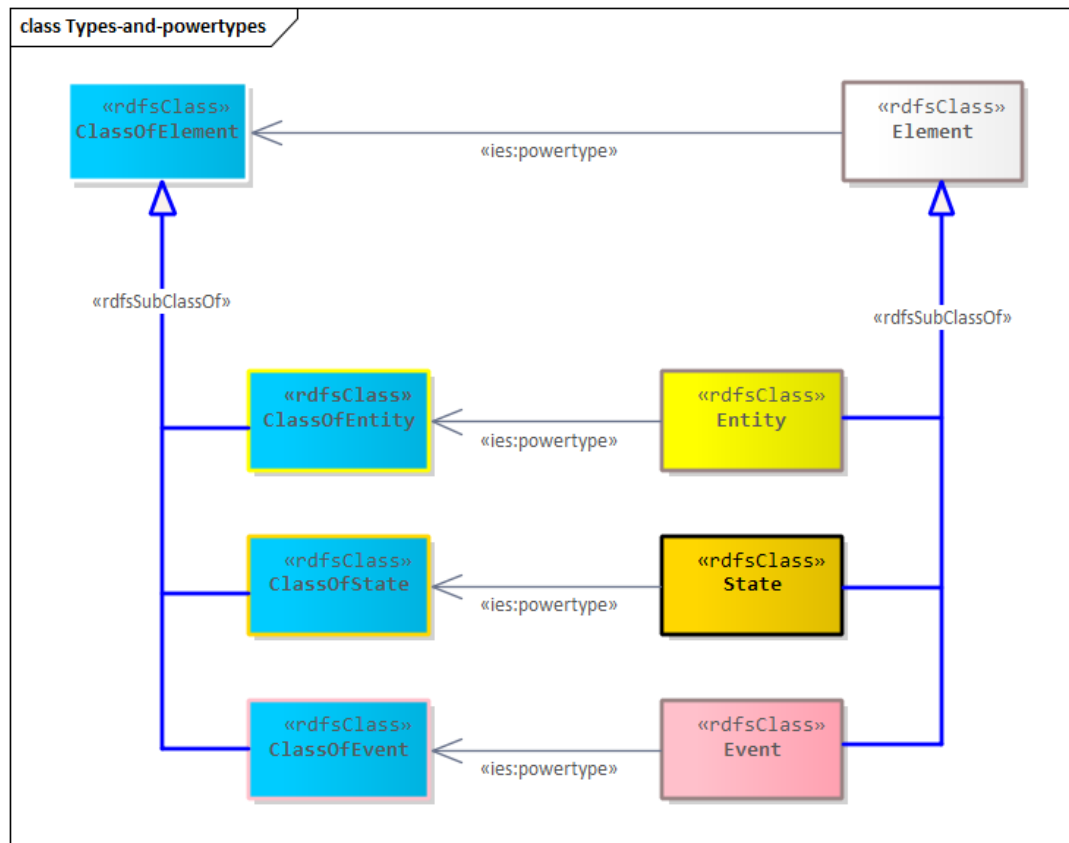


Figure 3 Types and Power Types in IES

4.4 Where the two encodings diverge

Both encodings express the same underlying semantics — the non-empty intersection of possible Elements with a ClassOfElement — and both use the same meta-classes: DispositionalClass as the parent class, with Capability and Tendency as its two sub-types. The ClassOfElement instances that appear in the range of disposition assertions are shared across both encodings. Where they diverge is in the RDF mechanism used to encode the relationship between an Element (or class of Elements) and those ClassOfElement instances.

The current encoding employs class membership. To say that Fighter XYZ-123 can fly at Mach 2, it makes the aircraft (or a State of the aircraft) a *member of* a DispositionalClass — specifically, a Capability — using `isDisposedTo` (a sub-property of `rdf:type`) or `allHaveDisposition` (a sub-property of `rdfs:subClassOf`). The modal semantics (“can”) are encoded within the definition of the class rather than in the relationship itself. Because the relationship is expressed through class membership, the semantics of `rdf:type` and `rdfs:subClassOf` — including transitivity and inheritance — are implicitly brought to bear. The current encoding targets Element/ClassOfElement in its domain and range declarations.

The proposed extension employs dedicated properties. To make the same assertion, it uses an explicit relationship — `hasDisposition` or `eachHasDisposition` — between the `State` and a `ClassOfState`. These properties are sub-properties of `ies:relationship` only, not of `rdf:type` or `rdfs:subClassOf`. The modal semantics are expressed by the property itself rather than by the class definition, and no class-membership inference is entailed. As motivated in §2.4, the proposed properties target `State/ClassOfState` specifically, aligning with the architectural direction in which every concrete individual is a `State`.

Both encodings can be read as expressing the same set-theoretic relationship, but only under the Interpretation 2 reading of the existing properties. The current encoding handles the core disposition requirements (§3.1–§3.4) effectively; the proposed extension additionally provides dedicated mechanisms for the dependency requirements (§3.5–§3.6) that the current encoding does not directly express. §5 presents the existing encoding first (§5.1) and then introduces the proposed extension (§5.2), applying both to the full set of requirements with worked examples.

5 The Disposition Pattern

This section presents the existing encoding and the proposed refinements, organised by the two-aspects framework from §1.3.

5.1 The Current Encoding (Aspect 1 Requirements)

5.1.1 The Meta-Class Infrastructure

The IES provides a family of meta-classes for modelling potential — what things *can* do or be. These meta-classes are shared infrastructure used by both the current encoding and the new properties described in §5.2. The parent class is `ies:DispositionalClass`, a subclass of `ies:ClassOfElement`, defined as:

A ClassOfElement whose instances all share the same disposition — e.g. Capability or Tendency.

Two specialisations refine the concept by intent. `ies:Capability` is a `DispositionalClass` where all instances share the same capability (e.g. “*Vehicles capable of Mach 2*”). `ies:Tendency` is a `DispositionalClass` where all instances share the same tendency (e.g. “*People who tend to violence*”). These meta-classes — `DispositionalClass`, `Capability`, and `Tendency` — define the *kinds* of dispositional grouping available. They do not, by themselves, determine how an `Element` or `ClassOfElement` is related to a `DispositionalClass`; that is the role of the encoding properties.

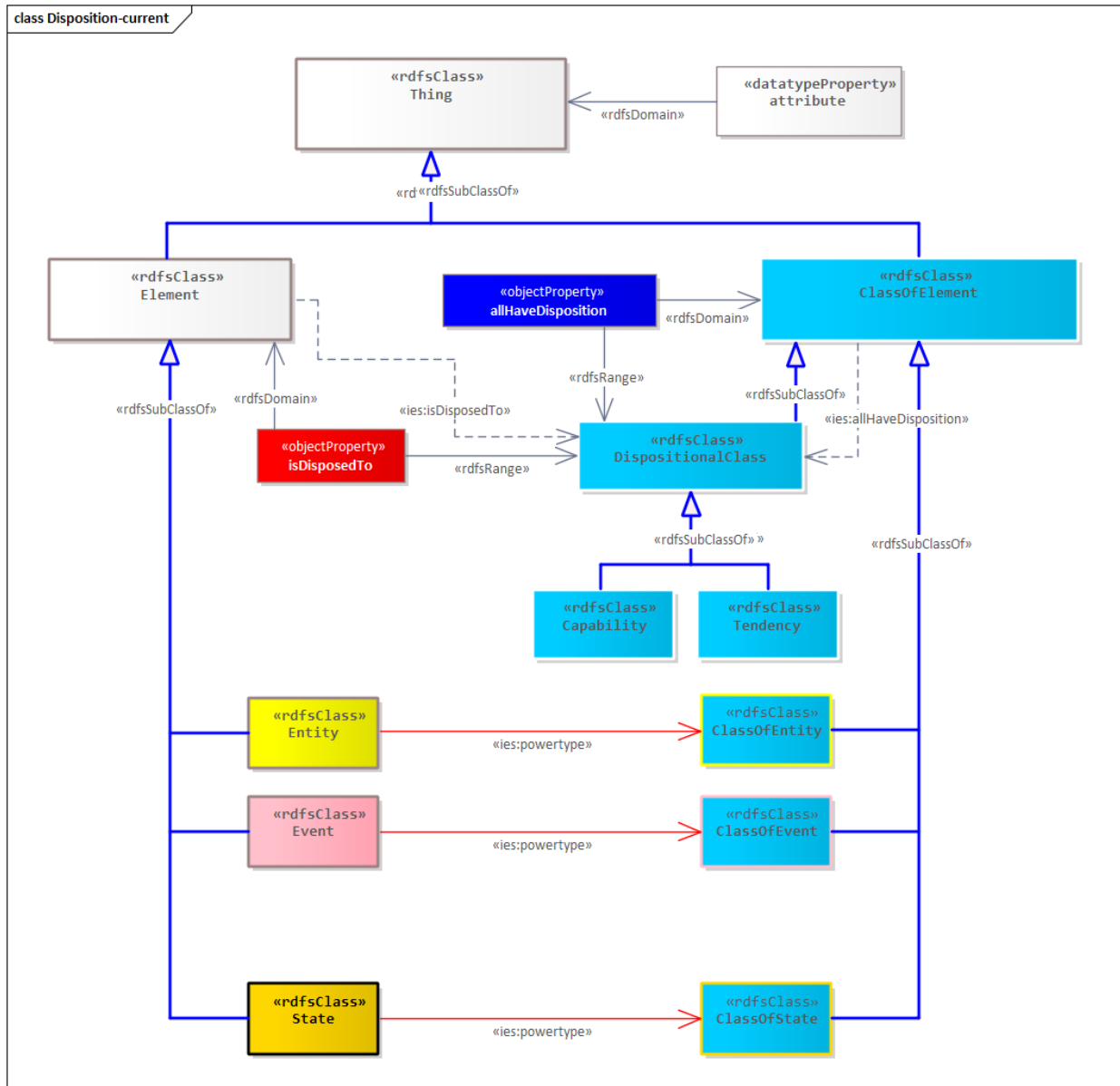


Figure 4 Disposition pattern in IES

What characterises the current encoding is the pair of properties that connect Elements and ClassOfElements to DispositionalClasses (see UML Class diagram above):

`ies:isDisposedTo` is an `owl:ObjectProperty` with domain `ies:Element` and range `ies:DispositionalClass`, documented as “an *rdf:type* relationship that asserts an *Element* is a member of a *DispositionalClass*.” It is declared as a sub-property of both `ies:relationship` and `rdf:type`.

`ies:allHaveDisposition` is an `owl:ObjectProperty` with range `ies:DispositionalClass`, documented as “an *rdfs:subClassOf* relationship that asserts that all instances of a

ClassOfElement share a disposition.” It is declared as a sub-property of both `ies:relationship` and `rdfs:subClassOf`.

The defining characteristic of the current encoding is precisely these sub-property declarations. By making `isDisposedTo` a sub-property of `rdf:type`, and `allHaveDisposition` a sub-property of `rdfs:subClassOf`, the current pattern expresses dispositions through the mechanism of class membership. This design choice gives rise to two valid interpretations of how the modal semantics — the “can” in “Fighter XYZ-123 can fly at Mach 2” — are encoded.

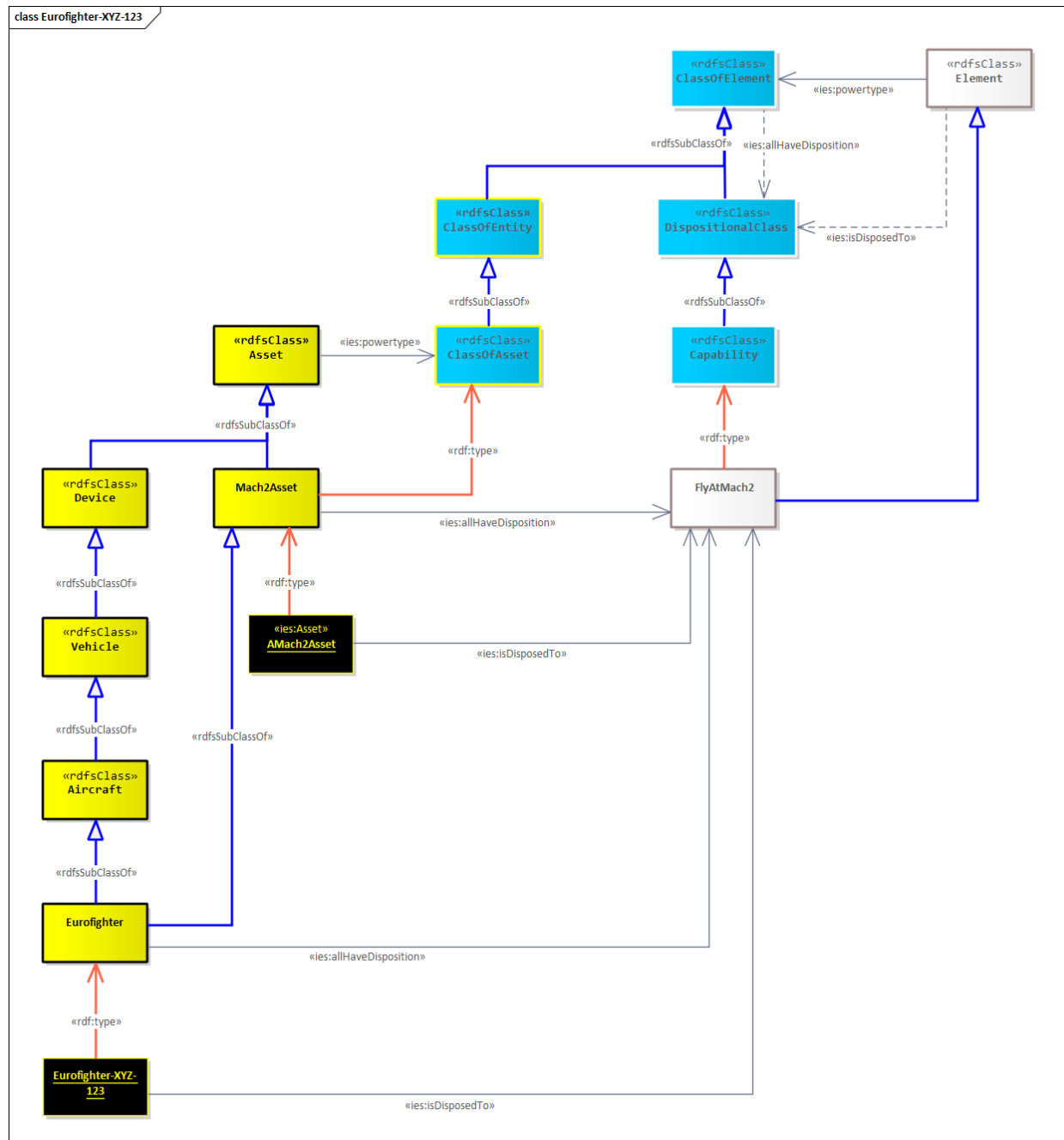


Figure 5 Applied at the class and individual levels

5.1.2 Two Interpretations

The two interpretations differ in where the semantic complexity resides.

Interpretation 1: The relationships are straightforward; the class is complicated.

Under this reading, `isDisposedTo` and `allHaveDisposition` are genuinely specialisations of `rdf:type` and `rdfs:subClassOf` respectively — that is, they carry no additional semantics beyond what those standard RDF/RDFS relationships already provide. Asserting `x isDisposedTo D` is equivalent to asserting `x rdf:type D`: it says that `x` is a member of the class `D`.

The consequence is that the `DispositionalClass D` must be defined so that its membership criteria encode the modal semantics. In the running example, the `Capability` “Aeroplane that can fly at Mach 2” is not simply the class of aeroplanes currently flying at Mach 2. Rather, it is the class of aeroplanes for which there exists at least one possible state of flying at Mach 2. This class has what we may call a *complicated definition*: its membership criteria go beyond what `rdf:type` and `rdfs:subClassOf` ordinarily convey. Standard class membership says “this thing *is* a member”; membership of this `Capability` says “this thing *can be* in states of a certain kind.” The modal possibility (“can”) is embedded in the class definition itself.

Interpretation 2: The class is straightforward; the relationships are complicated.

Under this reading, the `Capability` has a straightforward extensional definition — it is simply the class of states of a particular kind (e.g. states of flying at Mach 2). The modal semantics (“can”) are not part of the class definition but are carried by the relationships `isDisposedTo` and `allHaveDisposition`.

In this case, `isDisposedTo` and `allHaveDisposition` assert not merely class membership or subclass relationship, but that the entity *can* enter into states of the specified kind. The relationships themselves are “complicated” — they cannot be reduced to `rdf:type` and `rdfs:subClassOf` without loss of meaning. This is a central tension in the current encoding: if these relationships truly *are* specialisations of `rdf:type` and `rdfs:subClassOf` (as declared in the ontology via `rdfs:subPropertyOf`), then a reasoner will treat them as such, and the additional modal semantics must be conveyed by some other means. A sub-property inherits the semantics of its super-property; it cannot *alter* them. The modal meaning (“can have states of this class”) is not entailed by the standard semantics of `rdf:type` (“is a member of this class”).

Both interpretations are consistent with the IES documentation and the data exchanged. They produce the same triples in practice. They differ in where a reader (or reasoner) must look to find the modal semantics: in the definition of the `DispositionalClass` itself (Interpretation 1) or in the semantics of the relationship (Interpretation 2).

5.1.3 Set-Theoretic Description

Using the notation established in §4, the two interpretations can be expressed formally.

Let R be a class whose members are states of a particular kind — for example, the class of states of flying at Mach 2. Let S_x denote the set of all possible states of an Element x (that is, the set $\{s \mid s \text{ isStateOf } x\}$ across all possible worlds). Let D be a Capability — for example, “Aeroplane that can fly at Mach 2.” Let X be a ClassOfElement — for example, “Eurofighter.”

Interpretation 1: Complicated class, simple relationships

Under Interpretation 1, the Capability D is defined extensionally as:

$$D = \{x \in \text{Element} \mid S_x \cap R \neq \emptyset\}$$

That is, D is the set of all Elements that have at least one possible state in R . Asserting:

$x \text{ isDisposedTo } D$

is asserting $x \in D$, which is semantically identical to $x \text{ rdf:type } D$. Asserting:

$X \text{ allHaveDisposition } D$

is asserting $X \subseteq D$, which is semantically identical to $X \text{ rdfs:subClassOf } D$.

The modal semantics (“can”) are embedded entirely within the definition of D . The relationships isDisposedTo and $\text{allHaveDisposition}$ are standard set-membership and subset assertions respectively.

Interpretation 2: Simple class, complicated relationships

Under Interpretation 2, the relationships isDisposedTo and $\text{allHaveDisposition}$ are not reducible to rdf:type and rdfs:subClassOf . Instead, they directly assert the set-theoretic intersection. Asserting:

$x \text{ isDisposedTo } R$

means $S_x \cap R \neq \emptyset$ — that is, the set of possible states of x overlaps with R . Asserting:

$X \text{ allHaveDisposition } R$

means $\forall y \in X: S_y \cap R \neq \emptyset$ — that is, every member of X has at least one possible state in R .

The modal semantics are carried by the relationships. The range (R) has a straightforward definition as a class of states of a particular kind.

Equivalence

Both interpretations express the same underlying fact: that an entity (or every member of a class) has the potential to enter into states of a specified kind. They differ only in where the complexity is located — in the class definition or in the relationship semantics. This is significant for implementation: Interpretation 1 places the burden on anyone who needs to reason about

the *definition* of the Capability, whilst Interpretation 2 places it on anyone who needs to reason about the *meaning* of the relationship.

Recommended reading. This document adopts Interpretation 2 throughout. Under Interpretation 2, the Capability has a straightforward extensional definition as a class of states of a particular kind, and the modal semantics (“can”) are carried by the relationship. This is the more natural reading when the pattern is extended with the six new properties in §5.2, since those properties are explicitly designed to carry the modal semantics in the property rather than the class definition. Interpretation 1 remains valid; the choice does not affect the triples exchanged.

5.1.4 Narrowing to State/ClassOfState Semantics

Following the architectural direction established in §2, all Capability instances used in this document additionally specify `rdfs:subClassOf ies:State`. For example:

```
ex:FlyAtMach2 a ies:Capability ;  
  rdfs:label "Fly at Mach 2"@en-GB ;  
  rdfs:subClassOf ies:State .
```

This declaration narrows the Capability to ClassOfState semantics: its members are States — temporal parts of Entities during which the disposition is exercised. The `ies:Capability` meta-class is a subclass of `ies:DispositionalClass`, which is in turn a subclass of `ies:ClassOfElement`. Since `ies:ClassOfState` is also a subclass of `ies:ClassOfElement`, a Capability instance that additionally declares `rdfs:subClassOf ies:State` is both a Capability and a ClassOfState. This dual classification is legitimate and consistent with IES’s class hierarchy.

The practical consequence is that the domain of `isDisposedTo` (which is `ies:Element`) and `allHaveDisposition` (which is implicitly `ies:ClassOfElement`) are narrowed in practice to Entity/State and ClassOfEntity/ClassOfState respectively. Events — which are not subclasses of State in IES v5 — do not satisfy the narrowed range. This is a deliberate choice motivated by the architectural direction in §2: under Top and Core, Activities (the successor to Events) will be States and the exclusion will be resolved. No use cases for Event-bearing dispositions have been identified under v5.

5.1.5 Applying to the Requirements

The following subsections work through each of the requirements from §3, showing how the current encoding records the information.

All Turtle examples use the following prefixes:

```
@prefix ies: <http://informationexchangestandard.org/ont/ies/common/> .  
@prefix ex: <http://example.com/ontology#> .  
@prefix data: <http://example.com/data#> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .  
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
```

§3.1 Non-specific assets with potential

Requirement recap: In early mission planning, we need a class of thing that can fly at Mach 2, without yet knowing about “Eurofighter” or any specific aircraft type.

The current encoding addresses this by combining the `Capability` with a non-specific `ClassOfElement` representing the planning concept — “some type of asset that can fly at Mach 2.” The `Capability` defines the potential; the `ClassOfElement` represents the as-yet-unresolved class of asset that possesses it:

```
# The Capability expressing the potential
ex:FlyAtMach2 a ies:Capability ;
    rdfs:label "Fly at Mach 2"@en-GB ;
    rdfs:subClassOf ies:State .

# A non-specific class of asset with this potential
ex:FlyAtMach2Asset a ies:ClassOfElement ;
    rdfs:label "Asset that can fly at Mach 2"@en-GB ;
    ies:allHaveDisposition ex:FlyAtMach2 .
```

Note. The explicit class `ex:FlyAtMach2Asset` is a convenience, not a necessity. The `Capability` `ex:FlyAtMach2` already implicitly defines the set of entities disposed to it — those entities x for which $S_x \cap \text{ex:FlyAtMach2} \neq \emptyset$. A consumer can query for all entities or classes bearing this disposition without a named asset class:

```
sparql
# Individual assets with this disposition
SELECT ?asset WHERE { ?asset ies:isDisposedTo ex:FlyAtMach2 . }

# Classes whose members all have this disposition
SELECT ?class WHERE { ?class ies:allHaveDisposition ex:FlyAtMach2 . }
```

A named asset class becomes useful when additional constraints need to be bundled together as a reusable planning concept — for example, an asset type defined by a combination of Mach 2 flight, a minimum combat radius, and a particular payload capacity. For simple “I need something that can do X” requirements, the `Capability` alone suffices.

The non-specific asset class `ex:FlyAtMach2Asset` can now be referenced in planning data without any commitment to which specific platform — Eurofighter, F-35, or otherwise — will fill the role. Once the planning team selects a platform, that platform’s `ClassOfElement` (e.g. `ex:Eurofighter`) can be related to `ex:FlyAtMach2Asset` via `rdfs:subClassOf` or can independently assert its own `allHaveDisposition` relationship to `ex:FlyAtMach2`.

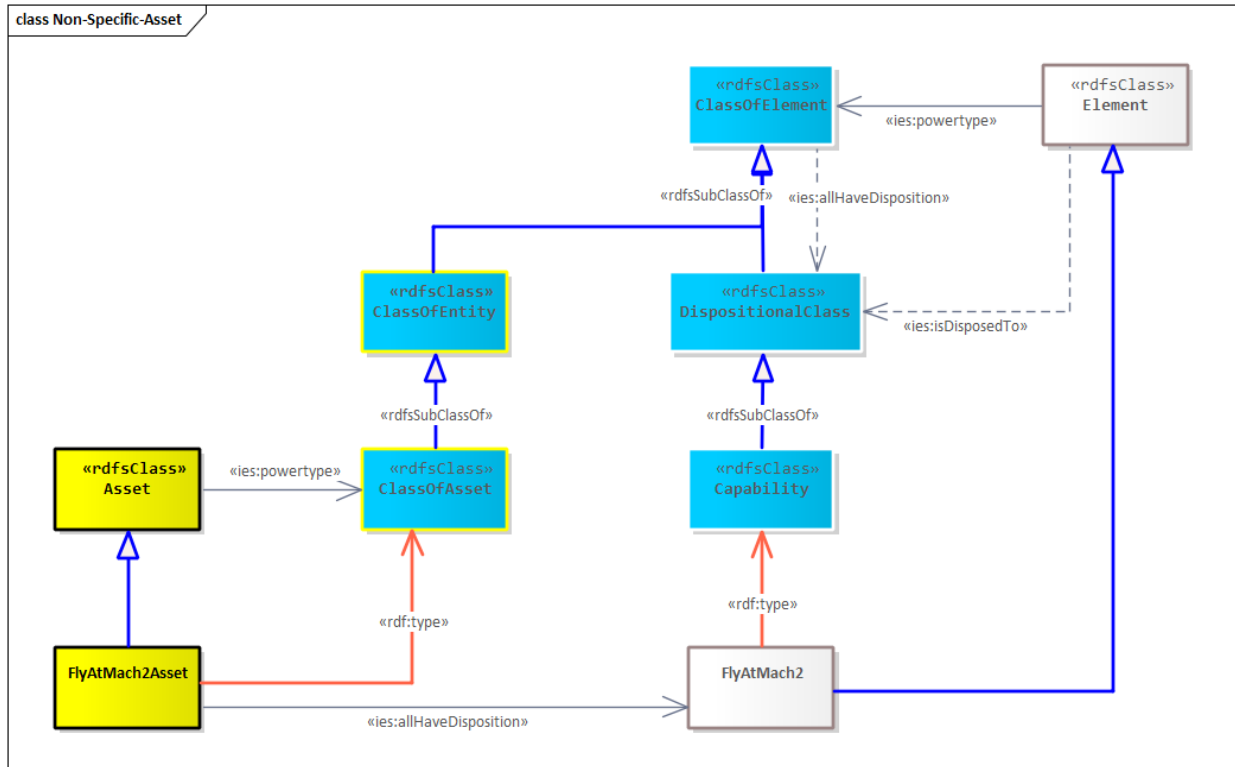
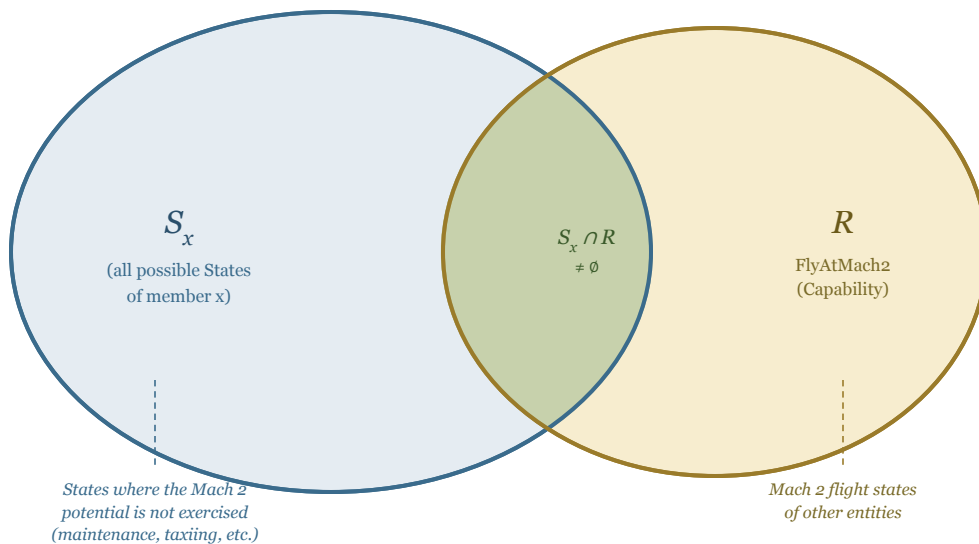


Figure 6 Non-specific asset class

States of a FlyAtMach2Asset member



The allHaveDisposition assertion guarantees $S_x \cap R \neq \emptyset$ for every member x of FlyAtMach2Asset, but does not require that all of x 's states are Mach 2 flight states.

Figure 7 A FlyAtMach2Asset can fly at Mach 2

The Capability and the non-specific asset class serve different roles. The Capability `ex:FlyAtMach2` is the dispositional grouping — the “type of potential.” The asset class `ex:FlyAtMach2Asset` is a planning concept — the “type of thing we need for this mission.” Since `allHaveDisposition` is a sub-property of `rdfs:subClassOf`, a reasoner will infer `ex:FlyAtMach2Asset rdfs:subClassOf ex:FlyAtMach2` — the non-specific asset class is nested inside the Capability in the class hierarchy. Under Interpretation 2 (adopted throughout this document), the modal semantics are additionally carried by the `allHaveDisposition` relationship itself, over and above its inherited `rdfs:subClassOf` behaviour. In neither case can the modal meaning be read off from the `rdfs:subClassOf` inference alone.

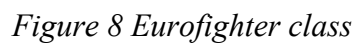
§3.2 Class-level disposition

Requirement recap: All Eurofighters can fly at Mach 2.

The current encoding records this using `allHaveDisposition` between the `ClassOfEntity` “Eurofighter” and the Capability:

```
ex:Eurofighter a ies:ClassOfEntity ;
  rdfs:label "Eurofighter"@en-GB ;
  rdfs:subClassOf ies:Aircraft .
```

Under Interpretation 2, the assertion carries the modal semantics directly: every member of the Eurofighter class can enter into states of flying at Mach 2. Since `allHaveDisposition` is declared as a sub-property of `rdfs:subClassOf`, a reasoner will also infer `ex:Eurofighter rdfs:subClassOf ex:FlyAtMach2`.



Requirement recap: Fighter XYZ-123 can fly at Mach 2.

The current encoding records this using `isDisposedTo` between the individual and the `Capability`:

```
data:FighterXYZ-123 a ex:Eurofighter ;  
  rdfs:label "Fighter XYZ-123"@en-GB .
```

```
data:FighterXYZ-123 ies:isDisposedTo ex:FlyAtMach2 .
```

Under Interpretation 2, the assertion directly records the modal relationship: Fighter XYZ-123 can have states of flying at Mach 2. Since `isDisposedTo` is a sub-property of `rdf:type`, a reasoner will also infer `data:FighterXYZ-123 rdf:type ex:FlyAtMach2`.

Where the class-level assertion from §3.2 is also present, the individual-level assertion is entailed (since Fighter XYZ-123 is a Eurofighter, and all Eurofighters have this disposition). An implementation may choose to assert both for clarity, or to rely on the class-level assertion alone.

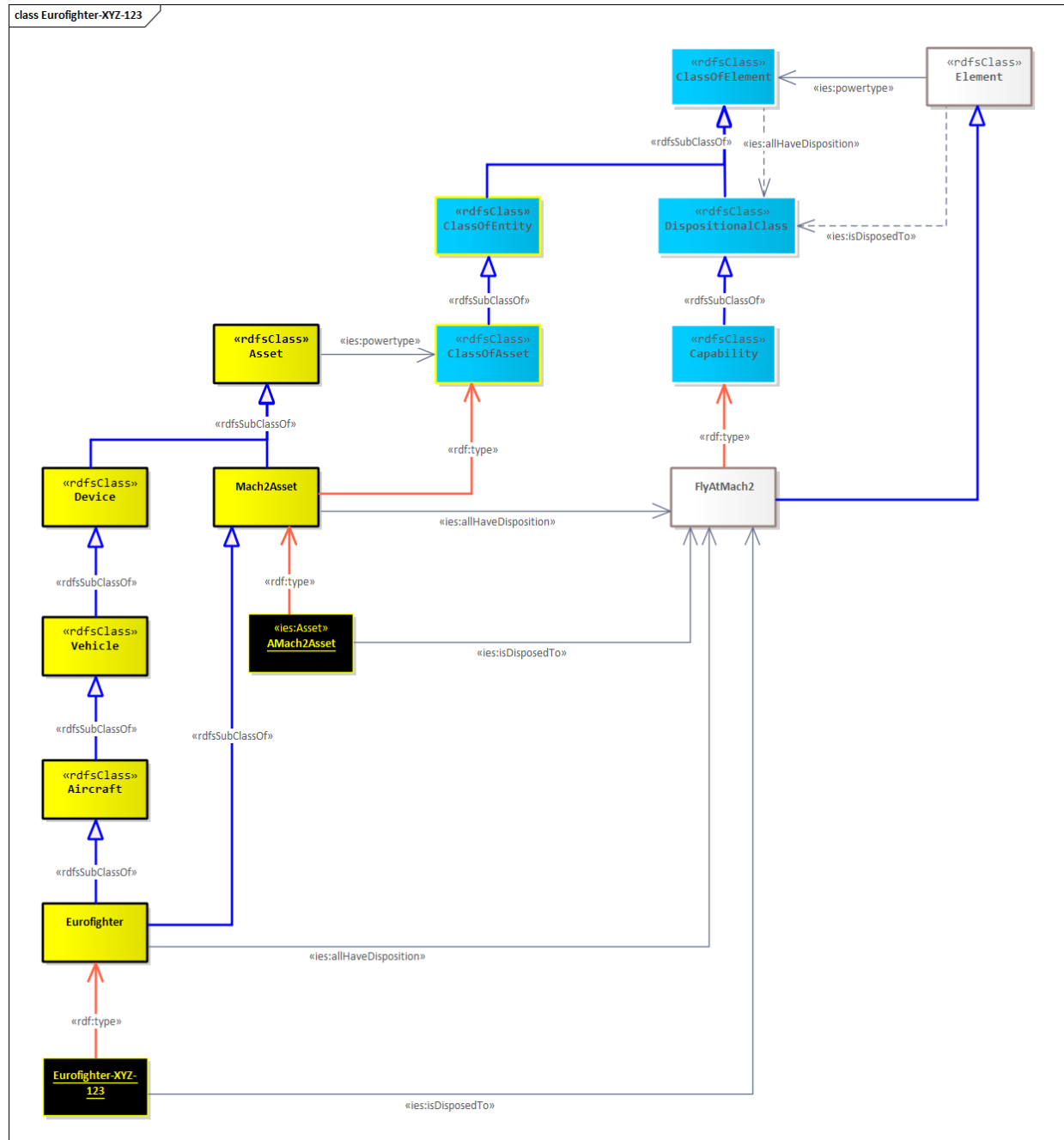


Figure 9 Individual Eurofighter XYZ-123 can fly at Mach 2

§3.4 Permitted and prohibited uses

Requirement recap: Mach 2 flight is permitted only when in operational condition. Low-altitude supersonic flight over populated areas is prohibited.

Permitted uses with preconditions

The condition “*only when in operational condition*” enriches the basic disposition with a precondition. The current IES specification provides relatively little guidance on how `DispositionalClass` and its sub-classes should be used in practice for conditional dispositions. This section explores two plausible approaches, with a view to seeking community guidance on which should be recommended.

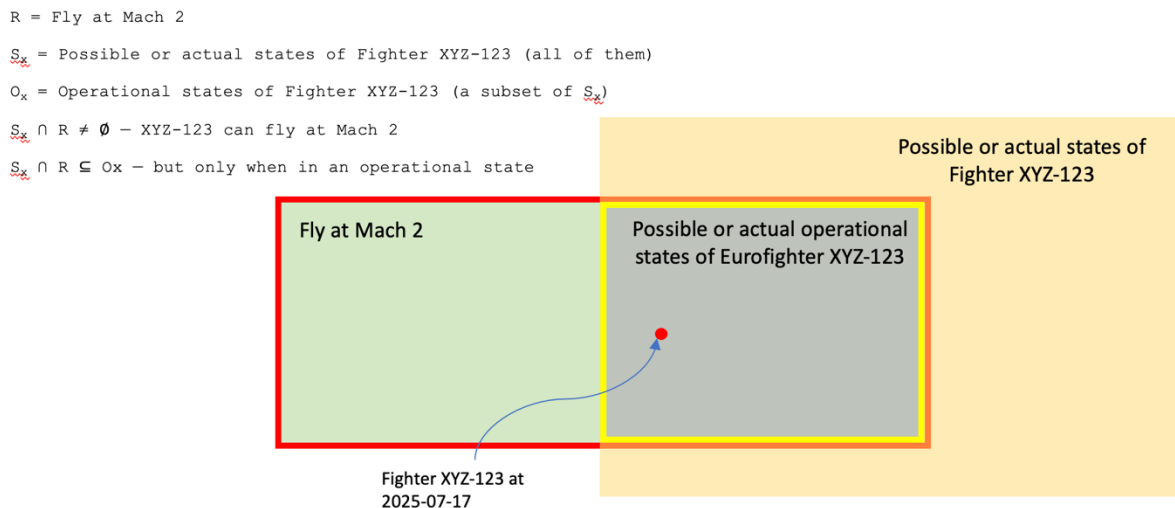


Figure 10 A particular Eurofighter can fly at Mach 2, if in an operational state

Approach 1: Bundle the condition into the Capability.

Under this approach, the precondition is encoded in the *definition* of the Capability:

```
# The simple Capability from §3.1
ex:FlyAtMach2 a ies:Capability ;
  rdfs:label "Fly at Mach 2"@en-GB ;
  rdfs:subClassOf ies:State .

# A Capability that bundles the precondition into its definition
ex:FlyAtMach2InAPermittedUse a ies:Capability ;
  rdfs:label "Fly at Mach 2 in a permitted use"@en-GB ;
  rdfs:subClassOf ex:FlyAtMach2 .

# A ClassOfElement: Eurofighter states
ex:EurofighterState a ies:ClassOfEntity ;
  rdfs:label "Eurofighter state"@en-GB ;
  rdfs:subClassOf ex:Eurofighter ;
  ies:allHaveDisposition ex:FlyAtMach2InAPermittedUse .

# At individual level: a State of Fighter XYZ-123 during 2025
```

```
data:FighterXYZ-123-in-2025 a ex:EurofighterState ;
    ies:isStateOf data:FighterXYZ-123 ;
    ies:isDisposedTo ex:FlyAtMach2InAPermittedUse .
```

This approach is concise, but the *Capability* definition is doing heavy lifting. The compound condition — performance *and* permission *and* operational state — is implicit in the class name and label. There is no machine-readable structure connecting the *Capability* to the operational condition.

Approach 2: Use a conditional *ClassOfElement* linked to the *Capability*.

Under this approach, the *Capability* retains its simple definition. The precondition is instead encoded by defining a conditional *ClassOfElement* — a class of Eurofighter in a particular state — and attaching the disposition to that conditional class:

```
# The simple Capability from §3.1
ex:FlyAtMach2 a ies:Capability ;
    rdfs:label "Fly at Mach 2"@en-GB ;
    rdfs:subClassOf ies:State .

# A conditional ClassOfElement: Eurofighters in operational state
ex:EurofighterInOperationalState a ies:ClassOfEntity ;
    rdfs:label "Eurofighter in operational state"@en-GB ;
    rdfs:subClassOf ex:Eurofighter ;
    ies:allHaveDisposition ex:FlyAtMach2 .

# At individual level: a State of Fighter XYZ-123 during 2025
data:FighterXYZ-123-in-2025 a ex:EurofighterInOperationalState ;
    ies:isStateOf data:FighterXYZ-123 ;
    ies:isDisposedTo ex:FlyAtMach2 .
```

Here the precondition is structurally explicit. The conditional class `ex:EurofighterInOperationalState` represents the subset of Eurofighters in operational condition. The `allHaveDisposition` assertion is scoped to this conditional subset. At the individual level, a temporal State of Fighter XYZ-123 is typed as a member of the conditional class and asserted to have the disposition.

This approach is more compositional: the *Capability*, the operational condition, and their relationship are all represented as distinct, machine-readable elements. The remainder of this document assumes Approach 2, on the grounds that it keeps *Capabilities* compositional and encodes preconditions in machine-readable structure rather than human-readable labels. This is a recommendation for community discussion, not a definitive ruling.

Prohibitions

The requirement from §3.4 also includes a prohibition: low-altitude supersonic flight over populated areas is prohibited. This is a different kind of assertion — not “this entity can have states of a certain kind” but “this entity must not have states of a certain kind.”

The current encoding models what an entity *can* do (by asserting membership of or subclass relationship with a *Capability*). It does not directly model what an entity *cannot* or *must not* do. Open world semantics are relevant here: the absence of an *isDisposedTo* assertion does not entail that the disposition does not exist — it merely means that no sender has asserted it.

To explicitly record a prohibition, one would need either a complementary pattern (e.g. a negation mechanism) or an extension to the current vocabulary. The current IES specification does not define a pattern for explicit prohibitions. A plausible extension might introduce a property such as *isProhibitedFrom* with range *ies:DispositionalClass*, mirroring *isDisposedTo*:

```
# Define the prohibited activity as a DispositionalClass
ex:LowAltitudeSupersonicOverPopulatedAreas a ies:DispositionalClass ;
    rdfs:label "Low-altitude supersonic flight over populated areas"@en-GB ;
    rdfs:subClassOf ies:State .

# Assert the prohibition at class level (hypothetical property)
ex:Eurofighter ex:allProhibitedFrom ex:LowAltitudeSupersonicOverPopulatedAreas .

# Assert the prohibition at individual level (hypothetical property)
data:FighterXYZ-123 ex:isProhibitedFrom
    ex:LowAltitudeSupersonicOverPopulatedAreas .
```

These properties do not exist in the current IES specification and are shown here only to illustrate what an extension might look like. More broadly, prohibitions may be best addressed as part of a wider treatment of normative modality — the encoding of obligations, permissions, and prohibitions — rather than as an isolated extension to the dispositional pattern. Such an extension is beyond the scope of this document.

[§3.5 Whole-part dependencies](#)

Requirement recap: The Eurofighter’s Mach 2 disposition depends on both EJ200 engines being functional.

This requirement asks for a relationship between the disposition of a whole (the Eurofighter) and the states of its parts (the two engines). The current encoding does not define a specific mechanism for expressing that one entity’s disposition depends upon the state of another entity, or of its parts.

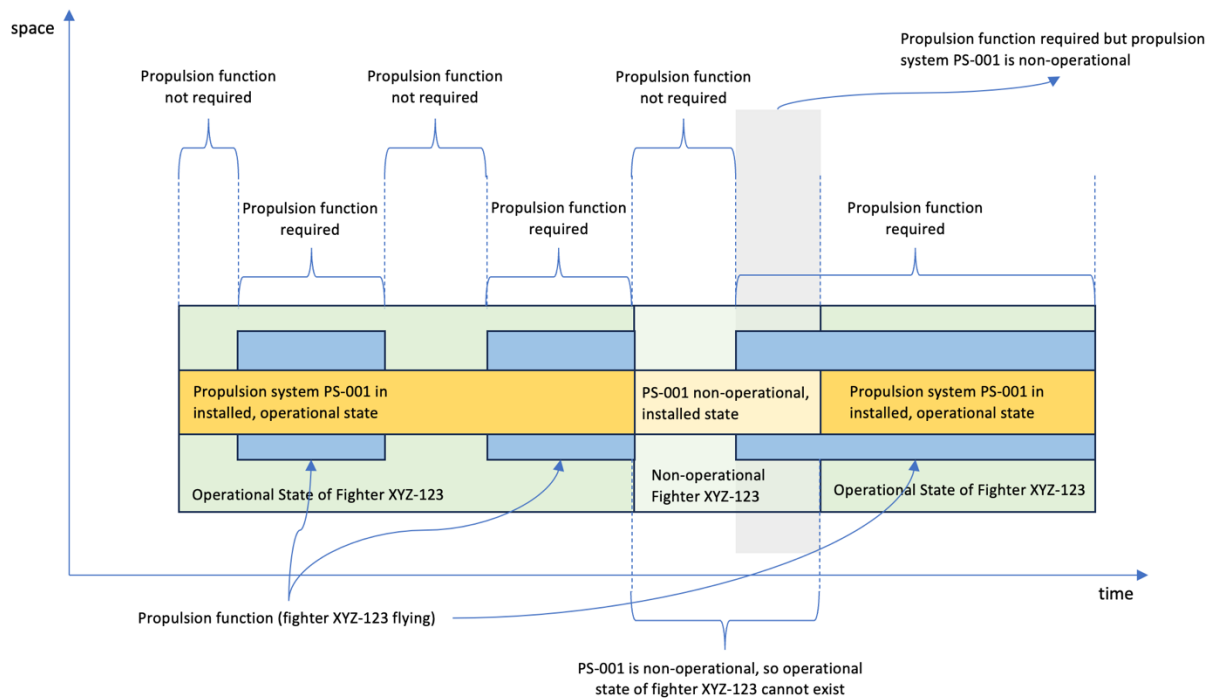


Figure 11 Spacetime diagram for whole-part dependency

In the spacetime diagram above, the grey area denotes a region of spacetime where the Eurofighter XYZ-123 requires thrust from propulsion system PS-001, but the latter is non-operational.

The requirement can be approached by combining the Capability pattern with IES's existing `isPartOf` relationship and additional Capabilities for the component parts:

```
# The engines as parts of the Eurofighter
data:EnginePort a ies:Asset ;
  ies:isPartOf data:FighterXYZ-123 ;
  rdfs:label "Port EJ200 engine of Fighter XYZ-123"@en-GB .

data:EngineStarboard a ies:Asset ;
  ies:isPartOf data:FighterXYZ-123 ;
  rdfs:label "Starboard EJ200 engine of Fighter XYZ-123"@en-GB .

# A Capability for engine thrust
ex:ProvideThrust a ies:Capability ;
  rdfs:label "Provide thrust"@en-GB ;
  rdfs:subClassOf ies:State .

# Assert each engine has this capability
```

```
data:EnginePort ies:isDisposedTo ex:ProvideThrust .
data:EngineStarboard ies:isDisposedTo ex:ProvideThrust .
```

This records that each engine can provide thrust, but does not express the dependency relationship — the fact that the Eurofighter’s Mach 2 disposition *requires* both engines to be providing thrust. The current IES specification does not define a specific mechanism for expressing that a whole’s disposition depends upon the state of its parts. Expressing such dependencies would require an extension.

§3.6 Cross-entity dependencies

Requirement recap: The Eurofighter’s extended-range disposition depends on the Voyager tanker’s refuelling disposition.

As with whole-part dependencies, the current encoding does not define a specific mechanism for expressing cross-entity dependency chains. The `DispositionalClass` pattern relates entities to their own potentials; it does not provide a vocabulary for asserting that one entity’s potential is contingent upon another entity’s potential.

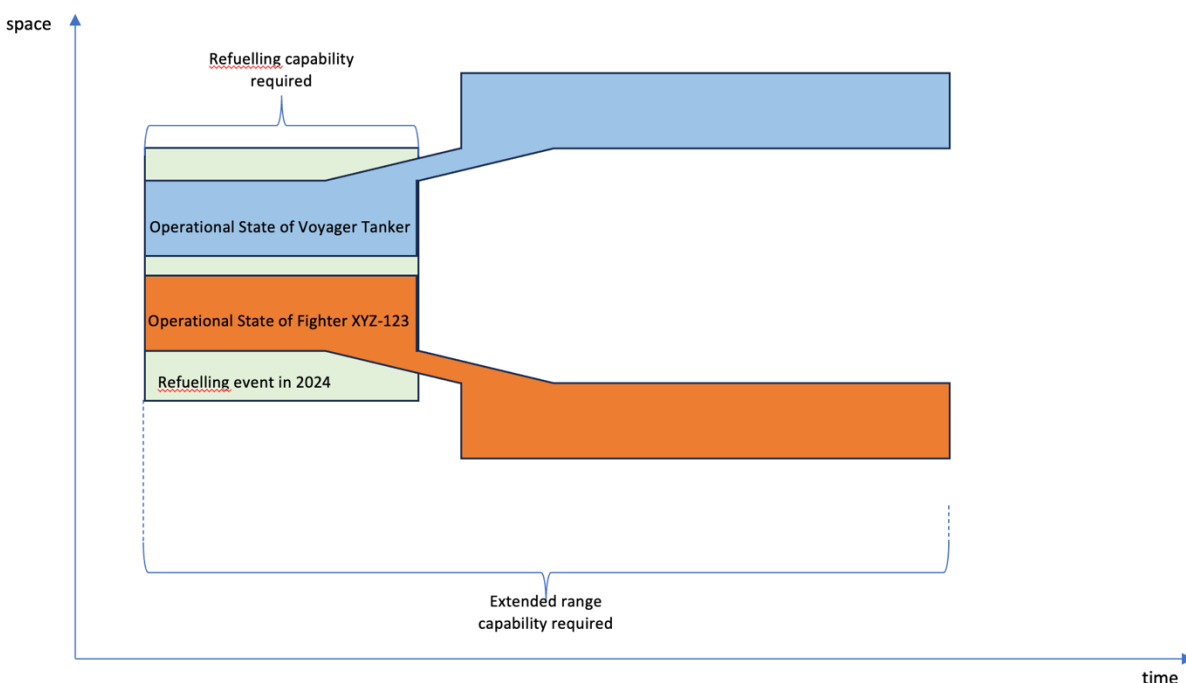


Figure 12 Actual refuelling of Eurofighter XYZ-123

A plausible approach within the current framework would combine `DispositionalClasses` with Event participation patterns, modelling the aerial refuelling as an Event in which both entities participate:

```

# The Voyager tanker and its refuelling capability
data:VoyagerTanker a ies:Entity ;
  rdfs:label "Voyager tanker"@en-GB .

ex:RefuelInMidair a ies:Capability ;
  rdfs:label "Refuel in mid-air"@en-GB ;
  rdfs:subClassOf ies:State .

data:VoyagerTanker ies:isDisposedTo ex:RefuelInMidair .

# The Eurofighter's extended-range capability
ex:FlyExtendedRange a ies:Capability ;
  rdfs:label "Fly extended-range operations"@en-GB ;
  rdfs:subClassOf ies:State .

data:FighterXYZ-123 ies:isDisposedTo ex:FlyExtendedRange .

# An aerial refuelling event linking both participants
data:RefuellingEvent2024 a ies:Event ;
  rdfs:label "Aerial refuelling event"@en-GB .

data:VoyagerTankerRefuellingState a ies:EventParticipant ;
  ies:isStateOf data:VoyagerTanker ;
  ies:isParticipantIn data:RefuellingEvent2024 .

data:FighterXYZ-123RefuellingState a ies:EventParticipant ;
  ies:isStateOf data:FighterXYZ-123 ;
  ies:isParticipantIn data:RefuellingEvent2024 .

```

This records the individual dispositions and shows both entities participating in a refuelling event. However, it models an *actual* refuelling event rather than the *dependency* between potentials. The data says “*these two entities participated in a refuelling event*” — it does not say “*the Eurofighter’s extended-range disposition requires the Voyager’s refuelling disposition.*” The distinction between recording actual co-participation and asserting a dependency between potentials is precisely what the current encoding does not capture.

5.1.6 Coverage Assessment

The existing encoding, when narrowed to State/ClassOfState semantics with Capabilities declared as subclasses of ies:State, addresses the core Aspect 1 requirements:

Requirement	Coverage	Notes
§3.1 Non-specific assets	Addressed	allHaveDisposition on a ClassOfElement.
§3.2 Class-level disposition	Addressed	allHaveDisposition on a named ClassOfEntity.
§3.3 Individual-level disposition	Addressed	isDisposedTo on an individual.
§3.4 Permitted uses	Addressed	Conditional ClassOfState hierarchy (Approach 2).
§3.4 Prohibitions	Gap	Open-world challenge; requires extension.

§3.5 Whole-part dependencies	Gap	Cannot express dependency relationships (Aspect 2).
§3.6 Cross-entity dependencies	Gap	Cannot express dependency chains (Aspect 2).
§3.7 Measured dispositions	Future work	Neither encoding addresses this.

The gaps in §3.5 and §3.6 correspond precisely to the Aspect 2 requirements — designs and plans — identified in §1.3. This alignment is not coincidental: the existing disposition mechanism is fundamentally a tool for recording modal possibility (Aspect 1), and the dependency requirements are fundamentally about compositional constraints (Aspect 2).

5.2 Part 2: Proposed Refinements for Aspect 1

5.2.1 Motivation

The current encoding’s `isDisposedTo` and `allHaveDisposition` are sub-properties of `rdf:type` and `rdfs:subClassOf` respectively. This design encodes dispositions through the class-membership mechanism, which works well for the core requirements (§3.1–§3.4) but has two practical consequences that become significant as the pattern is extended.

Reasoner behaviour and query separation. Because `isDisposedTo` is a sub-property of `rdf:type`, any RDFS or OWL reasoner will infer `rdf:type` triples from disposition assertions. Given:

```
data:FighterXYZ-123 ies:isDisposedTo ex:FlyAtMach2 .
```

a reasoner will infer:

```
data:FighterXYZ-123 rdf:type ex:FlyAtMach2 .
```

This inferred triple is indistinguishable from an ordinary class membership assertion. A SPARQL query that retrieves all `rdf:type` assertions for Fighter XYZ-123 will return both its actual types (e.g. `ies:Aircraft`) and its dispositional memberships (e.g. `ex:FlyAtMach2`), with no structural way to distinguish between the two without inspecting the class hierarchy of each result.

The semantic locus. Because `isDisposedTo` inherits the semantics of `rdf:type`, its range must be a class of which the entity *is a member*. The `DispositionalClass` must therefore be defined as a class of entities possessing the disposition — for example, “*Vehicles capable of Mach 2*” — encoding the very modality being asserted into the class definition. The result is a semantic circularity: the property asserts membership of a class whose defining criterion is the disposition that the property is meant to record. Under the proposed encoding, `hasDisposition` carries the modal semantics independently. The range is simply a `ClassOfState` — “*Fly at Mach 2*” — defined by what the state *is*, not by who possesses the potential for it. The class definition and the relationship each carry their own distinct semantic

contribution, without circularity. (See Appendix A for a supplementary analysis on the semantic locus for a detailed treatment.)

This is not a claim that the current encoding is incoherent — both interpretations are internally consistent (§5.1.2, §5.1.3). The point is that the new properties achieve the same semantic outcome with a more explicit mechanism: the modal semantics are visible in the property rather than implicit in either a complicated class definition or an overloaded reading of `rdf:type`. Both encodings operate within the same extensional framework, and the set-theoretic relationship ($Sx \cap R \neq \emptyset$) is identical in both cases.

5.2.2 Two New Properties

Two properties are proposed as backwards-compatible refinements to the existing disposition mechanism, addressing the same Aspect 1 requirements (§3.1–§3.4) with `State/ClassOfState` semantics:

`hasDisposition` (individual-level) and `eachHasDisposition` (class-level) encode the same set-theoretic relationship as `isDisposedTo` and `allHaveDisposition` through dedicated properties that are sub-properties of `ies:relationship` only — not of `rdf:type` or `rdfs:subClassOf`. This gives them a single interpretation: the modal semantics reside in the property, and the `ClassOfState` in the range has a straightforward extensional definition.

5.2.3 Applying to the Requirements

This section applies the new properties to each of the requirements from §3, using the same Eurofighter scenario and the same naming conventions as §5.1.5. For requirements §3.1–§3.4, both the current-pattern encoding and the new-property encoding are shown so the reader can see the equivalence. For requirements §3.5–§3.6, only the new properties are applicable.

The following prefixes are used throughout:

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix ies: <http://informationexchangestandard.org/ont/ies/common/> .
@prefix ex: <http://example.com/ontology#> .
@prefix data: <http://example.com/data#> .
```

§3.1 *Non-specific assets with potential*

Requirement recap: Early mission planning requires recording that a class of asset — not yet identified as “Eurofighter” — can fly at Mach 2.

Current encoding (from §5.1.5):

```
ex:FlyAtMach2 a ies:Capability ;
    rdfs:label "Fly at Mach 2"@en-GB ;
    rdfs:subClassOf ies:State .

ex:FlyAtMach2Asset a ies:ClassOfElement ;
```

```
rdfs:label "Asset that can fly at Mach 2"@en-GB ;
ies:allHaveDisposition ex:FlyAtMach2 .
```

New properties:

```
ex:FlyAtMach2 a ies:Capability ;
  rdfs:label "Fly at Mach 2"@en-GB ;
  rdfs:subClassOf ies:State .

ex:FlyAtMach2Asset a ies:ClassOfEntity ;
  rdfs:label "Fly at Mach 2 Asset"@en-GB ;
  ies:eachHasDisposition ex:FlyAtMach2 .
```

Note. As with the current encoding (§5.1.5), the explicit class `ex:FlyAtMach2Asset` is a convenience, not a necessity. The `Capability` `ex:FlyAtMach2` already implicitly defines the set of entities disposed to it — those entities x for which $S_x \cap \text{ex:FlyAtMach2} \neq \emptyset$. A consumer can query for all entities or classes bearing this disposition without a named asset class:

```
sparql
# Individual assets with this disposition
SELECT ?asset WHERE { ?asset ies:hasDisposition ex:FlyAtMach2 . }

# Classes whose members all have this disposition
SELECT ?class WHERE { ?class ies:eachHasDisposition ex:FlyAtMach2 . }
```

A named asset class becomes useful when additional constraints need to be bundled together as a reusable planning concept. For simple "I need something that can do X" requirements, the `Capability` alone suffices.

The `Capability` instance is identical in both encodings. The difference is in the relationship: `allHaveDisposition` (a sub-property of `rdfs:subClassOf`) versus `eachHasDisposition` (a sub-property of `ies:relationship`). Under the current encoding, a reasoner infers `ex:FlyAtMach2Asset rdfs:subClassOf ex:FlyAtMach2`, placing the asset class within the `Capability` hierarchy. Under the new encoding, the asset class and the `Capability` remain structurally separate, connected only by the `eachHasDisposition` arc.

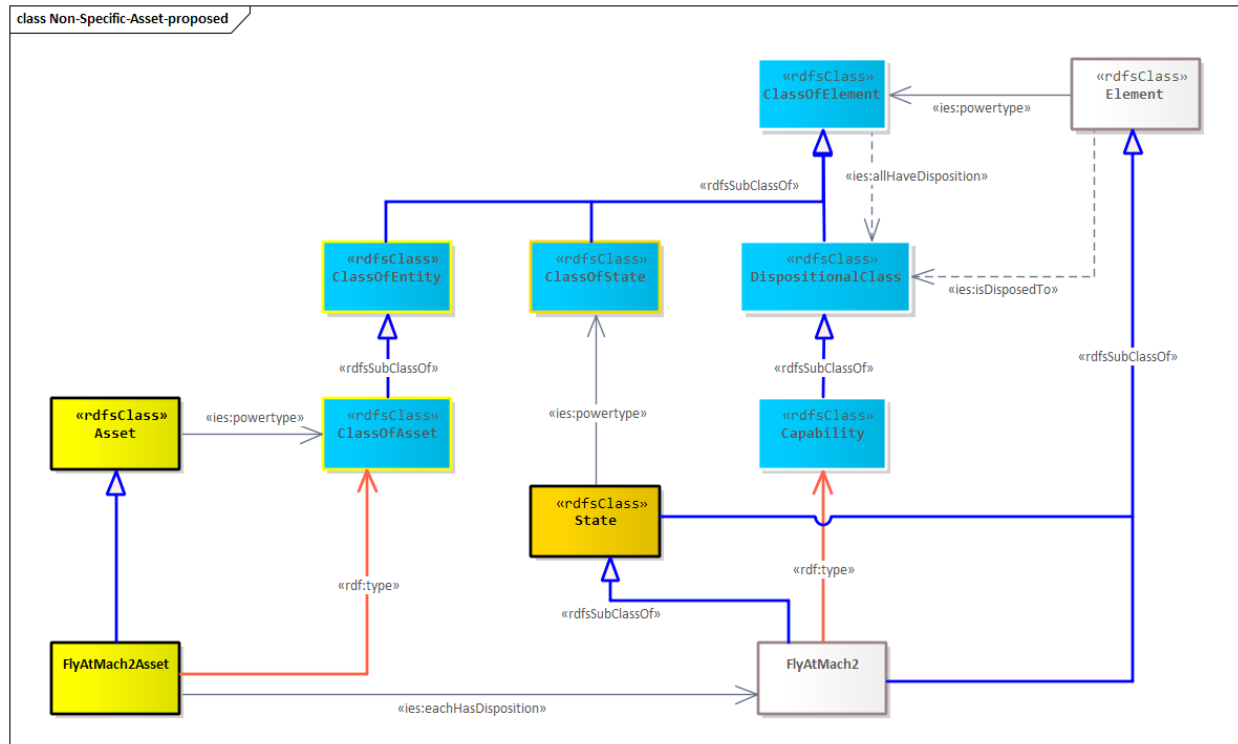


Figure 13 Proposed "eachHasDisposition" relationship

§3.2 Class-level disposition

Requirement recap: All Eurofighters can fly at Mach 2.

Current encoding:

```

ex:Eurofighter a ies:ClassOfEntity ;
  rdfs:label "Eurofighter"@en-GB ;
  rdfs:subClassOf ies:Aircraft ;
  ies:allHaveDisposition ex:FlyAtMach2 .

```

New properties:

```

ex:Eurofighter a ies:ClassOfEntity ;
  rdfs:label "Eurofighter"@en-GB ;
  rdfs:subClassOf ies:Aircraft ;
  ies:eachHasDisposition ex:FlyAtMach2 .

```

The assertion reads identically in natural language: every Eurofighter has the disposition to fly at Mach 2. The structural difference is the same as for §3.1: eachHasDisposition does not entail a rdfs:subClassOf inference.


```
data:FighterXYZ-123 a ies:Aircraft ;  
  ies:hasDisposition ex:FlyAtMach2 .
```

Under the current encoding, `isDisposedTo` entails `rdf:type ex:FlyAtMach2`. Under the new encoding, `hasDisposition` expresses modal possibility without a type inference. The domain of `hasDisposition` is `ies:State`; `data:FighterXYZ-123`, typed as `ies:Aircraft` (a subclass of `ies:Vehicle`, which is a subclass of `ies:VehicleState`, which is a subclass of `ies:State`), satisfies the domain directly.

The assertion can equally be placed on a temporal State of the Entity:

```
data:FighterXYZ-123-Operational-2024 a ies:VehicleState ;  
  ies:isStateOf data:FighterXYZ-123 ;  
  ies:hasDisposition ex:FlyAtMach2 .
```


Permitted uses with preconditions

As with the current encoding (§5.1.5), there are two plausible approaches. This document recommends Approach 2 (conditional ClassOfState) for both encodings.

Approach 2 under the current encoding (from §5.1.5):

```
ex:EurofighterInOperationalState a ies:ClassOfEntity ;
  rdfs:label "Eurofighter in operational state"@en-GB ;
  rdfs:subClassOf ex:Eurofighter ;
  ies:allHaveDisposition ex:FlyAtMach2 .

data:FighterXYZ-123-in-2025 a ex:EurofighterInOperationalState ;
  ies:isStateOf data:FighterXYZ-123 ;
  ies:isDisposedTo ex:FlyAtMach2 .
```

Approach 2 under the new properties:

```
ex:EurofighterInOperationalState a ies:ClassOfState ;
  rdfs:label "Eurofighter in operational state"@en-GB ;
  rdfs:subClassOf ies:VehicleState ;
  ies:eachHasDisposition ex:FlyAtMach2 .

data:FighterXYZ-123-in-2025 a ex:EurofighterInOperationalState ;
  ies:isStateOf data:FighterXYZ-123 ;
  ies:hasDisposition ex:FlyAtMach2 .
```

The structural difference is that the conditional class is typed as `ies:ClassOfState` rather than `ies:ClassOfEntity`, and the properties are `eachHasDisposition`/`hasDisposition` rather than `allHaveDisposition`/`isDisposedTo`.

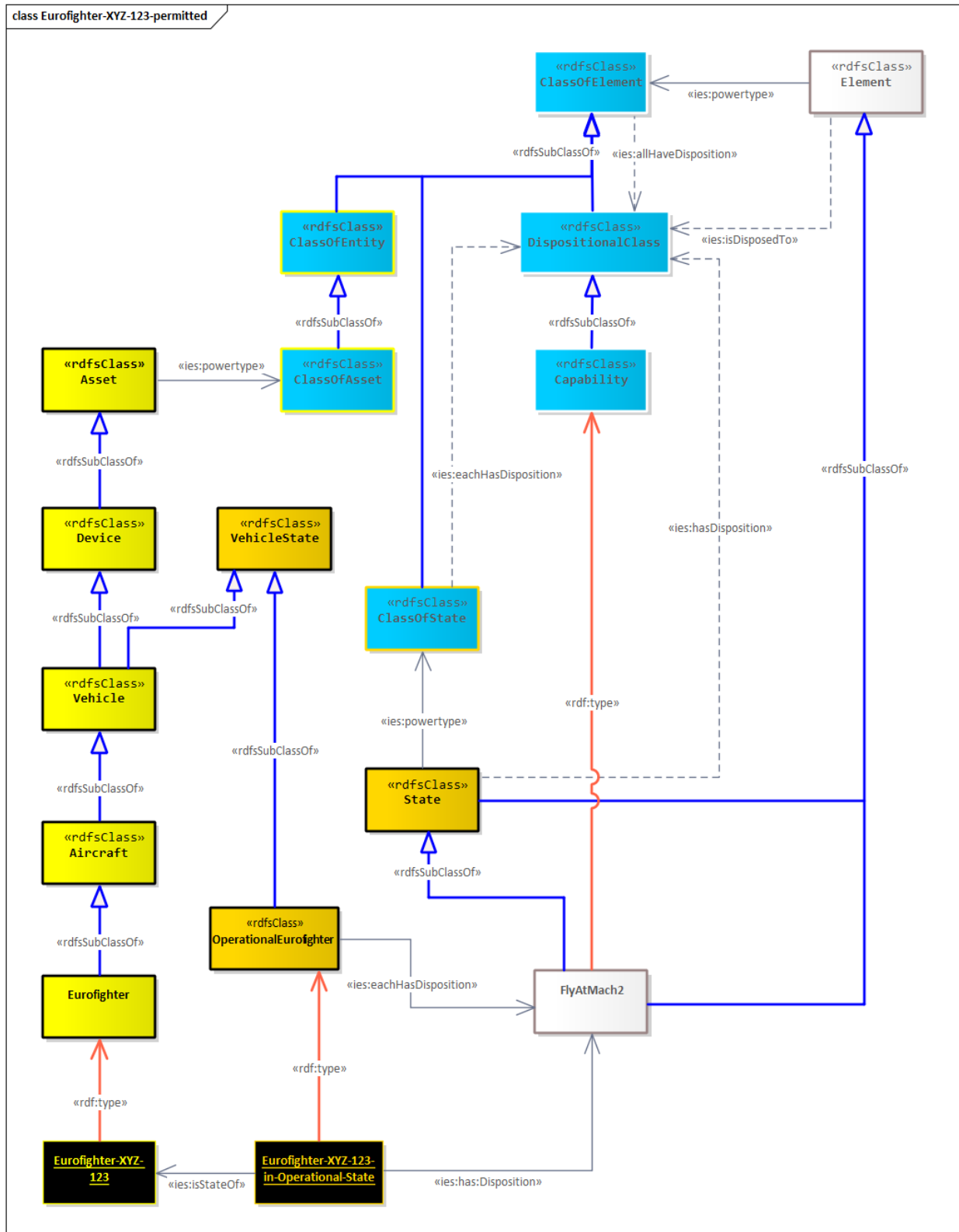


Figure 16 Only Eurofighters in an operational state can fly at Mach 2

Prohibitions

The new properties model what an entity *can* do (via `hasDisposition` and `eachHasDisposition`). They do not directly model what an entity *cannot* or *must not* do. The same considerations apply as for the current encoding (§5.1.5): explicit prohibition would require a complementary extension, potentially as part of a broader normative modality framework. A plausible extension might introduce `hasProhibitedDisposition` and `eachHasProhibitedDisposition`, mirroring the disposition properties:

```
# Hypothetical prohibition properties (not part of this proposal)
ex:LowAltitudeSupersonicOverPopulatedAreas a ies:Capability ;
    rdfs:label "Low-altitude supersonic flight over populated areas"@en-GB ;
    rdfs:subClassOf ies:State .

ex:Eurofighter ex:eachHasProhibitedDisposition
    ex:LowAltitudeSupersonicOverPopulatedAreas .

data:FighterXYZ-123 ex:hasProhibitedDisposition
    ex:LowAltitudeSupersonicOverPopulatedAreas .
```

Under open world semantics, the absence of a disposition assertion does not entail prohibition — the challenge is inherent in the open world framework, not in the choice of encoding mechanism. The extension approaches for both the current encoding and the new properties are structurally parallel.

5.2.4 Scope: The State-Bearing vs Event-Bearing Question

The preceding sub-sections address requirements §3.1–§3.4 in full. Before the coverage assessment (§5.2.5), one further scoping question raised during review is addressed: whether the `Capability` class should be defined in terms of `States` or `Events`.

Review has raised the question of whether a `Capability` should be defined as a class of `State` (being in a state of flying at Mach 2) or as a class of `Event` (participating in an event of flying at Mach 2). Both formulations are formally valid. The `State`-bearing reading yields a straightforward set-theoretic definition: $S_x \cap R \neq \emptyset$, where R is a `ClassOfState`. The `Event`-bearing reading introduces an intermediate step: the `Element` has a `State` that is a `Performer` participant in an `Event` typed to the relevant class.

The proposed refinement properties constrain to `ClassOfState`, aligning with the `Top` and `Core` architectural direction where `Activities` (the successor to `Events`) are themselves `States`. This means:

- In IES v5, `Event`-bearing dispositions are not expressible through the proposed properties. The existing `isDisposedTo` and `allHaveDisposition` (whose range is the broader `ClassOfElement`) remain available for such cases if needed.
- In IES v6 (`Top` and `Core`), `Activities` are `States`, and the distinction dissolves: both `State`-bearing and `Event`-bearing dispositions are expressible through the same `State/ClassOfState` properties.

No use cases requiring Event-bearing dispositions have been identified in the current requirements. This is acknowledged as a known limitation of the v5 scope, not a permanent exclusion.

5.3 Aspect 2: The Dependency Gap

The two new properties introduced in §5.2 — `hasDisposition` and `eachHasDisposition` — address the core disposition requirements (§3.1–§3.4). The dependency requirements (§3.5–§3.6) remain unaddressed by both the current encoding and the §5.2 proposals: neither the class-membership mechanism nor the new disposition properties naturally extend to expressing what must co-occur for a disposition to be exercised.

Recording that a Eurofighter can fly at Mach 2 (Aspect 1) is a modal assertion about possible states. Recording that a Eurofighter flying at Mach 2 requires both EJ200 engines to be operational (Aspect 2) is a constraint on composition — it specifies what must hold for the possible to be realised. These are different kinds of assertion, and the question of whether they should be handled by the same mechanism is worth examining before the IES-TG adopts any approach to the dependency requirements.

5.3.1 One Possible Approach: Dependency Properties

Four additional properties could extend the disposition vocabulary to cover co-occurrence constraints:

- `ies:requires` — an `ObjectProperty` with domain `ies:State` and range `ies:ClassOfState`. It asserts that a particular `State` requires the temporal co-occurrence of at least one member of the specified `ClassOfState`. A sub-property of `ies:relationship`.
- `ies:eachRequires` — an `ObjectProperty` with domain `ies:ClassOfState` and range `ies:ClassOfState`. It asserts that every member of the domain `ClassOfState` requires the temporal co-occurrence of at least one member of the range `ClassOfState`. A sub-property of `ies:relationship`.
- `ies:requiresPart` — an `ObjectProperty` with domain `ies:State` and range `ies:ClassOfState`. It asserts that a particular `State` requires, as a mereological part, at least one member of the specified `ClassOfState`. A sub-property of `ies:requires`.
- `ies:eachRequiresPart` — an `ObjectProperty` with domain `ies:ClassOfState` and range `ies:ClassOfState`. It asserts that every member of the domain `ClassOfState` has, as a mereological part, at least one member of the range `ClassOfState`. A sub-property of `ies:eachRequires`.

None of these properties is a sub-property of `rdf:type` or `rdfs:subClassOf`. The modal semantics are carried by the property definitions themselves.

This approach does not fully satisfy the dependency requirements. In particular, it does not express:

- **Cardinality constraints:** "*requires both EJ200 engines*" cannot be distinguished from "*requires at least one engine*."
- **State-specificity of required parts:** the constraint that the required part must itself be in a specific state (e.g. "operational") rather than merely present.
- **Transitivity of dependencies:** dependency chains cannot be expressed without explicit intermediate assertions.

A worked example illustrating this approach is included in Appendix B for information and discussion.

5.3.2 An Alternative View: Compositional Constraints

An alternative view, which emerged during project review, treats the dependency requirements as constraints on whole-part composition rather than extensions of the disposition mechanism. Under this view:

- The modal assertion (Aspect 1) records what is possible: "A Eurofighter can fly at Mach 2."
- The compositional constraint (Aspect 2) records what the design requires: "A Eurofighter flying at Mach 2 must have both engines fully operational."

The compositional constraint concerns the structure of the thing — what parts it has and what states those parts must be in — rather than its modal properties. This suggests that the dependency requirements might be better served by mechanisms rooted in IES's existing mereological vocabulary (*isPartOf*, class-level assertions about part composition) than by extending the disposition properties.

The choice between these approaches — four additional dependency properties, or a separate mechanism for compositional constraints — is an open question that this document brings to the IES-TG for guidance.

5.3.3 RDFS Expressibility

Both approaches face a common challenge in RDFS: expressing that all members of a class must have parts of a specified type in a specified state cannot be fully captured without moving beyond RDFS. The open-world assumption means that the absence of a part assertion does not entail that the part is absent. OWL restrictions — *owl:someValuesFrom*, *owl:allValuesFrom*, cardinality constraints — provide greater expressivity, but at the cost of departing from IES's RDFS-first philosophy.

This constraint applies equally to the disposition-property approach and to any mereological alternative. The IES-TG's guidance on acceptable expressivity trade-offs — in particular, whether selective use of OWL restrictions is warranted for compositional constraints — would be valuable.

The existing encoding and the proposed Aspect 1 refinements both leave the dependency requirements (§3.5–§3.6) unaddressed. More directly, the class-membership mechanism does not naturally extend to dependency relationships — what must co-occur for a disposition to be exercised. The current encoding cannot express that the Eurofighter's operational state requires an operational propulsion system (§3.5) or that extended-range flight requires the co-occurrence of a refuelling aircraft (§3.6).

6 Addressing Reviewer Feedback

The reviewer's feedback has materially improved this document. Specific, well-grounded concerns — particularly about whole-life Entities, expressivity, and the substantiation of claims — led to clearer explanations, the collapse of the proposed *ies:Capacity* class into the existing *ies:Capability*, and the acknowledgement of overclaims in the earlier proposal documentation. This section engages with each concern in turn, directing the reader to the evidence developed in §2–§5 and the semantic locus supplementary analysis. Where the original proposal's claims were overstated or insufficiently justified, this is acknowledged.

A note on terminology. The original proposal used the terms Capacity, hasCapacity, and eachHasCapacity. The analysis in §5 established that the proposed `ies:Capacity` class would be semantically equivalent to the existing `ies:Capability` — both identify `ClassOfElement` instances representing what things can do, using the same set-theoretic basis. Therefore, the existing meta-classes are retained and no new class is introduced. The proposed properties are now named `ies:hasDisposition` and `ies:eachHasDisposition`, reflecting the broader dispositional semantics that encompass both capabilities and tendencies. References to the original terminology in the reviewer’s feedback and in this section’s responses should be understood in this light; all cross-references to §3–§5 use the current terminology.

6.1 “Starting with a UML Conceptual Model would have helped”

The reviewer recommends beginning with a UML conceptual model, noting that this approach has proved effective in collaborative ontology development. This is sound methodological advice. UML diagrams created during development of the proposal have been included in this document to complement the Turtle examples, spacetime diagrams, and Venn diagrams.

6.2 “Whole-life Entities cannot have capacities”

The reviewer raised the concern that the original proposal appeared to exclude whole-life Entities from bearing dispositions, noting that “*whole-life Entities having modal properties would seem to be the typical case.*” This concern was well-founded — not as a structural objection to the State domain, but as a response to documentation that was genuinely misleading.

The original proposal used `ies:State` as the domain of `hasDisposition` (then named `hasCapacity`), and its accompanying text — “*Capacities attach to States (temporal parts) not whole-life Entities*” — actively obscured how the encoding actually works. A reader encountering that phrasing would reasonably conclude that whole-life Entities were excluded. The reviewer’s concern was an entirely appropriate reaction to poor documentation, and it prompted a much clearer explanation of the relationship between Entities and States in IES.

The State domain does accommodate whole-life Entities in IES v5. Every Entity subclass in IES v5 is also a subclass of State through the Entity-as-State subclassing pattern: `ies:Aircraft`, for example, is a subclass of both `ies:MovingVehicle` (an Entity subclass) and `ies:VehicleState` (a State subclass). A whole-life Entity such as Fighter XYZ-123, typed as `ies:Aircraft`, satisfies the `ies:State` domain constraint directly — no additional modelling apparatus is required, and no intermediate States need to be created. The Turtle examples in §5 demonstrate whole-life Entities bearing dispositions without indirection.

The reviewer’s contractual example — “*the aeroplane itself was expected to have this capacity*” in a delivery context — is handled at the whole-life Entity level under the proposed encoding. Fighter XYZ-123 satisfies the State domain and can bear a `hasDisposition` assertion directly. The temporal scoping mechanism — attaching assertions to specific States when a temporally bounded assertion is needed — remains available for cases where a disposition is time-limited.

The reviewer further observed that the Entity-as-State mechanism “*falls short of being able to model the standard modal properties of whole life entities.*” This observation was a fair reading of the original documentation. With the clearer explanation provided in this document, the objection is addressed: whole-life Entities are States in v5, and are therefore first-class bearers of dispositions under the proposed encoding.

§2 establishes the broader architectural context. In Top and Core, the Entity/State distinction is eliminated entirely — every concrete individual is an `ies_top:State` by definition. The State domain was retained for the proposed properties precisely because it aligns with this architectural direction. Under Top and Core, there is no Entity-as-State subclass pattern to explain; the domain simply covers everything. The reviewer’s concern, which exposed a genuine documentation gap, contributed to the decision to provide the architectural rationale in §2 from the outset.

6.3 “Better alignment with modal logic — unsubstantiated”

The reviewer notes that the claim of “better alignment with modal logic” is unsupported by references to the literature. This criticism is fair. The claim in the original proposal documentation was aspirational rather than formally justified, and the document does not establish a correspondence with any specific system of modal logic.

What the analysis does substantiate is a narrower and more defensible claim: both encodings express the same set-theoretic relationship (§4.2, §5), but the proposed encoding makes the modal intent more explicit in the RDF structure. Under the existing encoding, the modality is carried by class definitions or inferred from the label of a `DispositionalClass`, because `isDisposedTo` inherits “is a” semantics from `rdf:type`. Under the proposed encoding, the modality is carried by the property itself — `hasDisposition` means “can be in states of this kind.” This is a claim about explicitness of encoding, not about formal alignment with modal logic. §5 and the semantic locus supplementary analysis present the evidence for this narrower claim.

The original broad claim is withdrawn and replaced with this more precise formulation. Whether the encoding difference constitutes “*better alignment with modal logic*” in a formal sense is a question that would require further work beyond the scope of this document.

6.4 “Clearer spatio-temporal semantics — unsubstantiated”

The reviewer observes that the claim of “*clearer spatio-temporal semantics*” is, as presented in the original documentation, unsubstantiated. This is acknowledged.

The original claim was poorly argued. What is true is that the State domain means every disposition assertion is made on a State — a bounded spatiotemporal extent. This provides a degree of spatio-temporal anchoring by default: if the bearer is a temporal part of an Entity, the disposition assertion is inherently scoped to the temporal extent of that State. If the bearer is a whole-life Entity (which is also a State in v5), the assertion is scoped to the whole-life extent.

However, this is a consequence of the State/ClassOfState encoding choice, not a claimed advantage over the existing encoding. The existing encoding's `isDisposedTo` has domain `ies:Element` in v5, and the temporal scoping mechanism — making assertions on specific States of an Entity — is available under both encodings. Both encodings can attach assertions to temporally bounded States when precision is needed. The difference is that the proposed encoding's State domain makes this the structural default rather than an optional practice, but this is a design choice motivated by the architectural direction (§2), not a superiority claim.

6.5 “The proposal reduces expressivity”

The reviewer observed that the original proposal's domain (`ies:State`) excluded Events as bearers of dispositions, reducing expressivity relative to the existing encoding's domain (`ies:Element`). This was a technically correct observation in the context of IES v5: `ies:Event` is not a subclass of `ies:State` in v5, so Events do not satisfy the State domain.

This is a deliberate, known consequence of the State/ClassOfState encoding choice. The proposed properties target the domain and range that align with the Top and Core architectural direction (§2), accepting this v5 limitation rather than widening the domain to accommodate a class hierarchy that Top and Core will restructure.

No use cases for Event-bearing dispositions have been identified in the current analysis. The IES community has not raised requirements for asserting that an Event “can do” something in the dispositional sense; dispositions are characteristically borne by the entities that participate in events, not by the events themselves.

§2.3 establishes that Top and Core resolves the structural question: Activities (the successor to Events) are States under the new architecture, and any property with a State domain — including `hasDisposition` and `eachHasDisposition` — will admit Activities as bearers. The Event exclusion is therefore a temporary limitation of the v5 class hierarchy, not a permanent constraint of the encoding.

In the interim, the existing `isDisposedTo` and `allHaveDisposition` properties — which have `Element` domains — remain available. Should an Event-bearing disposition use case arise in v5, these properties provide a mechanism for expressing it. The proposed encoding does not remove any existing capability; it adds new properties with a domain that anticipates the architectural direction.

6.6 “The original model supports non-specific assets, class-level, and individual-level; the proposal excludes this”

The reviewer describes three levels of planning — non-specific assets with a required capability, specific types of aircraft, and specific individual assets — and states that “the original model supports this. But it looks as if the current proposal specifically excludes this.”

The proposed encoding supports all three levels. §5 addresses non-specific assets with potential, class-level dispositions (e.g. “every Eurofighter can fly at Mach 2”), and individual-

level dispositions (e.g. “Fighter XYZ-123 can fly at Mach 2”), with concrete Turtle examples at each level. The class-level property `eachHasDisposition` addresses the reviewer’s second case; the individual-level property `hasDisposition` addresses the third; and the meta-class apparatus (using `DispositionalClass` instances to characterise required capabilities without identifying specific bearers) addresses the first.

6.7 “That’s how extensional ontologies work. Fudging this won’t make it better.”

The reviewer’s point is that both encodings necessarily operate within extensional constraints, and that the proposed encoding cannot escape them. This is correct, and the document’s analysis confirms it.

Both encodings express the same set-theoretic relationship: $Sx \cap R \neq \emptyset$ (§4.2, §5). Neither invokes intensional or non-extensional machinery. The proposed encoding does not claim to escape the extensional framework; it offers a different encoding of the same extensional relationship within that framework.

The encoding difference does, however, have consequences that §5 and the semantic locus analysis examine: semantic circularity in class definitions, invisibility of modality to automated reasoners, and alignment of meta-level placement with documented intent. These are not claims that the proposed encoding transcends extensional ontology — they are claims about which encoding makes the extensional commitment more explicit and tractable in practice.

6.8 “Disposition and ‘capacity’ are not 4D problems, it’s set-theory and possible worlds”

Agreed. §4 establishes this as the shared foundation: the set-theoretic intersection of possible states with a `ClassOfElement` is the common ground for both encodings, and the formal apparatus is set theory and possible-worlds semantics. The question addressed by this document is not a 4D problem *per se*, but a question about how to express set-theoretic relationships in RDF within a 4D ontological framework.

If the original proposal documentation framed the issue as a “4D problem,” this framing was imprecise. The reviewer’s correction is well-taken: the challenge is set-theoretic; the 4D ontology provides the context (States as temporal parts, the Entity-as-State pattern) but does not generate the problem. The analysis in §4 and §5 reflects this distinction — §4 establishes the set-theoretic common ground, and §5 examines how the proposed encoding expresses it within the IES framework.

7 Findings and Questions for the IES-TG

7.1 Summary of Findings

This document set out to examine the requirements for recording what things *can* do within IES Common and to analyse how far the existing disposition mechanism meets those requirements. The analysis, driven by seven requirements (§3) and grounded in a shared set-theoretic foundation (§4), has reached the following findings:

- 1. The requirements involve two distinct aspects of capability.** Aspect 1 (possible worlds) concerns what things *could* do — modal assertions about potential. Aspect 2 (designs and plans) concerns what *must* hold for potential to be realised — constraints on composition. Both are well understood in the broader ontology literature but only Aspect 1 is currently addressed by IES.
- 2. The existing disposition mechanism addresses Aspect 1 well.** The existing `isDisposedTo` and `allHaveDisposition` properties, when interpreted under Interpretation 2 (modal semantics carried by the property) and narrowed to `State/ClassOfState` semantics, cover requirements §3.1–§3.4 with concrete, implementable encodings. The worked examples in §5.1 demonstrate this coverage.
- 3. Two refinement properties would strengthen the Aspect 1 encoding.** `hasDisposition` and `eachHasDisposition`, as sub-properties of `ies:relationship` rather than `rdf:type/rdfs:subClassOf`, provide a cleaner separation of disposition assertions from class membership. They give a single interpretation of the encoding mechanism and avoid the `rdf:type` inference that conflates dispositional potential with ordinary class membership. These are proposed as backwards-compatible additions alongside the existing properties.
- 4. Aspect 2 (designs and plans) is not addressed by the disposition pattern.** Requirements §3.5 (whole-part dependencies) and §3.6 (cross-entity dependencies) concern compositional constraints that are conceptually distinct from modal possibility. One possible approach extends the disposition properties with four dependency properties; an alternative roots the solution in IES’s mereological vocabulary. The choice between these approaches is an open question.
- 5. The State-bearing vs Event-bearing question is a scoping matter.** The proposed refinements constrain to `ClassOfState`, excluding Event-bearing dispositions in IES v5. No use cases requiring Event-bearing dispositions have been identified. Under Top and Core (IES v6), the distinction dissolves because Activities are States.
- 6. Several areas remain unresolved.** Prohibitions (the complementary assertion that something *must not* exercise a potential), measured and quantified dispositions (§3.7), and the full elaboration of the designs-and-plans mechanism are all identified as areas for further work.

7.2 Overclaims Acknowledged

The original proposal documentation made several claims that this analysis has found to be overstated or insufficiently justified. These shortcomings have been addressed through the analysis presented in this document.

“**Better alignment with modal logic**” was an aspirational claim without formal justification. The document does not establish a correspondence with any specific system of modal logic. The defensible replacement is a claim about explicitness of encoding: the proposed properties encode

the modal intent in dedicated properties rather than in class definitions, making modality visible to both human readers and automated processes (§5, §6.3).

“Clearer spatio-temporal semantics” was poorly argued in the original documentation. The State domain means every disposition assertion is made on a State — a bounded spatio-temporal extent — providing a degree of spatio-temporal anchoring by default. However, this is a consequence of the State/ClassOfState encoding choice motivated by the architectural direction (§2), not a claimed superiority over the existing encoding. Both encodings can attach assertions to temporally bounded States when precision is needed; the temporal scoping mechanism is available regardless of the encoding.

“Capacities attach to States not whole-life Entities” was imprecisely worded, and the reviewer’s concern was an entirely appropriate reaction to poor documentation. The State domain does accommodate whole-life Entities in IES v5, through the Entity-as-State subclassing pattern — every Entity subclass is also a subclass of State (§5.1, §6.2). Under Top and Core, the Entity/State distinction is eliminated entirely and the domain simply covers everything (§2.1). The reviewer’s feedback exposed a genuine documentation gap and contributed to the decision to provide the architectural rationale in §2 from the outset.

Collapse of Capacity into Capability. The proposed Capacity class was found to be semantically equivalent to the existing Capability — both identify ClassOfElement instances representing what things can do, using the same set-theoretic basis. Following the IES parsimony principle, the existing meta-classes are retained and no new class is introduced (§5.1).

7.3 Questions for the IES-TG

The IES-TG’s guidance is sought on the following:

Q1. Requirements validation. Does the IES-TG agree that the seven requirements in §3 are the right set? Are there requirements that should be added, removed, or reframed?

Q2. The two-aspects framing. Does the IES-TG accept the distinction between Aspect 1 (possible worlds / modal assertions about potential) and Aspect 2 (designs and plans / compositional constraints)? Is this the right way to structure further work?

Q3. Existing encoding endorsement. Does the IES-TG agree that the existing isDisposedTo/allHaveDisposition mechanism, under Interpretation 2 and with State/ClassOfState semantics, addresses requirements §3.1–§3.4?

Q4. Aspect 1 refinement properties. Should hasDisposition and eachHasDisposition be taken forward as a formal change proposal for inclusion in IES Common? If so, should they be proposed for v5.x (a minor release) or deferred to v6?

Q5. Aspect 2 direction. How should requirements §3.5–§3.6 (whole-part and cross-entity dependencies) be addressed? Specifically:

- (a) Should they be handled as an extension of the disposition pattern (the dependency-property approach)?
- (b) Should they be handled through a separate mechanism rooted in mereological and compositional constraints?

(c) Should a dedicated sub-group or workshop be commissioned to explore both approaches?

Q6. Scope of the Event-bearing case. Is the deliberate exclusion of Event-bearing dispositions from the v5 scope acceptable, given that no use cases have been identified and that the distinction dissolves under Top and Core?

Appendix A: The Semantic Locus of the Modal Commitment

This appendix provides a detailed analysis of *where* the modal semantics reside in the two encoding mechanisms described in §5 — the existing properties (`isDisposedTo`, `allHaveDisposition`) and the proposed extension properties (`hasDisposition`, `eachHasDisposition`). The analysis supports the document’s recommendation of Interpretation 2 (§5.1.2) and the adoption of the new properties (§5.2), and examines whether `ies:PossibleWorld` could provide the modal semantics that the class-membership mechanism cannot carry.

A.1 The Encoding Properties as the Determining Factor

§5.1 establishes that the existing encoding and the proposed extension share the same meta-class infrastructure — `ies:DispositionalClass`, `ies:Capability`, and `ies:Tendency` — and the same set-theoretic relationship ($Sx \cap R \neq \emptyset$). The difference between the two mechanisms therefore does not reside in the meta-classes, but in the *properties* that connect States and ClassOfStates to DispositionalClass members — and, consequently, in how those members must be defined.

The existing encoding provides two properties:

- `ies:isDisposedTo` — a sub-property of `rdf:type`, connecting a State to a DispositionalClass
- `ies:allHaveDisposition` — a sub-property of `rdfs:subClassOf`, connecting a ClassOfState to a DispositionalClass

The proposed extension provides two corresponding properties:

- `ies:hasDisposition` — an independent property (not a sub-property of `rdf:type`), connecting a State to a ClassOfState
- `ies:eachHasDisposition` — an independent property (not a sub-property of `rdfs:subClassOf`), connecting a ClassOfState to a ClassOfState

Because a sub-property inherits the semantics of its super-property and cannot alter them, `ies:isDisposedTo` carries “is a” semantics and `ies:allHaveDisposition` carries “is a subclass of” semantics. By contrast, `ies:hasDisposition` and `ies:eachHasDisposition` are free to carry their own modal semantics: “has the potential to be in states of this kind.”

This distinction has a direct and unavoidable consequence for how the *members* of each meta-class must be defined.

A.2 Effect on Member Definitions

Consider the requirement from §3.3: "Fighter XYZ-123 can fly at Mach 2." In both encodings, we need a member of a `DispositionalClass` to represent "Mach 2 flight." But the semantics of the connecting property dictate what that member can be — and, consequently, where the semantic complexity must reside.

Existing encoding. The assertion:

```
data:FighterXYZ-123 ies:isDisposedTo ex:SomeMember .
```

entails, via the sub-property relationship:

```
data:FighterXYZ-123 rdf:type ex:SomeMember .
```

This means `ex:SomeMember` must be a class of which the aircraft *is a member* — extensional class membership, with no modal content. If the member were defined straightforwardly as a class of states of flying at Mach 2, the entailed triple would assert that the aircraft *is* a state of flying at Mach 2, which is a type error. To avoid this, the member must be defined as a class of *entities* that possess the potential in question:

```
ex:VehiclesCapableOfMach2 a ies:Capability ;  
    rdfs:label "Vehicles capable of Mach 2"@en-GB ;  
    rdfs:comment "The class of vehicles that are capable of flying at Mach 2."  
"@en-GB .
```

```
data:FighterXYZ-123 ies:isDisposedTo ex:VehiclesCapableOfMach2 .
```

The member class is now defined in terms of the very disposition being asserted: "vehicles that are *capable of*..." The modality ("capable of") is embedded in the class definition and label, because the relationship (`rdf:type`) cannot carry it. This introduces a form of *definitional circularity*: `isDisposedTo` asserts membership of a class whose defining criterion *is* the disposition. Note, however, that the circularity is a consequence of the semantic architecture — `rdf:type` cannot carry modal content — rather than a logical fallacy. The encoding is consistent and usable; the cost is that the modal content is expressed in human-readable natural language (the label) rather than in machine-readable structure.

Proposed encoding. The assertion:

```
data:FighterXYZ-123 ies:hasDisposition ex:FlyAtMach2 .
```

carries modal semantics directly: the aircraft *has the potential* to be in states of the kind described by `ex:FlyAtMach2`. The member class can therefore be defined simply and non-circularly as a `ClassOfState`:

```
ex:FlyAtMach2 a ies:Capability ;  
    rdfs:label "Fly at Mach 2"@en-GB ;  
    rdfs:comment "The class of states in which an entity is flying at Mach 2."  
"@en-GB ;  
    rdfs:subClassOf ies:State .
```

```
data:FighterXYZ-123 ies:hasDisposition ex:FlyAtMach2 .
```

No circularity arises: `hasDisposition` asserts the potential; the member class simply describes the type of state. The class definition and the relationship each carry their own distinct semantic contribution. The shift is not costless, however: under this encoding, the modal semantics reside in the *property definition* rather than the class label. A consuming system must consult the ontology definition of `hasDisposition` to understand that the assertion expresses potential rather than actuality. In the existing encoding the modal content is visible in a human-readable label; in the proposed encoding it is visible in the property's machine-readable identity — accessible to a system that knows the convention, but not self-evident from the triple alone.

This is the pattern that §5.1.2 describes as Interpretation 2 — the class is straightforward, and the relationships carry the modal semantics. The proposed properties make Interpretation 2 the only available reading, whereas the existing properties leave both interpretations open (§5.1.2).

A.3 Three Consequences

The difference in property semantics produces three interrelated consequences.

1. Definitional circularity. In the existing encoding, the member class must be defined in terms of the disposition being asserted, because `rdf:type` carries no modal content. The class "Vehicles capable of Mach 2" is defined by the capability of Mach 2 flight, and `isDisposedTo` asserts that an entity has that capability by placing it in that class. The assertion and the class definition are mutually dependent. This circularity is not a logical inconsistency — the encoding remains valid and usable — but it means the modal content is expressed in natural language rather than in machine-readable structure. Under the proposed encoding, `hasDisposition` independently asserts the potential, so the member class need only describe what the state *is*, not who possesses the potential for it. The semantic complexity is redistributed: it moves from the class definition (where it is visible in a label) to the property definition (where it is visible to a system that consults the ontology).

2. Modality is invisible to consuming applications that do not consult the ontology. When `isDisposedTo` is treated as a sub-property of `rdf:type`, any RDF or OWL reasoner will infer standard class membership. The modal intent — that the aircraft *can* fly at Mach 2, not that it *is* flying at Mach 2 — exists only in the human-readable `rdfs:label` and `rdfs:comment` of the `DispositionalClass`. No automated process operating on the triples alone can distinguish a dispositional assertion from an ordinary type assertion. Under the proposed encoding, the modal semantics are carried by a distinct property (`hasDisposition`) that is structurally separate from `rdf:type`. A SPARQL query or consuming application that knows the convention can therefore distinguish dispositional assertions from type assertions by graph structure alone — a distinction that is invisible in the existing encoding. The improvement is tractability for applications that understand the convention, not autonomous reasoning about possibility by a generic OWL reasoner. §5.2.6 demonstrates this difference at the SPARQL level.

3. Meta-level placement. In IES v5, `ies:DispositionalClass` is a subclass of `ies:ClassOfElement`, meaning its members are classes of *Elements*. But the documented intent — "capable of Mach 2 but has not yet flown that fast" — is about states: temporal parts of entities during which some property holds. A member class such as "Vehicles capable of Mach 2" is a class of entities defined by reference to a class of states, but the reference to states is implicit, not formally expressed. The proposed encoding defines the range explicitly as `ies:ClassOfState`, directly modelling what the documentation intends: classes of states that entities have the potential to be in.

This meta-level placement issue is precisely what the Top and Core architecture (§2) resolves. Under Top and Core, `ies_core:Disposition` is a subclass of `ies_top:SetOfStates` — the powertype of the unified `ies_top:State` class. The meta-level placement aligns with the semantic content by design, and the tension identified here between the meta-class hierarchy and the documented intent no longer arises.

A.4 Could PossibleWorld Provide the Missing Modality?

IES contains a concept that touches on modal reasoning: `ies:PossibleWorld`, defined as “*an Element that encompasses a number of Events, Entities and States that may occur / have occurred.*” The IES specification describes it as a placeholder, noting that “more work is needed on this in later IES versions,” but its basic mechanism is clear: elements are grouped into a PossibleWorld via `ies:isPartOf` to assert that they share the same hypothetical truth. §4.1 notes that both encoding mechanisms are compatible with PossibleWorld. This raises a natural question: could PossibleWorld be used to supply the modal semantics that `rdf:type` cannot carry, thereby making the existing encoding properties sufficient for all cases?

The most faithful attempt would be to create a possible world containing a state instance of the relevant type:

```
data:PossibleWorld-1 a ies:PossibleWorld .

data:Mach2FlightState-XYZ-123 a ies:State ;
  rdf:type ex:FlyAtMach2 ;
  ies:isStateOf data:FighterXYZ-123 ;
  ies:isPartOf data:PossibleWorld-1 .
```

This asserts: “there is a possible world in which XYZ-123 has a state of flying at Mach 2.” The potentiality is expressed by the state existing in a possible world rather than in the actual world. This avoids the circularity identified in §A.2 (the state class is straightforwardly defined) and makes the modality visible to reasoners in principle (a consuming system can check whether a state belongs to a PossibleWorld).

However, PossibleWorld and the disposition properties are solving fundamentally different problems, and substituting one for the other introduces difficulties that the proposed encoding avoids.

Dispositional vs epistemic modality. When we record that “XYZ-123 can fly at Mach 2,” we are asserting an *intrinsic property* of the aircraft — a disposition assessed, certified, or

determined through engineering analysis. The aircraft possesses this potential in the actual world, right now, regardless of whether it has exercised it. This is *dispositional* modality. A PossibleWorld, by contrast, expresses *epistemic* modality: “in this scenario, these things are true.” Saying “*there is a possible world in which XYZ-123 flies at Mach 2*” is a statement about what we can imagine or hypothesise, not about what the aircraft can actually do. A cardboard glider also exists in some possible world where it flies at Mach 2 — but it does not *have the disposition* for Mach 2 flight. The distinction matters: a disposition is grounded in the assessed properties of the entity; possible-world membership is not.

Instance-level burden. The PossibleWorld approach requires minting individual state instances and individual possible worlds for each disposition of each entity. To record that one aircraft has three dispositions requires three possible worlds (or one possible world with careful partitioning), three state instances, and at least nine triples (`rdf:type`, `isStateOf`, and `isPartOf` for each). The proposed encoding requires three triples — one `hasDisposition` assertion per disposition. For a fleet of aircraft with multiple assessed dispositions, the difference in data volume and production complexity is substantial.

Placeholder status. The IES specification is candid that PossibleWorld is a “very simple placeholder” without well-defined semantics for how possible worlds interact with the rest of the ontology. Building a disposition-modelling pattern on this foundation would depend on semantics that have not yet been specified, creating a fragile dependency on future work that may take a different direction.

Where PossibleWorld *does* have a natural complementary role is in modelling *conditional* or *hypothetical* dispositions — potentials that are contingent on changes that have not yet occurred. For example: “if XYZ-123 is fitted with upgraded engines, it could fly at Mach 2.5.” This is not a current disposition of the aircraft; it is a hypothetical disposition contingent on a modification. A PossibleWorld could group together the engine-upgrade event, the resulting post-upgrade state of the aircraft, and the Mach 2.5 disposition into a coherent scenario. In this usage, PossibleWorld performs its proper function (scenario grouping) alongside the disposition properties performing their proper function (expressing potential), rather than one substituting for the other.

A.5 Summary

The difference between the existing encoding properties and the proposed extension properties is not the meta-class — `ies:DispositionalClass`, `ies:Capability`, and `ies:Tendency` are shared infrastructure used by both (§5.1.1). The difference is the *property semantics*, and the downstream effect those semantics have on how member classes are defined, where modality is expressed, and whether the formal machinery aligns with the documented intent.

In the existing encoding, the properties inherit “is a” semantics from `rdf:type` and `rdfs:subClassOf`, forcing modality into class definitions where it becomes circular and invisible to automated reasoning. The existing `ies:PossibleWorld` concept, while relevant to modal reasoning more broadly, addresses epistemic rather than dispositional modality and cannot substitute for a dedicated disposition-modelling mechanism without significant cost in complexity and expressiveness.

In the proposed encoding, the properties carry the modal semantics independently, allowing member classes to be clean, non-circular descriptions of state types. PossibleWorld remains available as a complementary tool for conditional or hypothetical scenarios — a role for which it is better suited than as a substitute for dedicated disposition properties.

The circularity argument (§A.2), the reasoner-visibility observation (§A.3, consequence 2), and the PossibleWorld assessment (§A.4) are not replicated in the main body of the document. They provide the detailed analytical support for the encoding recommendations made in §5.2 and §7.2.

Appendix B Disposition and Dependencies

For discussion, this appendix provides a worked example of possible modal properties applied to cross-dependency requirements and whole-part capability dependencies.

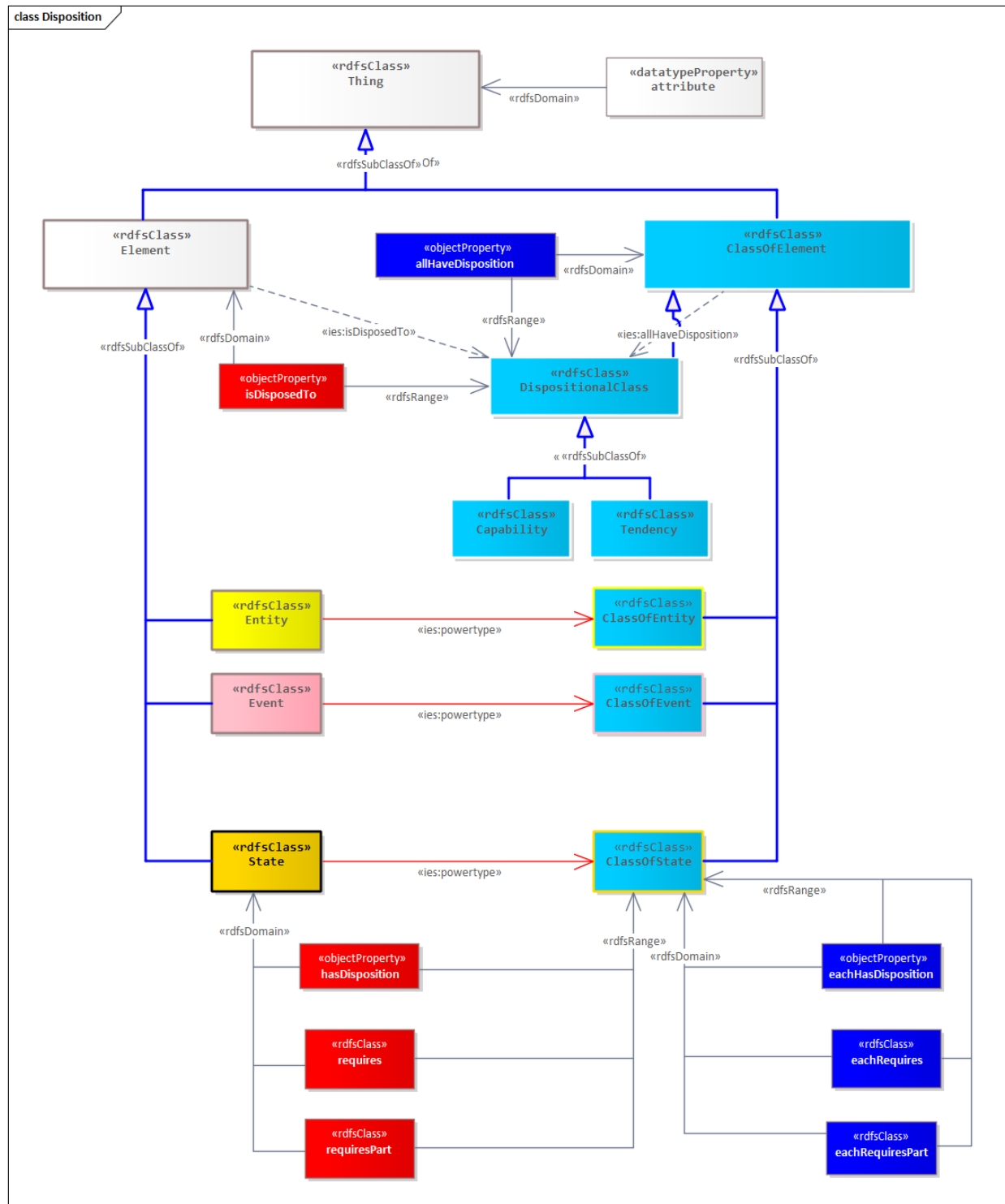


Figure 17 The proposed three pairs of relationships

Turtle definitions

```
# --- Disposition properties ---

ies:hasDisposition a owl:ObjectProperty ;
  rdfs:label "has Disposition"@en-GB ;
  rdfs:comment ""Asserts that a State (or Entity, which is also a State in
IES)
    can, in some possible world, have a temporal part that is a member of
the
    specified ClassOfState. This is the individual-level disposition
    assertion.""@en-GB ;
  rdfs:domain ies:State ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:relationship .

ies:eachHasDisposition a owl:ObjectProperty ;
  rdfs:label "each Has Disposition"@en-GB ;
  rdfs:comment ""Asserts that every member of the domain ClassOfState can,
    in some possible world, have a temporal part that is a member of the
    range ClassOfState. This is the class-level disposition
    assertion.""@en-GB ;
  rdfs:domain ies:ClassOfState ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:relationship .

# --- Cross-entity dependency properties ---

ies:requires a owl:ObjectProperty ;
  rdfs:label "requires"@en-GB ;
  rdfs:comment ""Asserts that for a particular State to exercise a
    disposition, it requires the temporal co-occurrence of at least one
    member of the specified ClassOfState. This is the individual-level
    cross-entity dependency assertion.""@en-GB ;
  rdfs:domain ies:State ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:relationship .

ies:eachRequires a owl:ObjectProperty ;
  rdfs:label "each Requires"@en-GB ;
  rdfs:comment ""Asserts that every member of the domain ClassOfState
    requires the temporal co-occurrence of at least one member of the
    range ClassOfState. This is the class-level cross-entity dependency
    assertion.""@en-GB ;
  rdfs:domain ies:ClassOfState ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:relationship .

# --- Whole-part dependency properties ---
```

```

ies:requiresPart a owl:ObjectProperty ;
  rdfs:label "requires Part"@en-GB ;
  rdfs:comment ""Asserts that a particular State requires, as a
    mereological part, at least one member of the specified ClassOfState.
    This is the individual-level whole-part dependency
    assertion.""@en-GB ;
  rdfs:domain ies:State ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:requires .

ies:eachRequiresPart a owl:ObjectProperty ;
  rdfs:label "each Requires Part"@en-GB ;
  rdfs:comment ""Asserts that every member of the domain ClassOfState has,
    as a mereological part, at least one member of the range ClassOfState
    .
    This is the class-level whole-part dependency assertion.""@en-GB ;
  rdfs:domain ies:ClassOfState ;
  rdfs:range ies:ClassOfState ;
  rdfs:subPropertyOf ies:eachRequires .

```

B.1 Set Theoretic Description

The new properties express the same set-theoretic relationships described in §4 and formalised for the current encoding in §5.1.3. The difference is in how these relationships are encoded in RDF, and in the explicit use of `ClassOfState` for both domain and range.

B.1.2 Individual-level disposition

Let x be a State (or Entity). Let R be a `ClassOfState`.

Define the set of all possible states of x :

$$Sx = \{ s \mid s \text{ isStateOf } x \text{ in some possible world } w \}$$

The assertion x hasDisposition R holds if and only if:

$$Sx \cap R \neq \emptyset$$

Equivalently: $\exists s \mid (s \text{ isStateOf } x) \wedge (s \in R)$

That is, there exists at least one possible world in which x has a state that is a member of R .

B.1.3 Class-level disposition

Let X be a `ClassOfState`. Let R be a `ClassOfState`.

The assertion X eachHasDisposition R holds if and only if:

$$\forall x \in X, Sx \cap R \neq \emptyset$$

Equivalently: $\forall x \in X (x \text{ hasDisposition } R)$

That is, for every member x of X , there exists at least one possible world in which x has a state that is a member of R .

Dependency semantics

The dependency properties express constraints on the co-occurrence of States.

Cross-entity dependency. The assertion x requires R (where x is a State and R is a ClassOfState) holds if and only if:

For x to exercise a disposition, $\exists y \in R \mid y$ temporally co-occurs with x

At the class level, X eachRequires R holds if and only if:

$\forall x \in X$, for x to exercise a disposition, $\exists y \in R \mid y$ temporally co-occurs with x

Whole-part dependency. The assertion x requiresPart R (where x is a State and R is a ClassOfState) holds if and only if:

$\exists s \in R \mid s$ isPartOf x

At the class level, R_1 eachRequiresPart R_2 holds if and only if:

$\forall s_1 \in R_1, \exists s_2 \in R_2 \mid s_2$ isPartOf s_1

Since isPartOf entails spatio-temporal containment, this is a stronger constraint than requires: the required State must be a mereological part of the dependent State, not merely a temporally co-occurring State.

§3.5 Whole-part dependencies

Requirement recap: The Eurofighter's Mach 2 disposition depends on both EJ200 engines being functional.

This is where the new properties extend beyond what the current encoding can express. The requiresPart and eachRequiresPart properties provide a dedicated mechanism for whole-part dependency assertions.

B.1.4 The dependency in spacetime

Consider a spacetime view of Fighter XYZ-123 and its installed propulsion system PS-001. The fighter alternates between periods when propulsion function is required (flight) and when it is not (on the ground). During flight, the fighter's operational state depends on PS-001 being in an installed, operational state. If PS-001 becomes non-operational while propulsion function is required, the operational state of Fighter XYZ-123 cannot exist during that period. The whole's operational state is temporally bounded by the part's operational state.

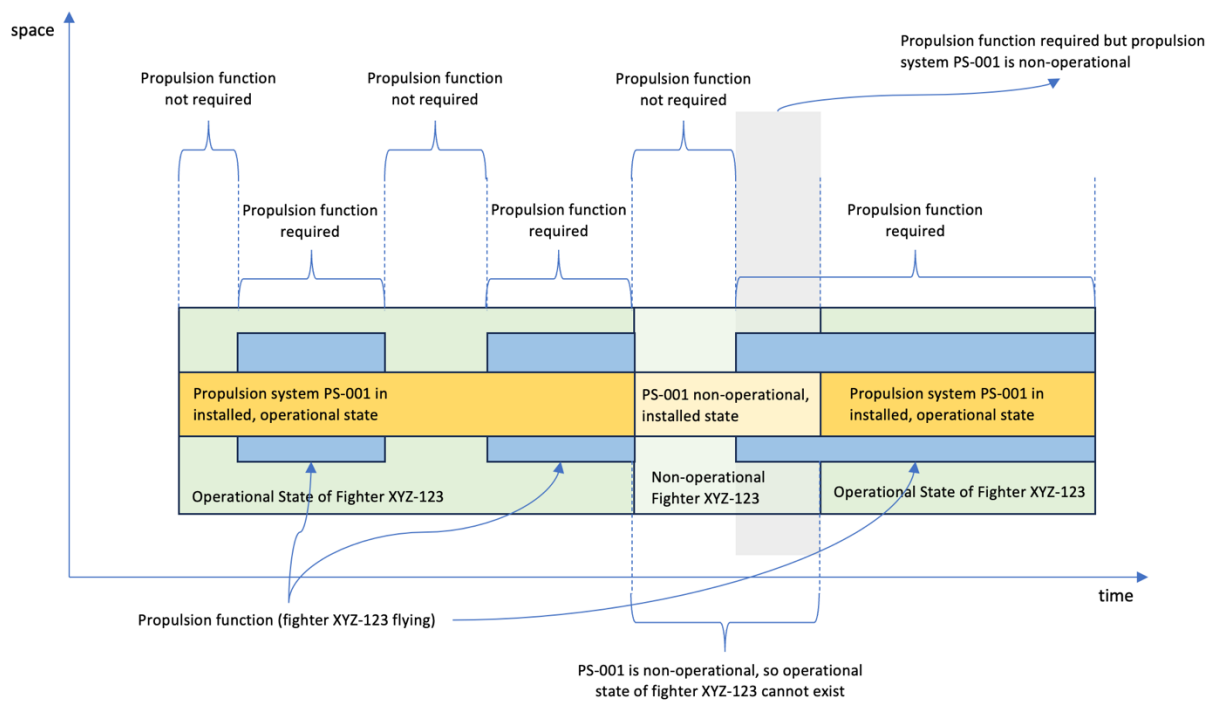


Figure 18 Dependency between the whole and its parts

B.1.5 Modelling the dependency

Using the proposed properties, we can record this as shown in the UML class diagram.

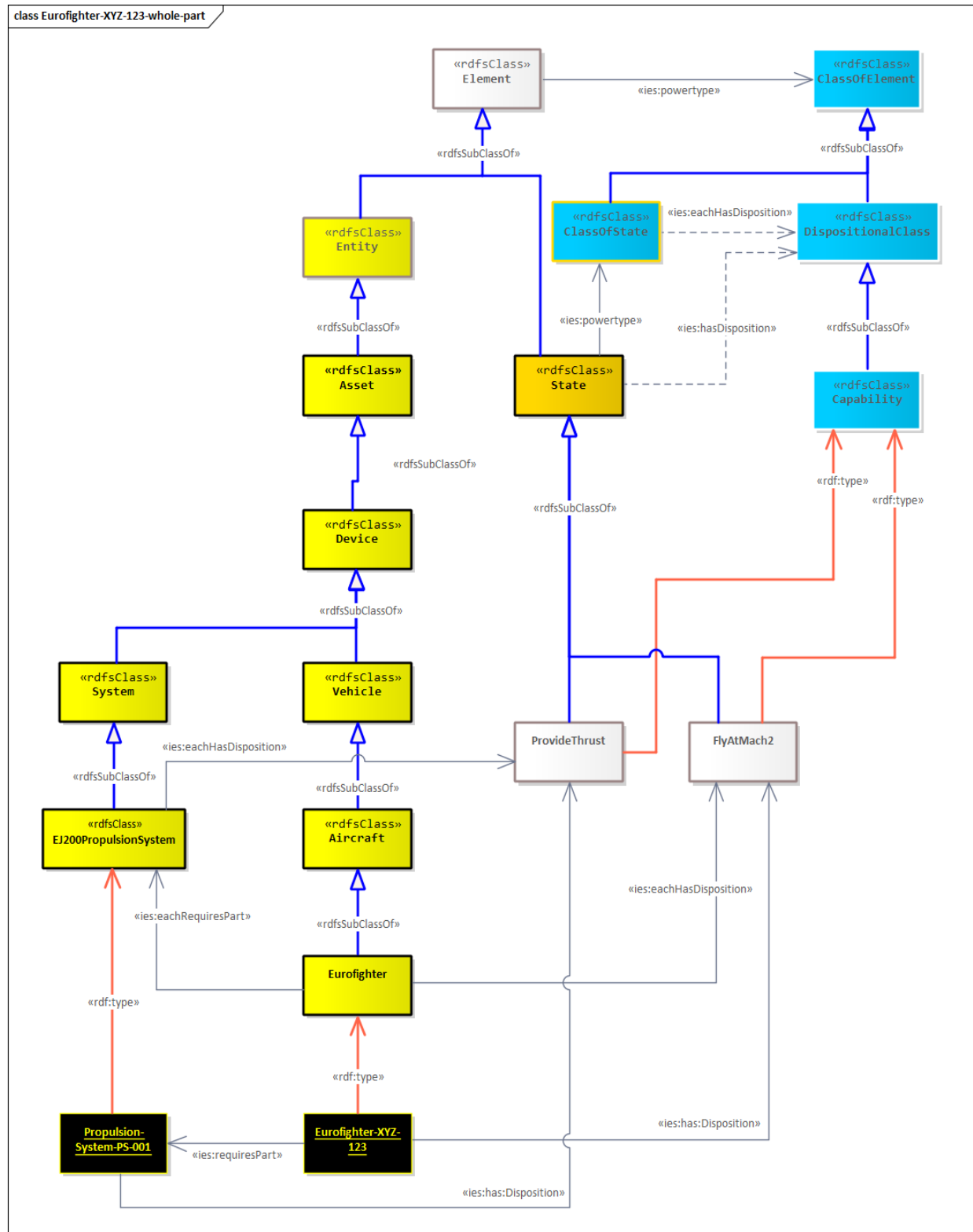


Figure 19 Whole-part capability dependency

First, the class structure for the fighter and its propulsion system:

```
# Eurofighter state classes
ex:EurofighterState a ies:ClassOfState ;
  rdfs:label "Eurofighter State"@en-GB ;
  rdfs:subClassOf ies:VehicleState .

ex:OperationalEurofighterState a ies:ClassOfState ;
  rdfs:label "Operational Eurofighter State"@en-GB ;
  rdfs:subClassOf ex:EurofighterState .

ex:NonOperationalEurofighterState a ies:ClassOfState ;
  rdfs:label "Non-Operational Eurofighter State"@en-GB ;
  rdfs:subClassOf ex:EurofighterState .

# Individual fighter state classes
ex:FighterXYZ-123-State a ies:ClassOfState ;
  rdfs:label "Fighter XYZ-123 State"@en-GB ;
  rdfs:subClassOf ex:EurofighterState .

ex:OperationalStateOfFighterXYZ-123 a ies:ClassOfState ;
  rdfs:label "Operational State of Fighter XYZ-123"@en-GB ;
  rdfs:subClassOf ex:FighterXYZ-123-State ,
    ex:OperationalEurofighterState .

# EJ200 propulsion system state classes
ex:EurojetEJ200PropulsionSystemState a ies:ClassOfState ;
  rdfs:label "Eurojet EJ200 Propulsion System State"@en-GB ;
  rdfs:subClassOf ies:DeviceState .

ex:OperationalStateOfEJ200PS a ies:ClassOfState ;
  rdfs:label "Operational State of EJ200 Propulsion System"@en-GB ;
  rdfs:subClassOf ex:EurojetEJ200PropulsionSystemState .

# Individual propulsion system state classes
ex:EJ200-PS-001-State a ies:ClassOfState ;
  rdfs:label "EJ200-PS-001 State"@en-GB ;
  rdfs:subClassOf ex:EurojetEJ200PropulsionSystemState .

ex:OperationalStateOfEJ200-PS-001 a ies:ClassOfState ;
  rdfs:label "Operational State of EJ200-PS-001"@en-GB ;
  rdfs:subClassOf ex:EJ200-PS-001-State ,
    ex:OperationalStateOfEJ200PS .
```

Second, the disposition assertions:

```
# The Capability instances
ex:FlyAtMach2 a ies:Capability ;
  rdfs:label "Fly at Mach 2"@en-GB ;
  rdfs:subClassOf ies:State .
```

```

ex:ProvideThrust a ies:Capability ;
  rdfs:label "Provide Thrust"@en-GB ;
  rdfs:subClassOf ies:State .

# Class-level disposition assertions
ex:OperationalEurofighterState
  ies:eachHasDisposition ex:FlyAtMach2 .

ex:OperationalStateOfEJ200PS
  ies:eachHasDisposition ex:ProvideThrust .

# Instance-level disposition assertions
data:FighterXYZ-123 a ies:Aircraft ;
  ies:hasDisposition ex:FlyAtMach2 .

data:PS-001 a ies:Device ;
  ies:isPartOf data:FighterXYZ-123 ;
  ies:hasDisposition ex:ProvideThrust .

```

Third, the dependency itself. At the class level, every operational state of Fighter XYZ-123 must contain, as a part, an operational state of its installed propulsion system:

```

# Class-level: every operational state of Fighter XYZ-123 has, as a part,
# an operational state of its EJ200 propulsion system
ex:OperationalStateOfFighterXYZ-123
  ies:eachRequiresPart ex:OperationalStateOfEJ200-PS-001 .

```

This assertion captures the dependency: if there is no state of PS-001 that is a member of `ex:OperationalStateOfEJ200-PS-001` during some period, then the `eachRequiresPart` constraint cannot be satisfied, and consequently no member of `ex:OperationalStateOfFighterXYZ-123` can exist during that period either.

The same relationship can be applied at the type level:

```

# The type-level dependency: every operational Eurofighter state has
# an operational EJ200 propulsion system state as a part
ex:OperationalEurofighterState
  ies:eachRequiresPart ex:OperationalStateOfEJ200PS .

```

B.1.6 Actualisation and temporal coincidence

When Fighter XYZ-123 actually flies at Mach 2, this is modelled as an actual state with the whole-part dependency satisfied:

```

# The fighter's actual Mach 2 flight state
data:FighterXYZ-123-FlyingMach2-20240717 a ex:OperationalStateOfFighterXYZ-123 ;
  ies:hasDisposition ex:FlyAtMach2 ;
  ies:isStateOf data:FighterXYZ-123 ;

```

```

    ies:requiresPart ex:OperationalStateOfEJ200-PS-001 .

# The propulsion system's actual operational state – a part of the
# fighter's operational state
data:operational-EJ200-PS-001-20240717 a ex:OperationalStateOfEJ200PS ;
    ies:isStateOf data:PS-001 ;
    ies:hasDisposition ex:ProvideThrust ;
    ies:isPartOf data:FighterXYZ-123-FlyingMach2-20240717 .

# Temporal bounding of the fighter's state
data:FlyingMach2Start a ies:BoundingState ;
    ies:isStartOf data:FighterXYZ-123-FlyingMach2-20240717 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-07-17T09:30:00Z"^^xsd:dateTime ] .

data:FlyingMach2End a ies:BoundingState ;
    ies:isEndOf data:FighterXYZ-123-FlyingMach2-20240717 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-07-17T10:15:00Z"^^xsd:dateTime ] .

```

The `ies:isPartOf` assertion between the propulsion system’s operational state and the fighter’s flight state is the instance-level realisation of what `eachRequiresPart` asserts at the class level. Since `isPartOf` is a mereological relationship, the part’s spatio-temporal extent is contained within the whole’s spatio-temporal extent — temporal coincidence is guaranteed by the mereological semantics.

§3.6 Cross-entity dependencies

Requirement recap: The Eurofighter’s extended-range disposition depends on the Voyager tanker’s refuelling disposition.

This extends §3.5 to cross-entity dependencies where the entities are not in a part-whole relationship. The Voyager tanker is not a part of the Eurofighter — they are independent entities whose dispositions become interdependent in certain operational contexts. Unlike the propulsion system case, this is not a mereological dependency: the Voyager’s refuelling state does not occupy the same space as the Eurofighter’s flight state, so `requiresPart` and `eachRequiresPart` would stretch the mereological semantics beyond their proper scope.

What is needed is the weaker pair of relationships: not “*has as a part*” but “*requires the co-occurrence of*.” This is expressed using `requires` (individual-level) and `eachRequires` (class-level).

The distinction is important. The current encoding’s Event-based approach (shown in §5.1.5, §3.6) records what *happened* — both entities participating in a refuelling event. The `eachRequires` relationship records what *must co-occur* for a disposition to be exercisable. The former is descriptive (what did occur); the latter is prescriptive (what must obtain). This conceptual difference — between recording co-participation and asserting dependency between potentials — is precisely what motivates the dedicated dependency properties.

B.1.7 Recording the dispositions

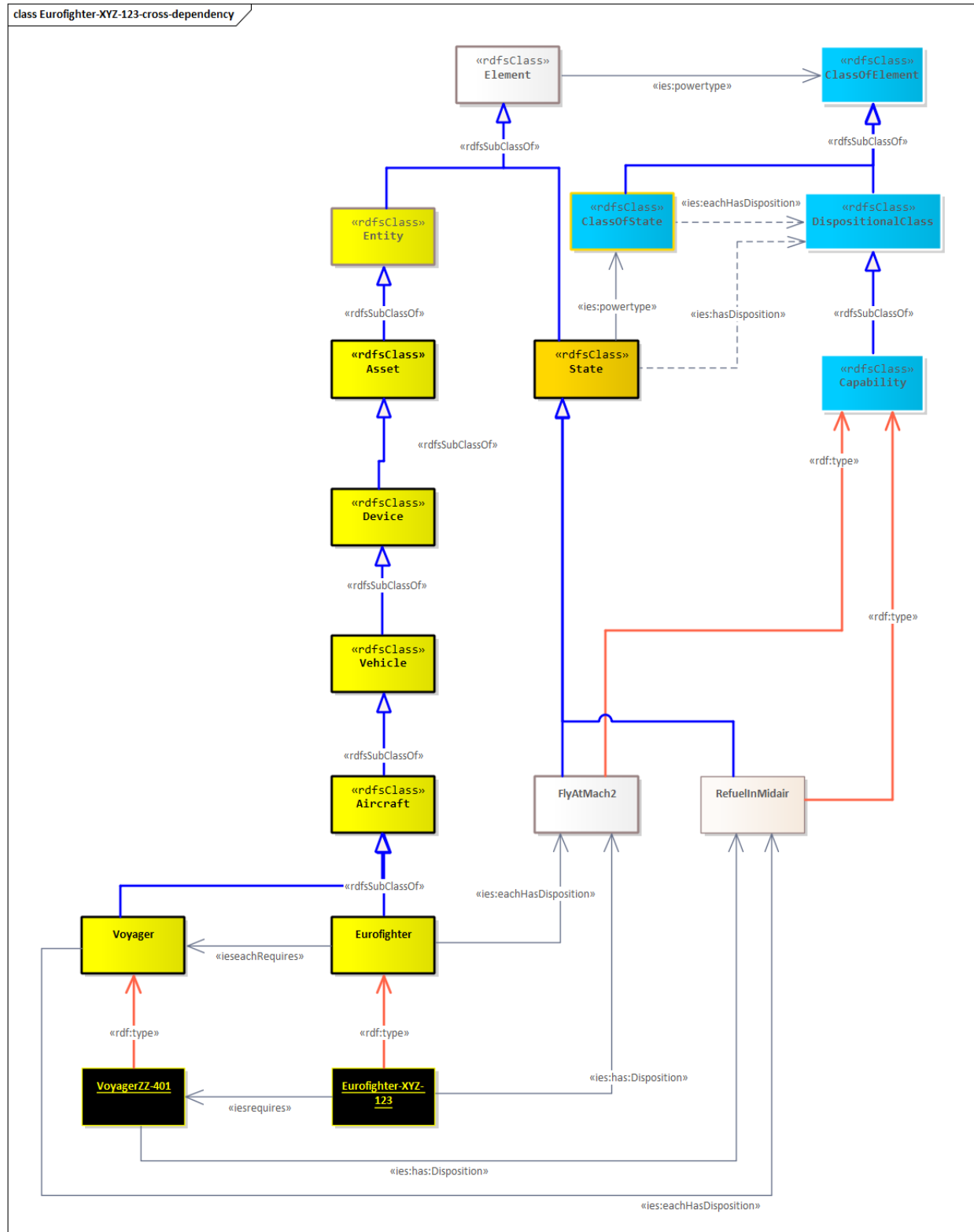


Figure 20 UML recording of cross-dependency dispositions

Each entity's dispositions are recorded independently:

```
# The Eurofighter's extended-range disposition
ex:FlyExtendedRange a ies:Capability ;
    rdfs:label "Perform Extended-Range Flight"@en-GB ;
    rdfs:subClassOf ies:State .

data:FighterXYZ-123 ies:hasDisposition ex:FlyExtendedRange .

# The Voyager's refuelling disposition
ex:RefuelInMidair a ies:Capability ;
    rdfs:label "Refuel in mid-air"@en-GB ;
    rdfs:subClassOf ies:State .

ex:RefuellingAircraft a ies:ClassOfEntity ;
    rdfs:label "Refuelling Aircraft"@en-GB ;
    rdfs:subClassOf ies:Aircraft ;
    ies:eachHasDisposition ex:RefuelInMidair .

data:VoyagerZZ-401 a ex:RefuellingAircraft ;
    ies:hasDisposition ex:RefuelInMidair .
```

B.1.8 Recording the dependency

At the class level, the co-dependency is expressed using `eachRequires`:

```
# Class-level: every extended-range flight state requires a co-occurring
# refuelling aircraft state
ex:FlyExtendedRange
    ies:eachRequires ex:RefuellingAircraft .
```

This reads: every member of `ex:FlyExtendedRange` requires the co-occurrence of a member of `ex:RefuellingAircraft`. In natural language: an aircraft performing an extended-range flight requires a refuelling aircraft — one that has the disposition to refuel in mid-air — during the same period.

Note that `ex:RefuellingAircraft` is typed in the `ies:ClassOfEntity`, hierarchy, which satisfies the `ies:ClassOfState` range of `eachRequires` through the Entity-as-State subclass chain.

At the individual level:

```
# Individual-level: Fighter XYZ-123 requires the Voyager tanker
# for its extended-range operations
data:FighterXYZ-123 ies:requires data:VoyagerZZ-401 .
```

Temporal coincidence. The `eachRequires` and `requires` relationships assert temporal co-occurrence: every member of the domain class (or the specific *State*) requires a *temporally overlapping* member of the range class. This temporal semantics is intrinsic to the meaning of “co-occurrence” and distinguishes these relationships from a weaker claim that members of

both classes merely exist at some point. Since `eachRequiresPart` is a sub-property of `eachRequires`, mereological containment implies co-occurrence, but not vice versa. This gives a coherent family of dependency relationships with increasing strength.

B.1.9 Recording the actualisation

When the dependency is satisfied in the actual world, both entities participate in a shared Event. IES's existing Event and EventParticipant patterns record this realisation:

```
# The extended-range sortie event, temporally bounded
data:Sortie-2024-08-15 a ies:Event ;
    rdfs:label "Extended-range sortie 2024-08-15"@en-GB .

data:SortieStart a ies:BoundingState ;
    ies:isStartOf data:Sortie-2024-08-15 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-08-15T06:00:00Z"^^xsd:dateTime ] .

data:SortieEnd a ies:BoundingState ;
    ies:isEndOf data:Sortie-2024-08-15 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-08-15T14:00:00Z"^^xsd:dateTime ] .

# The Eurofighter's participation
data:XYZ-123-in-ExtendedRangeFlight a ies:State ;
    ies:isStateOf data:FighterXYZ-123 ;
    ies:hasDisposition ex:FlyExtendedRange ;
    ies:requires ex:RefuellingAircraft ;
    ies:isParticipantIn data:Sortie-2024-08-15 .

# The Voyager's participation
data:ZZ-401-Refuelling a ies:State ;
    ies:isStateOf data:VoyagerZZ-401 ;
    ies:hasDisposition ex:RefuelInMidair ;
    ies:isParticipantIn data:Sortie-2024-08-15 .
```

The Event's temporal bounds implicitly bound all its participants: both the Eurofighter's extended-range flight state and the Voyager's refuelling state fall within the Event's temporal extent. The class-level assertions (`eachHasDisposition` and `eachRequires`) establish the general pattern; the individual-level assertions on the actual participating states reinforce this. Together, the prescriptive class-level relationships and the descriptive individual-level assertions provide a complete picture — what *must* co-occur for a disposition to be exercised, and what *did* co-occur in this particular event.

Appendix C Degradation Spectra and Capability Enhancement

C.1 Introduction

§3.5 introduces whole-part dependencies through a binary example: the Eurofighter's Mach 2 disposition depends on its propulsion system being operational. If the propulsion system is non-operational, the Mach 2 disposition is lost. §5.2.5 encodes this dependency using `eachRequiresPart` and `requiresPart`, which assert that the whole's operational state must contain, as a part, an operational state of the relevant component.

In practice, however, degradation is rarely a binary event. A complex asset does not simply flip between "fully operational" and "non-operational." It may pass through a succession of intermediate states — each with a different set of dispositions — as components fail, degrade, or are removed. Conversely, an asset may gain new dispositions through component upgrades, additions, or enhancements. What we colloquially call a "degradation spectrum" is the partially ordered set of operational states between fully operational and fully non-operational, and what we might call an "enhancement spectrum" is its upward counterpart: states with dispositions beyond the original design baseline.

This appendix demonstrates that the disposition pattern introduced in §5 — the combination of `eachHasDisposition`, `hasDisposition`, `eachRequiresPart`, and `requiresPart` — handles graduated degradation and enhancement without any additional ontological machinery. The key insight is that each level of degradation or enhancement is simply a distinct `ClassOfState` with its own disposition assertions and its own part-state requirements. The "spectrum" emerges from the subclass relationships between these `ClassOfState` definitions and the different Capability instances they are associated with.

C.2 The Degradation Spectrum as a Lattice of States

Consider an asset whose operational condition can be characterised at several levels of detail. At the coarsest level, it is either operational or non-operational. At a finer level, "operational" subdivides into "fully operational" and one or more degraded modes — states in which the asset retains some dispositions but has lost others. These states form a partial order: fully operational is the most capable state, each degraded mode is less capable, and non-operational is the least capable.

In the disposition pattern, each such level corresponds to a `ClassOfState`. Each `ClassOfState` carries its own `eachHasDisposition` assertions — declaring what members of that class can do — and its own `eachRequiresPart` assertions — declaring what part-states must be present for the whole to be in that state. The subclass relationships between these `ClassOfState` definitions encode the partial order: a more capable state class is a subclass of a less capable one (every fully operational state is also an operational state, but not vice versa).

No new properties or meta-classes are required. The spectrum is an application of the existing pattern at multiple levels of granularity.

C.3 Worked Example: Eurofighter Propulsion Degradation

C.3.1 The scenario

Fighter XYZ-123 has twin EJ200 jet engines. The propulsion system PS-001 can be in one of three broad conditions:

1. **Fully operational** — both engines (EJ200-001 and EJ200-002) are installed and operational. The fighter can fly at Mach 2.
2. **Degraded ("limp mode")** — one engine has failed; the other remains operational. The fighter can still fly (for example, to return to base) but cannot achieve Mach 2.
3. **Non-operational** — both engines have failed or the propulsion system is otherwise non-functional. The fighter cannot fly.

These three conditions of the propulsion system produce three corresponding conditions of the fighter as a whole: a Mach 2 state, a limp-mode state, and a grounded state. Each has different dispositions and different part-state requirements.

C.3.2 State classes for the propulsion system

The propulsion system's conditions are modelled as a hierarchy of `ClassOfState` definitions. §5.2.5 already establishes `ex:OperationalStateOfEJ200PS` and `ex:OperationalStateOfEJ200-PS-001`. We extend this with finer-grained state classes:

```
# --- Propulsion system state classes ---

# The broadest class: any state of an EJ200 propulsion system
ex:EurojetEJ200PropulsionSystemState a ies:ClassOfState ;
    rdfs:label "Eurojet EJ200 Propulsion System State"@en-GB ;
    rdfs:subClassOf ies:DeviceState .

# Operational: at least one engine functional — can provide some thrust
ex:OperationalStateOfEJ200PS a ies:ClassOfState ;
    rdfs:label "Operational State of EJ200 Propulsion System"@en-GB ;
    rdfs:subClassOf ex:EurojetEJ200PropulsionSystemState .

# Fully operational: both engines functional — can provide full thrust
ex:FullyOperationalStateOfEJ200PS a ies:ClassOfState ;
    rdfs:label "Fully Operational State of EJ200 Propulsion System"@en-GB ;
    rdfs:subClassOf ex:OperationalStateOfEJ200PS .

# Degraded: one engine operational, one failed — reduced thrust
ex:DegradedStateOfEJ200PS a ies:ClassOfState ;
    rdfs:label "Degraded State of EJ200 Propulsion System"@en-GB ;
    rdfs:comment "One engine operational, one failed. Sufficient thrust for
flight but not for Mach 2."@en-GB ;
    rdfs:subClassOf ex:OperationalStateOfEJ200PS .

# Non-operational: no engines functional
ex:NonOperationalStateOfEJ200PS a ies:ClassOfState ;
    rdfs:label "Non-Operational State of EJ200 Propulsion System"@en-GB ;
```

```
rdfs:subClassOf ex:EurojetEJ200PropulsionSystemState .
```

The subclass hierarchy encodes the partial order:

- Every FullyOperationalStateOfEJ200PS is an OperationalStateOfEJ200PS (both engines working implies at least one engine working).
- Every DegradedStateOfEJ200PS is also an OperationalStateOfEJ200PS (one engine working implies at least one engine working).
- NonOperationalStateOfEJ200PS is *not* a subclass of OperationalStateOfEJ200PS.

The same structure applies at the individual level for PS-001, using the naming convention from §5.2.5:

```
# --- Individual propulsion system state classes ---

ex:FullyOperationalStateOfEJ200-PS-001 a ies:ClassOfState ;
  rdfs:label "Fully Operational State of EJ200-PS-001"@en-GB ;
  rdfs:subClassOf ex:OperationalStateOfEJ200-PS-001 ,
    ex:FullyOperationalStateOfEJ200PS .

ex:DegradedStateOfEJ200-PS-001 a ies:ClassOfState ;
  rdfs:label "Degraded State of EJ200-PS-001"@en-GB ;
  rdfs:subClassOf ex:OperationalStateOfEJ200-PS-001 ,
    ex:DegradedStateOfEJ200PS .

ex:NonOperationalStateOfEJ200-PS-001 a ies:ClassOfState ;
  rdfs:label "Non-Operational State of EJ200-PS-001"@en-GB ;
  rdfs:subClassOf ex:EJ200-PS-001-State ,
    ex:NonOperationalStateOfEJ200PS .
```

C.3.3 State classes for the fighter

The fighter's conditions mirror the propulsion system's, since the propulsion system is the determining factor for the dispositions in question:

```
# --- Fighter state classes: degradation spectrum ---

# Operational: the fighter can fly (at least one engine working)
# Already defined in §5.2.5:
#   ex:OperationalEurofighterState
#   ex:OperationalStateOfFighterXYZ-123

# Fully operational: the fighter can fly at Mach 2
ex:FullyOperationalEurofighterState a ies:ClassOfState ;
  rdfs:label "Fully Operational Eurofighter State"@en-GB ;
  rdfs:subClassOf ex:OperationalEurofighterState .

ex:FullyOperationalStateOfFighterXYZ-123 a ies:ClassOfState ;
  rdfs:label "Fully Operational State of Fighter XYZ-123"@en-GB ;
  rdfs:subClassOf ex:OperationalStateOfFighterXYZ-123 ,
    ex:FullyOperationalEurofighterState .

# Degraded ("limp mode"): the fighter can fly, but not at Mach 2
```

```

ex:DegradedEurofighterState a ies:ClassOfState ;
    rdfs:label "Degraded Eurofighter State"@en-GB ;
    rdfs:comment "The fighter can fly but with reduced performance. Commonly
referred to as 'limp mode'."@en-GB ;
    rdfs:subClassOf ex:OperationalEurofighterState .

ex:DegradedStateOfFighterXYZ-123 a ies:ClassOfState ;
    rdfs:label "Degraded State of Fighter XYZ-123"@en-GB ;
    rdfs:subClassOf ex:OperationalStateOfFighterXYZ-123 ,
        ex:DegradedEurofighterState .

```

C.3.4 Capability instances

The dispositions available at each level are modelled as distinct Capability instances:

```

# Already defined in §5.2.5:
#   ex:FlyAtMach2 a ies:Capability .
#   ex:ProvideThrust a ies:Capability .

# New Capability for degraded-mode flight
ex:FlyDegraded a ies:Capability ;
    rdfs:label "Fly in Degraded Mode"@en-GB ;
    rdfs:comment "Flight at reduced performance on a single engine.
Sufficient to return to base or reach a safe landing zone."@en-GB ;
    rdfs:subClassOf ies:State .

```

The relationship between the Capabilities reflects the degradation: `FlyAtMach2` represents the full potential; `FlyDegraded` represents the reduced potential available in limp mode. Note that `FlyAtMach2` and `FlyDegraded` are not in a subclass relationship — they are distinct Capabilities representing qualitatively different kinds of state. A state of flying at Mach 2 is not a state of limping back to base, and vice versa.

C.3.5 Disposition and dependency assertions

With the state classes and Capabilities defined, the disposition pattern from §5.2 is applied at each level of the spectrum.

Fully operational: Mach 2 flight. The fully operational state of the fighter has the Mach 2 disposition and requires a fully operational propulsion system:

```

# Class-level dispositions
ex:FullyOperationalEurofighterState
    ies:eachHasDisposition ex:FlyAtMach2 .

ex:FullyOperationalStateOfEJ200PS
    ies:eachHasDisposition ex:ProvideThrust .

# Class-level dependency: Mach 2 requires full propulsion
ex:FullyOperationalEurofighterState
    ies:eachRequiresPart ex:FullyOperationalStateOfEJ200PS .

# Individual-level dependency

```

```
ex:FullyOperationalStateOfFighterXYZ-123
  ies:eachRequiresPart ex:FullyOperationalStateOfEJ200-PS-001 .
```

Degraded: limp-mode flight. The degraded state of the fighter has only the degraded-flight disposition and requires a degraded propulsion system (at least one engine operational):

```
# Class-level dispositions
ex:DegradedEurofighterState
  ies:eachHasDisposition ex:FlyDegraded .

# Class-level dependency: limp mode requires at least degraded propulsion
ex:DegradedEurofighterState
  ies:eachRequiresPart ex:OperationalStateOfEJ200PS .

# Individual-level dependency
ex:DegradedStateOfFighterXYZ-123
  ies:eachRequiresPart ex:OperationalStateOfEJ200-PS-001 .
```

Note the asymmetry in the dependency assertions. The fully operational state requires `FullyOperationalStateOfEJ200PS` (both engines working); the degraded state requires only `OperationalStateOfEJ200PS` (at least one engine working). This is precisely how the disposition pattern distinguishes between the two levels of the spectrum: different `ClassOfState` definitions in the range of `eachRequiresPart` capture different part-state requirements.

Non-operational: grounded. The non-operational state has no flight dispositions and no dependency assertions — there is nothing to depend on:

```
# The non-operational state has no eachHasDisposition assertions
# for flight Capabilities. A fighter in this state cannot fly.
ex:NonOperationalEurofighterState a ies:ClassOfState ;
  rdfs:label "Non-Operational Eurofighter State"@en-GB ;
  rdfs:subClassOf ex:EurofighterState .
```

The absence of a disposition assertion for `FlyAtMach2` or `FlyDegraded` on `NonOperationalEurofighterState` correctly represents the lack of flight potential. Under open-world semantics, this absence does not *prove* the fighter cannot fly — but it is the established IES convention for representing that no such disposition has been asserted.

C.3.6 Disposition inheritance through the subclass hierarchy

A property of the subclass hierarchy deserves explicit attention. Because `FullyOperationalEurofighterState` is a subclass of `OperationalEurofighterState`, any disposition asserted on `OperationalEurofighterState` is inherited by the fully operational subclass. If we were to assert:

```
# Hypothetical: asserting a disposition on the broader operational class
ex:OperationalEurofighterState
  ies:eachHasDisposition ex:FlyDegraded .
```

then every member of `FullyOperationalEurofighterState` would also inherit the degraded-flight disposition via the subclass chain. This is semantically correct: an aircraft that can fly at Mach 2 can certainly also fly in a degraded mode (it simply would not need to). The inheritance reflects the monotonic nature of class-level disposition assertions: more specific states may have *additional* dispositions, but they do not lose the dispositions of their parent classes.

The worked example above places `eachHasDisposition ex:FlyDegraded` on `DegradedEurofighterState` rather than on `OperationalEurofighterState`. This is a modelling choice, not a constraint imposed by the pattern. If the data producer wishes to assert that all operational Eurofighters — whether fully operational or degraded — have the degraded-flight disposition, the assertion should be placed on `OperationalEurofighterState`. If the data producer wishes to assert the degraded-flight disposition only when the fighter is actually in a degraded state, the assertion should be placed on `DegradedEurofighterState`. The pattern supports both.

C.3.7 Instance data: a degradation event

When Fighter XYZ-123 actually enters limp mode — for example, following an engine failure during flight — the instance data records the transition:

```
# The fighter's degraded-mode flight state
data:FighterXYZ-123-LimpMode-20240717 a ex:DegradedStateOfFighterXYZ-123 ;
    ies:isStateOf data:FighterXYZ-123 ;
    ies:hasDisposition ex:FlyDegraded ;
    ies:requiresPart data:degraded-EJ200-PS-001-20240717 .

# The propulsion system's degraded state — a part of the fighter's
# degraded state
data:degraded-EJ200-PS-001-20240717 a ex:DegradedStateOfEJ200-PS-001 ;
    ies:isStateOf data:PS-001 ;
    ies:isPartOf data:FighterXYZ-123-LimpMode-20240717 .

# Temporal bounding of the degraded state
data:LimpModeStart a ies:BoundingState ;
    ies:isStartOf data:FighterXYZ-123-LimpMode-20240717 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-07-17T10:15:00Z"^^xsd:dateTime ] .

data:LimpModeEnd a ies:BoundingState ;
    ies:isEndOf data:FighterXYZ-123-LimpMode-20240717 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2024-07-17T10:52:00Z"^^xsd:dateTime ] .
```

Compare this with the Mach 2 flight instance data in §5.2.5. The structure is identical — the only differences are the `ClassOfState` types and the `Capability` instance. A consuming system that understands the state class hierarchy can determine, from the `rdf:type` assertion alone, that this is a degraded state with reduced dispositions. A system that also understands the `eachHasDisposition` assertions on the class hierarchy can infer which dispositions are available and which are not.

[Diagram placeholder: A spacetime diagram showing the temporal succession of states: Fighter XYZ-123 in a fully operational state (flying at Mach 2), followed by an engine failure event, followed by a degraded state (limp mode, flying on one engine), followed by landing. The propulsion system's states (fully operational, then degraded) should be shown as parts of the fighter's states, with the transition point marking the change in dispositions.]

C.4 Interaction with Measured Dispositions

§3.7 notes that a degraded engine state might reduce the maximum speed potential from Mach 2 to Mach 1.2 rather than eliminating the speed disposition entirely. This is the point at which degradation spectra and measured dispositions intersect.

The worked example in §B.3 treats degradation qualitatively: the fighter either has the Mach 2 disposition or it does not. In practice, the degraded-mode speed capability is a *quantified* disposition — the fighter can fly at *up to Mach 1.2* rather than at *up to Mach 2*. Recording this requires associating a measured value with the disposition assertion.

The disposition pattern provides the structural foundation for this association. The degraded state class (`DegradedEurofighterState`) and its disposition assertion (`eachHasDisposition` ex:FlyDegraded) establish the qualitative framework. To add quantification, one would associate a Measure or Attribute with the Capability or with the disposition relationship itself, using IES's existing measurement patterns:

```
# Sketch: associating a speed limit with the degraded-flight Capability.
# The full encoding of measured dispositions is identified as future
# work in §3.7 and §7.4.
ex:FlyDegraded a ies:Capability ;
  rdfs:label "Fly in Degraded Mode"@en-GB ;
  rdfs:subClassOf ies:State .

data:FighterXYZ-123-LimpMode-20240717
  ies:hasDisposition ex:FlyDegraded .

# The measured characteristic of this particular degraded state
data:LimpModeSpeedLimit a ies:Measure ;
  ies:isStateOf data:FighterXYZ-123-LimpMode-20240717 ;
  ies:hasValue [ a ies:ValueInKilometresPerHour ;
    ies:representationValue "1,469.7"^^xsd:string ] .
```

This sketch is illustrative, not normative. The full encoding of measured and quantified dispositions — including the relationship between design-level and assessed values, the interaction with the IES Attribute and Measure patterns, and the combination of quantification with dependency structures — remains future work as stated in §3.7 and §7.4. The key observation for this appendix is that degradation spectra create the structural context in which measured dispositions become most relevant: it is precisely at the intermediate states of the spectrum that "how much" matters most.

C.5 Capability Enhancement

The degradation spectrum has an upward counterpart. Where degradation reduces the set of available dispositions, enhancement extends it — adding new dispositions that were not part of the original design baseline. The disposition pattern handles enhancement symmetrically, using exactly the same mechanism as for degradation but with the subclass hierarchy extending above the original fully operational state rather than below it.

C.5.1 The scenario

Fighter XYZ-123's propulsion system PS-001 is upgraded: thrust vectoring nozzles (TVN) are fitted to both EJ200 engines. The upgrade gives the fighter a new disposition — enhanced manoeuvrability — that it did not possess before the modification.

C.5.2 Enhanced state classes

The upgrade produces a new state class for both the propulsion system and the fighter:

```
# --- Enhanced propulsion system state ---
ex:EnhancedStateOfEJ200PS a ies:ClassOfState ;
    rdfs:label "Enhanced State of EJ200 Propulsion System (TVN)"@en-GB ;
    rdfs:comment "Both engines operational with thrust vectoring nozzles
fitted."@en-GB ;
    rdfs:subClassOf ex:FullyOperationalStateOfEJ200PS .

ex:EnhancedStateOfEJ200-PS-001 a ies:ClassOfState ;
    rdfs:label "Enhanced State of EJ200-PS-001 (TVN)"@en-GB ;
    rdfs:subClassOf ex:FullyOperationalStateOfEJ200-PS-001 ,
                    ex:EnhancedStateOfEJ200PS .

# --- Enhanced fighter state ---
ex:EnhancedEurofighterState a ies:ClassOfState ;
    rdfs:label "Enhanced Eurofighter State (TVN)"@en-GB ;
    rdfs:comment "Fully operational with thrust vectoring, providing enhanced
manoeuvrability."@en-GB ;
    rdfs:subClassOf ex:FullyOperationalEurofighterState .

ex:EnhancedStateOfFighterXYZ-123 a ies:ClassOfState ;
    rdfs:label "Enhanced State of Fighter XYZ-123 (TVN)"@en-GB ;
    rdfs:subClassOf ex:FullyOperationalStateOfFighterXYZ-123 ,
                    ex:EnhancedEurofighterState .
```

The subclass relationship `EnhancedEurofighterState rdfs:subClassOf FullyOperationalEurofighterState` correctly asserts that every enhanced state is also a fully operational state. The enhanced state inherits all the dispositions of its parent classes — including `FlyAtMach2` from `FullyOperationalEurofighterState` — and adds new ones.

C.5.3 The new disposition

```
ex:EnhancedManoeuvrability a ies:Capability ;
    rdfs:label "Enhanced Manoeuvrability (TVN)"@en-GB ;
```

```

    rdfs:comment "Post-stall manoeuvrability provided by thrust vectoring
nozzles."@en-GB ;
    rdfs:subClassOf ies:State .

# Class-level disposition
ex:EnhancedEurofighterState
    ies:eachHasDisposition ex:EnhancedManoeuvrability .

# Class-level dependency: enhanced manoeuvrability requires an enhanced
# propulsion system
ex:EnhancedEurofighterState
    ies:eachRequiresPart ex:EnhancedStateOfEJ200PS .

# Individual-level dependency
ex:EnhancedStateOfFighterXYZ-123
    ies:eachRequiresPart ex:EnhancedStateOfEJ200-PS-001 .

```

C.5.4 Disposition inheritance in the enhanced case

Through the subclass chain, the enhanced state class inherits dispositions from all its ancestors:

- From `FullyOperationalEurofighterState`: `eachHasDisposition ex:FlyAtMach2`
- From `EnhancedEurofighterState` (itself): `eachHasDisposition ex:EnhancedManoeuvrability`

A member of `EnhancedEurofighterState` therefore has both the Mach 2 flight disposition and the enhanced manoeuvrability disposition. This is the mirror image of degradation: where degradation narrows the set of available dispositions by placing the state in a less capable class, enhancement broadens it by placing the state in a more capable class.

C.6 The Complete Spectrum

Taken together, the degradation and enhancement examples produce a five-level spectrum of `ClassOfState` definitions for the Eurofighter, ordered from most to least capable:

1. `EnhancedEurofighterState` — TVN upgrade fitted; Mach 2 flight and enhanced manoeuvrability.
2. `FullyOperationalEurofighterState` — both engines operational; Mach 2 flight.
3. `DegradedEurofighterState` — one engine operational; degraded flight only.
4. `OperationalEurofighterState` — at least one engine operational; some level of flight.
5. `NonOperationalEurofighterState` — no flight capability.

This ordering is not a flat enumeration but a partial order encoded through subclass relationships. Level 1 is a subclass of level 2, which is a subclass of level 4. Level 3 is also a subclass of level 4 but is *not* in a subclass relationship with level 2 (a degraded state is not a fully operational state). Level 5 is a sibling of level 4 under the common parent `EurofighterState`.

[Diagram placeholder: A subclass hierarchy diagram showing the five-level spectrum. $\text{EnhancedEurofighterState} \subset \text{FullyOperationalEurofighterState} \subset \text{OperationalEurofighterState}$,

and DegradedEurofighterState $\subset$ OperationalEurofighterState, with NonOperationalEurofighterState as a sibling. Each class annotated with its eachHasDisposition and eachRequiresPart assertions.]

At each level, the disposition pattern's properties — `eachHasDisposition`, `hasDisposition`, `eachRequiresPart`, and `requiresPart` — provide the same structural mechanisms. No additional ontological machinery is needed to represent the spectrum; the expressiveness comes from the combination of `ClassOfState` subclass hierarchies, distinct `Capability` instances, and the specificity of the part-state requirements in `eachRequiresPart` assertions.

The spectrum can be extended arbitrarily. If a new degradation mode were identified — for example, a state in which both engines are operational but at reduced power — it would be modelled as a new `ClassOfState` slotted into the hierarchy between `FullyOperationalEurofighterState` and `DegradedEurofighterState`, with its own `Capability` instances and dependency assertions. Similarly, further enhancements (additional avionics, weapons systems, sensor upgrades) each produce new state classes extending the hierarchy upward. The pattern scales without structural change.

C.7 Summary

The disposition pattern introduced in §5 handles degradation spectra and capability enhancement without extension. The mechanism is the `ClassOfState` subclass hierarchy: each level of degradation or enhancement is a distinct `ClassOfState` with its own disposition and dependency assertions. The hierarchy's subclass relationships encode which dispositions are inherited and which are specific to a particular level. The dependency properties (`eachRequiresPart`, `requiresPart`) capture the different part-state requirements at each level.

Three observations from this appendix inform the main document:

1. **No new ontological machinery is required.** The six properties proposed in §5.2, combined with the `ClassOfState` hierarchy, are sufficient to represent arbitrarily complex degradation and enhancement spectra. The pattern's expressiveness comes from its compositional nature, not from dedicated degradation or enhancement constructs.
 2. **The spectrum is a modelling choice, not an ontological commitment.** Data producers choose the granularity of the spectrum based on the information they need to exchange. A simple binary (operational/non-operational) may suffice for some exchanges; a detailed multi-level spectrum may be needed for others. The pattern supports both extremes and everything in between.
 3. **Measured dispositions become most relevant at intermediate states.** The qualitative framework established here — different `Capability` instances at different levels of the spectrum — provides the structural context for the future quantification work identified in §3.7 and §7.4. It is at the intermediate, degraded states that the question "how much capability remains?" is most pressing.
-

Appendix D Evacuation Route

This appendix provides a worked example of the proposed disposition properties applied to a civil emergency scenario on the Isle of Wight. The example exercises several requirements from §3 — class-level dispositions (§3.2), individual-level dispositions (§3.3), permitted and prohibited uses (§3.4), whole-part dependencies (§3.5), and cross-entity dependencies (§3.6) — in a domain that is quite different from the military aviation scenario used in the main document. Where the Eurofighter examples deal with a single complex asset and its capabilities, this example deals with infrastructure composed of multiple road segments, each with its own operating constraints, that must work together to provide an end-to-end evacuation capability.

The scenario is drawn from the Isle of Wight NDTP (National Digital Twin Programme) use cases — specifically, Use Case 6: temporary change of use in the road network. The route chosen is a simplified representation of the principal road corridor from Sandown to Ryde, a distance of approximately 6.7 miles along the A3055, passing through Brading. This corridor provides road access to the ferry terminal at Ryde, and would be a critical evacuation route in an emergency affecting the eastern coast of the island.

D.1 Scenario

An emergency planner is developing evacuation plans for a coastal flooding event affecting Sandown and the surrounding area. The plan designates a road evacuation route from Sandown to Ryde, where evacuees can board cross-Solent ferries to Portsmouth. The route follows the A3055 northward through Brading to Ryde.

There are two aspects to the evacuation route that must be distinguished:

The infrastructure — the physical road corridor — is composed of three road sections (Links) connected at junctions (Nodes), all part of the Isle of Wight highway network:

- **Link 1: A3055 Sandown to Brading** — the section of the A3055 from the edge of Sandown to the junction with the A3056 at Brading.
- **Link 2: A3055 Brading to Westridge Cross** — the section from Brading northward to the junction at Westridge Cross.
- **Link 3: A3055 Westridge Cross to Ryde** — the section from Westridge Cross into Ryde and the ferry terminal approach.

The four nodes are:

- **Node S** — the start point at Sandown (the southern boundary of the route).
- **Node B** — the junction at Brading where the A3056 joins from the west.
- **Node W** — the junction at Westridge Cross where routes diverge toward Ryde, Bembridge, and Seaview.
- **Node R** — the end point at Ryde ferry terminal (the northern boundary of the route).

The flow — the actual or potential movement of evacuees along the corridor — is a `FlowAlongPath` in IES terms: the matter (vehicles carrying people) that passes along the path defined by the infrastructure. In IES, `Flow` is a subclass of `State`, so flows are temporal entities — they have a beginning, a duration, and an end. The road sections *contain* the flow via the Containment pattern: the road is the Container, and the flow of vehicles is the Contained.

This distinction is important for the disposition pattern. It is the *flow* — the potential movement of evacuees — that is the subject of the disposition assertions. The flow has the disposition to carry evacuation traffic; the infrastructure has the disposition to be physically passable. The end-to-end evacuation capability depends on both.

The following information needs to be recorded:

1. **The infrastructure as a network structure.** The road sections, junctions, and their connectivity form part of the Isle of Wight highway network.
2. **The evacuation route as a flow.** The designated flow of evacuees from Sandown to Ryde, contained by the road sections.
3. **Class-level disposition.** All A-road sections on the Isle of Wight have the disposition to carry general traffic (i.e. they are general-purpose carriageways).
4. **Individual-level disposition.** Each of the three specific road sections has been assessed as suitable for use as part of an evacuation route, and the end-to-end flow has been assessed as a viable evacuation corridor.
5. **Permitted uses during the emergency.** When the evacuation is activated, the route is designated for evacuation (egress) traffic only. Ingress traffic is restricted to emergency services vehicles. This represents a temporary change of `PermittedUse`.
6. **Whole-part dependency.** The end-to-end evacuation flow's capability depends on all three constituent road sections being in a passable state. If any section is blocked (e.g. by flooding or an accident), the end-to-end capability is lost.
7. **Cross-entity dependency.** The route's effectiveness as an evacuation corridor depends on the Ryde ferry terminal being operational — an entirely separate entity, not part of the road network, but one whose operational state is a prerequisite for the route to fulfil its purpose.

D.2 Prefixes and Namespaces

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ies: <http://informationexchangestandard.org/ont/ies/common/> .
@prefix ex: <http://example.org/iow-emergency#> .
@prefix data: <http://example.org/iow-emergency/data#> .
```

The `ies:` prefix refers to IES Common. The `ex:` prefix is used for domain-specific class and property definitions (the extension ontology for this scenario). The `data:` prefix is used for instance data — the specific road sections, junctions, and states being recorded.

D.3 Domain Extension Classes

Before recording instance data, we define the domain-specific classes that specialise the IES Common vocabulary for this scenario. These follow the pattern established in the IoW highway network examples.

```
# --- Network classes ---

ex:HighwayNetwork rdfs:subClassOf ies:Network ;
    rdfs:label "Highway Network"@en-GB ;
    rdfs:comment "A network of roads and road junctions."@en-GB .

ex:RoadSection rdfs:subClassOf ies:Link ;
    rdfs:label "Road Section"@en-GB ;
    rdfs:comment "A section of road between junctions."@en-GB .

ex:RoadJunction rdfs:subClassOf ies:Node ;
    rdfs:label "Road Junction"@en-GB ;
    rdfs:comment "A junction where two or more road sections meet."@en-GB .

ex:RouteTerminus rdfs:subClassOf ies:Node ;
    rdfs:label "Route Terminus"@en-GB ;
    rdfs:comment "A node marking the start or end point of a route."@en-GB .

# --- Flow classes ---

ex:FlowOfVehicles rdfs:subClassOf ies:FlowAlongPath ;
    rdfs:label "Flow of Vehicles"@en-GB ;
    rdfs:comment "A FlowAlongPath that is a flow of vehicles along a
        road or other carriageway."@en-GB .

# --- Facility classes ---

ex:FerryTerminal rdfs:subClassOf ies:Facility ;
    rdfs:label "Ferry Terminal"@en-GB ;
    rdfs:comment "A facility providing embarkation and disembarkation for
        ferry services."@en-GB .
```

We also define the `ClassOfState` members that will serve as dispositions and dependencies. These are subtypes of `ies:Capability` (itself a subtype of `ies:DispositionalClass` and `ies:ClassOfState`):

```
# --- Disposition classes (ClassOfState members) ---

ex:CarryGeneralTraffic a ies:Capability ;
    rdfs:label "Carry general traffic"@en-GB ;
    rdfs:comment "A state of a road section in which it carries general
        mixed traffic under normal operating conditions."@en-GB ;
    rdfs:subClassOf ies:State .

ex:CarryEvacuationTraffic a ies:Capability ;
    rdfs:label "Carry evacuation traffic"@en-GB ;
    rdfs:comment "A state of a road section in which it carries
        evacuation (egress) traffic under emergency conditions."@en-GB ;
```

```

    rdfs:subClassOf ies:State .

ex:CarryEmergencyServicesOnly a ies:Capability ;
    rdfs:label "Carry emergency services traffic only"@en-GB ;
    rdfs:comment "A state of a road section in which it is restricted to
        emergency services vehicles only."@en-GB ;
    rdfs:subClassOf ies:State .

ex:PassableState a ies:Capability ;
    rdfs:label "Passable state"@en-GB ;
    rdfs:comment "A state of a road section in which it is physically
        passable by vehicles – i.e. not blocked by flooding,
        obstruction, or structural failure."@en-GB ;
    rdfs:subClassOf ies:State .

ex:EvacuationRouteCapable a ies:Capability ;
    rdfs:label "Evacuation-route-capable state"@en-GB ;
    rdfs:comment "A state of a route in which it provides end-to-end
        evacuation capability from its start to its finish."@en-GB ;
    rdfs:subClassOf ies:State .

ex:FerryOperational a ies:Capability ;
    rdfs:label "Ferry terminal operational"@en-GB ;
    rdfs:comment "A state of a ferry terminal in which it is operational
        and accepting passengers for cross-Solent services."@en-GB ;
    rdfs:subClassOf ies:State .

```

D.4 Network Structure

The route and its constituent parts are recorded using the IES Network pattern: Links connected at Nodes, all part of a Network.

```

# --- The network ---

data:IoWHighwayNetwork a ex:HighwayNetwork ;
    rdfs:label "Isle of Wight Highway Network"@en-GB .

# --- Nodes ---

data:NodeS a ex:RouteTerminus ;
    rdfs:label "Sandown (route start)"@en-GB ;
    ies:isPartOf data:IoWHighwayNetwork .

data:NodeB a ex:RoadJunction ;
    rdfs:label "Brading junction (A3055 / A3056)"@en-GB ;
    ies:isPartOf data:IoWHighwayNetwork .

data:NodeW a ex:RoadJunction ;
    rdfs:label "Westridge Cross junction"@en-GB ;
    ies:isPartOf data:IoWHighwayNetwork .

data:NodeR a ex:RouteTerminus ;
    rdfs:label "Ryde ferry terminal approach"@en-GB ;
    ies:isPartOf data:IoWHighwayNetwork .

```

```

# --- Links ---

data:Link1_Sandown_Brading a ex:RoadSection ;
  rdfs:label "A3055 Sandown to Brading"@en-GB ;
  rdfs:comment "Section of the A3055 from the edge of Sandown to the
    junction at Brading."@en-GB ;
  ies:hasEnd data:NodeS ;
  ies:hasEnd data:NodeB ;
  ies:isPartOf data:IoWHighwayNetwork .

data:Link2_Brading_Westridge a ex:RoadSection ;
  rdfs:label "A3055 Brading to Westridge Cross"@en-GB ;
  rdfs:comment "Section of the A3055 from the Brading junction
    northward to Westridge Cross."@en-GB ;
  ies:hasEnd data:NodeB ;
  ies:hasEnd data:NodeW ;
  ies:isPartOf data:IoWHighwayNetwork .

data:Link3_Westridge_Ryde a ex:RoadSection ;
  rdfs:label "A3055 Westridge Cross to Ryde"@en-GB ;
  rdfs:comment "Section of the A3055 from Westridge Cross into Ryde and
    the ferry terminal approach."@en-GB ;
  ies:hasEnd data:NodeW ;
  ies:hasEnd data:NodeR ;
  ies:isPartOf data:IoWHighwayNetwork .

# --- Road identifiers ---

data:Link1_Sandown_Brading ies:isIdentifiedBy data:Link1_RoadNumber .
data:Link1_RoadNumber a ies:Identifier ;
  ies:representationValue "A3055"^^xsd:string .

data:Link2_Brading_Westridge ies:isIdentifiedBy data:Link2_RoadNumber .
data:Link2_RoadNumber a ies:Identifier ;
  ies:representationValue "A3055"^^xsd:string .

data:Link3_Westridge_Ryde ies:isIdentifiedBy data:Link3_RoadNumber .
data:Link3_RoadNumber a ies:Identifier ;
  ies:representationValue "A3055"^^xsd:string .

# --- The ferry terminal (a separate entity) ---

data:RydeFerryTerminal a ex:FerryTerminal ;
  rdfs:label "Ryde ferry terminal"@en-GB ;
  rdfs:comment "The Wightlink passenger ferry terminal at Ryde Pier Head,
    providing cross-Solent services to Portsmouth."@en-GB .

```

This records the basic topology: three links forming a chain from Sandown to Ryde, connected at Brading and Westridge Cross, all part of the Isle of Wight highway network. The ferry terminal is recorded as a separate entity — it is not part of the highway network.

D.4a The Evacuation Flow

The evacuation route — the designated movement of evacuees from Sandown to Ryde — is modelled as a `FlowAlongPath`. In IES, `Flow` is a subclass of `State`: a flow is a temporal entity representing the matter, energy, or signal that passes along a path. Here, the "matter" is vehicles carrying evacuees.

The flow is *contained by* the road sections via the IES Containment pattern. The road is the Container; the flow of vehicles is the Contained.

```
# --- The evacuation flow ---

data:SandownRydeEvacFlow a ex:FlowOfVehicles ;
  rdfs:label "Evacuation flow from Sandown to Ryde"@en-GB ;
  rdfs:comment "The designated flow of vehicles carrying evacuees from
    Sandown to Ryde ferry terminal via the A3055."@en-GB .

# --- Containment: the road sections contain the flow ---
# Each road section contains the portion of the flow that passes
# through it. The containment is recorded using the IES Containment
# pattern: a Containment element has a Container (the road section)
# and a Contained (the flow).

data:Link1ContainsFlow a ies:Containment ;
  rdfs:label "Flow contained by A3055 Sandown to Brading"@en-GB .

data:Link1_Sandown_Brading_AsContainer a ies:Container ;
  ies:isStateOf data:Link1_Sandown_Brading ;
  ies:isPartOf data:Link1ContainsFlow .

data:SandownRydeEvacFlow_InLink1 a ies:Contained ;
  ies:isStateOf data:SandownRydeEvacFlow ;
  ies:isPartOf data:Link1ContainsFlow .

data:Link2ContainsFlow a ies:Containment ;
  rdfs:label "Flow contained by A3055 Brading to Westridge"@en-GB .

data:Link2_Brading_Westridge_AsContainer a ies:Container ;
  ies:isStateOf data:Link2_Brading_Westridge ;
  ies:isPartOf data:Link2ContainsFlow .

data:SandownRydeEvacFlow_InLink2 a ies:Contained ;
  ies:isStateOf data:SandownRydeEvacFlow ;
  ies:isPartOf data:Link2ContainsFlow .

data:Link3ContainsFlow a ies:Containment ;
  rdfs:label "Flow contained by A3055 Westridge to Ryde"@en-GB .

data:Link3_Westridge_Ryde_AsContainer a ies:Container ;
  ies:isStateOf data:Link3_Westridge_Ryde ;
  ies:isPartOf data:Link3ContainsFlow .

data:SandownRydeEvacFlow_InLink3 a ies:Contained ;
  ies:isStateOf data:SandownRydeEvacFlow ;
```

```
ies:isPartOf data:Link3ContainsFlow .
```

Note that the Containment pattern uses States — the container is a State of the road section, and the contained is a State of the flow. This reflects the 4D nature of containment: the flow is contained by the road section for a temporal extent, not for all time. In the planning phase, these containment relationships record the *intended* containment; during the actual evacuation, temporal states would be scoped to the relevant period.

D.5 Class-Level Disposition (§3.2)

All A-road sections on the Isle of Wight have the disposition to carry general traffic. This is a class-level assertion: it applies to every member of the class, not to any specific individual.

Using the proposed `eachHasDisposition`:

```
ex:IoWARoadSection a ies:ClassOfState ;  
  rdfs:label "Isle of Wight A-road section"@en-GB ;  
  rdfs:comment "The class of A-road sections on the Isle of Wight."@en-GB ;  
  rdfs:subClassOf ex:RoadSection .
```

```
ex:IoWARoadSection ies:eachHasDisposition ex:CarryGeneralTraffic .
```

This reads: "*Every Isle of Wight A-road section can be in a state of carrying general traffic.*" The three specific links in our route are members of this class:

```
data:Link1_Sandown_Brading a ex:IoWARoadSection .  
data:Link2_Brading_Westridge a ex:IoWARoadSection .  
data:Link3_Westridge_Ryde a ex:IoWARoadSection .
```

By virtue of the class-level disposition, each of these links inherits the general traffic disposition without requiring individual assertions.

For comparison, the equivalent using the existing encoding:

```
ex:IoWARoadSection ies:allHaveDisposition ex:CarryGeneralTraffic .
```

Both express the same set-theoretic fact: for every member x of `ex:IoWARoadSection`, $S_x \cap \text{ex:CarryGeneralTraffic} \neq \emptyset$.

D.6 Individual-Level Disposition (§3.3)

Each of the three road sections has been individually assessed as suitable for use in an evacuation route. This is an individual-level assertion: it records the outcome of a specific assessment for each link.

```
data:Link1_Sandown_Brading ies:hasDisposition ex:CarryEvacuationTraffic .  
data:Link2_Brading_Westridge ies:hasDisposition ex:CarryEvacuationTraffic .  
data:Link3_Westridge_Ryde ies:hasDisposition ex:CarryEvacuationTraffic .
```

Each assertion reads: *"This road section can be in a state of carrying evacuation traffic."* Note that this records a *potential* — the road section is suitable for evacuation use — not that evacuation traffic is actually flowing. The road is not yet in a state of `ex:CarryEvacuationTraffic`; it merely has the disposition to be.

The end-to-end flow also has a disposition — the potential to serve as an evacuation corridor:

```
data:SandownRydeEvacFlow ies:hasDisposition ex:EvacuationRouteCapable .
```

Since `FlowAlongPath` is a subclass of `Flow`, which is a subclass of `State`, the evacuation flow sits naturally in the domain of `hasDisposition`. This assertion reads: *"The Sandown-to-Ryde evacuation flow can be in a state of providing end-to-end evacuation capability."*

For comparison, the existing encoding for the road sections:

```
data:Link1_Sandown_Brading ies:isDisposedTo ex:CarryEvacuationTraffic .
data:Link2_Brading_Westridge ies:isDisposedTo ex:CarryEvacuationTraffic .
data:Link3_Westridge_Ryde ies:isDisposedTo ex:CarryEvacuationTraffic .
```

D.7 Permitted Uses During the Emergency (§3.4)

When the evacuation is activated, the operating conditions on the route change. This is modelled as a temporal `State` of each road section during the emergency period, with disposition assertions scoped to those states.

First, we define a `ClassOfState` representing road sections that are in an active evacuation condition:

```
ex:RoadSectionInEvacuationMode a ies:ClassOfState ;
  rdfs:label "Road section in evacuation mode"@en-GB ;
  rdfs:comment "A state of a road section during an active evacuation,
    in which normal traffic rules are suspended and emergency
    operating conditions apply."@en-GB ;
  rdfs:subClassOf ies:State .
```

During the emergency, each road section enters this state. The disposition assertions are scoped to these states:

```
# --- The emergency event ---

data:SandownEvacuation2025 a ies:Event ;
  rdfs:label "Sandown coastal flooding evacuation, June 2025"@en-GB .

# --- Temporal states of each link during the evacuation ---

data:Link1_EvacState a ex:RoadSectionInEvacuationMode ;
  ies:isStateOf data:Link1_Sandown_Brading ;
  rdfs:label "A3055 Sandown to Brading during evacuation"@en-GB .

data:Link2_EvacState a ex:RoadSectionInEvacuationMode ;
```

```

    ies:isStateOf data:Link2_Brading_Westridge ;
    rdfs:label "A3055 Brading to Westridge during evacuation"@en-GB .

data:Link3_EvacState a ex:RoadSectionInEvacuationMode ;
    ies:isStateOf data:Link3_Westridge_Ryde ;
    rdfs:label "A3055 Westridge to Ryde during evacuation"@en-GB .

# --- Permitted use: evacuation traffic on the northbound carriageway ---

data:Link1_EvacState ies:hasDisposition ex:CarryEvacuationTraffic .
data:Link2_EvacState ies:hasDisposition ex:CarryEvacuationTraffic .
data:Link3_EvacState ies:hasDisposition ex:CarryEvacuationTraffic .

# --- Restricted use: emergency services only on the southbound carriageway ---
--

data:Link1_EvacState ies:hasDisposition ex:CarryEmergencyServicesOnly .
data:Link2_EvacState ies:hasDisposition ex:CarryEmergencyServicesOnly .
data:Link3_EvacState ies:hasDisposition ex:CarryEmergencyServicesOnly .

```

This records that during the evacuation, each road section has two permitted uses: evacuation traffic (northbound, toward Ryde) and emergency services traffic only (southbound, toward Sandown). The general traffic disposition — inherited from the class-level assertion in §C.5 — still exists as a whole-life disposition. The evacuation-mode states represent a temporary narrowing of permitted use during the emergency period.

The relationship between the general traffic disposition and the emergency dispositions illustrates a point raised in §3.4: the disposition to carry general traffic is not lost during the evacuation (the road *can* still carry general traffic in the physical sense), but the *permitted use* changes. A fuller treatment of the distinction between physical capability and regulatory permission would require the prohibition pattern discussed in §5.1.4 — recording that general traffic is *prohibited* during the evacuation, even though the physical capability remains.

D.8 Whole-Part Dependency (§3.5)

The evacuation flow's end-to-end capability depends on all three constituent road sections being passable. This is a whole-part dependency: the flow is contained by the road sections, and if any containing section is blocked, the end-to-end flow cannot exist.

The containment relationships established in §C.4a already record that the road sections contain the flow. The dependency assertion records that the flow's evacuation capability *requires* each containing section to be in a passable state:

```

data:SandownRydeEvacFlow ies:requiresPart ex:PassableState .

```

The `requiresPart` assertion reads: "The Sandown–Ryde evacuation flow requires, as a mereological part, at least one member of `ex:PassableState`." The mereological relationship here arises through the containment: the road sections contain the flow, and a temporal state of a containing road section is a part of the containment (and hence of the overall spatio-temporal extent within which the flow exists).

At the class level, we can assert this for all evacuation flows:

```
ex:EvacuationFlow a ies:ClassOfState ;
  rdfs:label "Evacuation flow"@en-GB ;
  rdfs:comment "A flow of vehicles along a designated evacuation
    route."@en-GB ;
  rdfs:subClassOf ex:FlowOfVehicles .

ex:EvacuationFlow ies:eachRequiresPart ex:PassableState .
```

This class-level assertion reads: "Every member of `ex:EvacuationFlow` requires, as a mereological part, at least one member of `ex:PassableState`." In practice, this means that each road section containing the evacuation flow must itself be in a passable state.

This mirrors the Eurofighter engine example from §5.2.4: just as the Eurofighter's Mach 2 capability requires both EJ200 engines to be operational, the evacuation flow's end-to-end capability requires all three road sections to be passable. If the Brading-to-Westridge section is flooded, the flow as a whole loses its evacuation capability — even though the Sandown-to-Brading and Westridge-to-Ryde sections remain individually passable.

D.9 Cross-Entity Dependency (§3.6)

The evacuation flow's effectiveness depends on the Ryde ferry terminal being operational. The ferry terminal is not part of the road network — it is a separate entity with its own lifecycle, operating conditions, and dispositions. But without an operational ferry terminal, evacuees reaching Ryde have nowhere to go.

```
data:SandownRydeEvacFlow ies:requires ex:FerryOperational .
```

This reads: "*The Sandown–Ryde evacuation flow requires the temporal co-occurrence of at least one member of `ex:FerryOperational`.*" That is, for the flow to fulfil its purpose, the ferry terminal must be in an operational state at the same time as the flow is active.

Note the structural parallel with the Eurofighter–Voyager dependency from §3.6 and §5.2.5. The Eurofighter's extended-range disposition requires the Voyager tanker's refuelling disposition. Here, the evacuation route's end-to-end capability requires the ferry terminal's operational disposition. In both cases, the dependent entity (the Eurofighter, the evacuation route) has a disposition that can only be realised when an entirely separate entity (the Voyager, the ferry terminal) is in the required state.

The cross-entity dependency can also be expressed at the class level:

```
ex:CoastalEvacuationRoute a ies:ClassOfState ;
  rdfs:label "Coastal evacuation route"@en-GB ;
  rdfs:comment "The class of states of routes that serve as coastal
    evacuation corridors to ferry terminals."@en-GB ;
  rdfs:subClassOf ies:State .

ex:CoastalEvacuationRoute ies:eachRequires ex:FerryOperational .
```

This asserts that every coastal evacuation route, as a class, depends on a ferry terminal being operational. Any specific route typed as a member of this class inherits the dependency.

D.10 Putting It Together

The following SPARQL queries illustrate how the recorded data can be used to answer planning questions.

Query 1: What dispositions does the evacuation flow have?

```
PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX data: <http://example.org/iow-emergency/data#>

SELECT ?disposition ?label
WHERE {
    data:SandownRydeEvacFlow ies:hasDisposition ?disposition .
    ?disposition rdfs:label ?label .
}
```

Query 2: What are the dependencies of the evacuation flow?

```
PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX data: <http://example.org/iow-emergency/data#>

SELECT ?dependency ?label ?depType
WHERE {
    {
        data:SandownRydeEvacFlow ies:requiresPart ?dependency .
        BIND ("whole-part" AS ?depType)
    } UNION {
        data:SandownRydeEvacFlow ies:requires ?dependency .
        BIND ("cross-entity" AS ?depType)
    }
    ?dependency rdfs:label ?label .
}
```

This returns both the whole-part dependency on `ex:PassableState` and the cross-entity dependency on `ex:FerryOperational`, distinguished by type.

Query 3: Which road sections are designated for evacuation use?

```
PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX ex: <http://example.org/iow-emergency#>

SELECT ?link ?linkLabel
WHERE {
    ?link ies:hasDisposition ex:CarryEvacuationTraffic ;
        rdfs:label ?linkLabel .
    ?link a ex:RoadSection .
}
```

D.11 Summary

This example demonstrates the proposed disposition properties in a civil infrastructure context:

Requirement	Property / pattern used	What it records
Infrastructure	Network pattern (Link, Node, <code>hasEnd</code> , <code>isPartOf</code>)	Road sections, junctions, and connectivity
Flow and containment	<code>FlowAlongPath</code> + Containment pattern	The evacuation flow, contained by the road sections
§3.2 Class-level disposition	<code>eachHasDisposition</code>	All IoW A-road sections can carry general traffic
§3.3 Individual-level disposition	<code>hasDisposition</code>	Each road section can carry evacuation traffic; the flow can serve as an evacuation corridor
§3.4 Permitted uses	<code>hasDisposition</code> (on temporal States)	During the evacuation, specific operating conditions apply
§3.5 Whole-part dependency	<code>requiresPart</code> / <code>eachRequiresPart</code>	The flow's capability depends on each containing section being passable
§3.6 Cross-entity dependency	<code>requires</code> / <code>eachRequires</code>	The flow's capability depends on the ferry terminal being operational

The example also illustrates several features of the proposed properties and their interaction with existing IES patterns:

1. **The Flow/Containment pattern and the disposition pattern work together naturally.** The evacuation flow is a `FlowAlongPath` (a subclass of `State`), which sits naturally in the domain of `hasDisposition`. The road sections contain the flow, and the dependency properties record that the flow's capability requires the containing infrastructure to be in a passable state. The two patterns — Flow/Containment for the physical relationship, dispositions for the modal relationship — are complementary rather than overlapping.
2. **Disposition and dependency are cleanly separated.** The flow's own capability (`hasDisposition ex:EvacuationRouteCapable`) is recorded independently from the conditions under which it can be realised (`requiresPart ex:PassableState`, `requires ex:FerryOperational`). In the existing encoding, these would need to be bundled into the class definition.
3. **Whole-part and cross-entity dependencies use structurally parallel properties.** The `requiresPart` and `requires` assertions have the same form — State-to-ClassOfState — and differ only in the mereological constraint. This makes it straightforward to query for all dependencies of any kind.
4. **The temporal scoping of permitted uses is handled by the standard IES State mechanism.** The evacuation-mode states are temporal parts of the road sections, and the disposition assertions on those states apply only during the relevant period. No special temporal extension is needed.

<!-- DIAGRAM PLACEHOLDER (Figure D.1): A map-style diagram showing the evacuation route from Sandown to Ryde, with the three road sections (Links), four nodes, and the ferry terminal marked. The flow direction (northbound, Sandown to Ryde) should be indicated. --> <!-- DIAGRAM PLACEHOLDER (Figure D.2): A UML-style IES recording diagram showing the network structure — the three Links, four Nodes, and their hasEnd / isPartOf relationships to the IoW Highway Network. The FlowAlongPath (evacuation flow) and the Containment relationships linking the flow to the road sections should be shown. --> <!-- DIAGRAM PLACEHOLDER (Figure D.3): A UML-style diagram showing the disposition and dependency relationships: hasDisposition on the flow and road sections, requiresPart linking the flow to PassableState, and requires linking the flow to FerryOperational. The ferry terminal should be shown as a separate entity connected by the cross-entity dependency. --> <!-- DIAGRAM PLACEHOLDER (Figure D.4): A spacetime diagram showing the temporal states during the evacuation — the normal-use states, the evacuation-mode states, and how the permitted uses change during the emergency period. The flow's temporal extent and the ferry terminal's operational state should be shown as temporally co-occurring extents. -->

Appendix E: Sandown Water Treatment Works Degradation

This appendix provides a worked example of the proposed disposition properties applied to a critical infrastructure scenario on the Isle of Wight. The example exercises class-level dispositions (§3.2), individual-level dispositions (§3.3), whole-part dependencies (§3.5), cross-entity dependencies (§3.6), and the degradation spectrum pattern (Appendix C) — in a domain that involves cascading infrastructure dependencies, identified as a deferred annex in §7.4.

Where Appendix C deals with degradation within a single complex asset (the Eurofighter's propulsion system), and Appendix D deals with a linear infrastructure corridor (the evacuation route), this example deals with a facility whose operational state depends on an external utility supply, and whose loss triggers a cascade from full capability to a degraded mode with reduced output. The example also records the actuality of the power failure event and the resulting state transitions.

The scenario is drawn from the Sandown Water Treatment Works (WTW) dependency chain — specifically, the external capacity dependencies and dependency chain use cases. The WTW is a simplified but representative model of a water treatment facility whose pumping capability depends on electrical power, and which has a local backup power source that can sustain only a reduced level of operation.

E.1 Scenario

The Sandown Water Treatment Works is a facility on the Isle of Wight that treats and pumps potable water to consumers in the Sandown area. Under normal conditions, the WTW draws its electrical power from the Isle of Wight electricity distribution network via a local sub-station (Sandown Sub-Station). The WTW has the following operational characteristics:

1. **Fully operational** — mains power is available from the sub-station. All pumps are powered. The WTW can treat and pump water at its full design capacity (notionally, 10 megalitres per day).
2. **Degraded ("backup power mode")** — mains power has been lost (the sub-station has failed or the connection is severed). The WTW's local backup generator activates automatically. The generator can power a reduced set of pumps, giving a reduced pumping capability (notionally, 4 megalitres per day) — sufficient for essential supply but not full demand.
3. **Non-operational** — both mains power and backup power have failed. No pumping is possible.

The following information needs to be recorded:

1. **The WTW and its key subsystems.** The WTW contains a pumping system and a backup generator, both of which are parts of the facility.
2. **The external power supply infrastructure.** The Sandown Sub-Station provides electrical power to the WTW via a connection. The sub-station is part of the wider IoW electricity distribution network, but is not part of the WTW itself.
3. **Class-level dispositions.** Water treatment works of this type have the disposition to treat and pump water. Electrical sub-stations have the disposition to supply electrical power.
4. **Individual-level dispositions.** The Sandown WTW has been assessed as having full pumping capability (when mains-powered) and reduced pumping capability (when on backup power). The backup generator has the disposition to provide emergency electrical power.
5. **Cross-entity dependency.** The WTW's full operational capability depends on the sub-station being in an operational state — a cross-entity dependency, since the sub-station is not part of the WTW.
6. **Whole-part dependency.** The WTW's backup-power capability depends on the backup generator (a part of the WTW) being in an operational state — a whole-part dependency.
7. **The degradation spectrum.** The three operational levels (fully operational, degraded, non-operational) form a partial order of ClassOfState definitions, exactly as in Appendix C.
8. **The actuality.** On 15 January 2026, the Sandown Sub-Station suffers a failure. The WTW transitions from a fully operational state to a degraded (backup power) state. The event and the resulting state transitions are recorded as instance data.

E.2 Prefixes and Namespaces

```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix ies: <http://informationexchangestandard.org/ont/ies/common/> .
@prefix ex: <http://example.org/iow-water#> .
@prefix data: <http://example.org/iow-water/data#> .
```

The `ies:` prefix refers to IES Common. The `ex:` prefix is used for domain-specific class and property definitions. The `data:` prefix is used for instance data.

E.3 Domain Extension Classes

Before recording instance data, we define the domain-specific classes that specialise the IES Common vocabulary for this scenario.

```
# --- Facility and system classes ---

ex:WaterTreatmentWorks rdfs:subClassOf ies:Facility ;
    rdfs:label "Water Treatment Works"@en-GB ;
    rdfs:comment "A facility that treats raw water and pumps potable
        water to consumers."@en-GB .

ex:PumpingSystem rdfs:subClassOf ies:Device ;
    rdfs:label "Pumping System"@en-GB ;
    rdfs:comment "A system of pumps within a water treatment works that
        moves water through the treatment and distribution process."@en-GB .

ex:BackupGenerator rdfs:subClassOf ies:Device ;
    rdfs:label "Backup Generator"@en-GB ;
    rdfs:comment "A local electricity generator that provides emergency
        power when mains supply is unavailable."@en-GB .

ex:ElectricalSubStation rdfs:subClassOf ies:Facility ;
    rdfs:label "Electrical Sub-Station"@en-GB ;
    rdfs:comment "A facility within an electricity distribution network
        that transforms and distributes electrical power to local
        consumers."@en-GB .

ex:ElectricityDistributionNetwork rdfs:subClassOf ies:Network ;
    rdfs:label "Electricity Distribution Network"@en-GB ;
    rdfs:comment "A network of power stations, sub-stations, and
        transmission infrastructure that distributes electrical
        power."@en-GB .
```

We also define the Capability instances — the ClassOfState members that serve as dispositions:

```
# --- Capability instances (disposition targets) ---

ex:PumpWaterFullCapacity a ies:Capability ;
    rdfs:label "Pump water at full capacity"@en-GB ;
    rdfs:comment "A state of a water treatment works in which it treats
        and pumps water at its full design capacity."@en-GB ;
    rdfs:subClassOf ies:State .

ex:PumpWaterReducedCapacity a ies:Capability ;
    rdfs:label "Pump water at reduced capacity"@en-GB ;
    rdfs:comment "A state of a water treatment works in which it treats
        and pumps water at a reduced capacity, sufficient for essential
        supply only."@en-GB ;
    rdfs:subClassOf ies:State .

ex:SupplyElectricalPower a ies:Capability ;
    rdfs:label "Supply electrical power"@en-GB ;
    rdfs:comment "A state of an electrical sub-station or generator in
        which it supplies electrical power to connected consumers."@en-GB ;
```

```

    rdfs:subClassOf ies:State .

ex:ProvideEmergencyPower a ies:Capability ;
    rdfs:label "Provide emergency electrical power"@en-GB ;
    rdfs:comment "A state of a backup generator in which it provides
        emergency electrical power, typically at reduced capacity
        compared to a mains supply."@en-GB ;
    rdfs:subClassOf ies:State .

```

E.4 Instance Data: The Infrastructure

The WTW, its internal subsystems, the sub-station, and the wider network are recorded as instance data. The WTW contains a pumping system and a backup generator as parts. The sub-station is part of the wider electricity distribution network but is *not* part of the WTW — this is the structural basis for the cross-entity dependency.

```

# --- The electricity distribution network ---

data:IoWElectricityNetwork a ex:ElectricityDistributionNetwork ;
    rdfs:label "Isle of Wight Electricity Distribution Network"@en-GB .

# --- The sub-station (part of the electricity network, NOT part of the WTW)
---

data:SandownSubStation a ex:ElectricalSubStation ;
    rdfs:label "Sandown Sub-Station"@en-GB ;
    ies:isPartOf data:IoWElectricityNetwork .

# --- The water treatment works ---

data:SandownWTW a ex:WaterTreatmentWorks ;
    rdfs:label "Sandown Water Treatment Works"@en-GB .

# --- Parts of the WTW ---

data:SandownPumpingSystem a ex:PumpingSystem ;
    rdfs:label "Sandown WTW Pumping System"@en-GB ;
    ies:isPartOf data:SandownWTW .

data:SandownBackupGenerator a ex:BackupGenerator ;
    rdfs:label "Sandown WTW Backup Generator"@en-GB ;
    ies:isPartOf data:SandownWTW .

```

The connection between the sub-station and the WTW is recorded using the IES Connection pattern. The connection represents the physical power supply link — the cable or distribution line through which electricity flows:

```

# --- The power supply connection ---

data:MainsPowerConnection a ies:Connection ;
    rdfs:label "Mains power connection: Sandown Sub-Station to
        Sandown WTW"@en-GB ;
    ies:isConnectedTo data:SandownSubStation ;
    ies:isConnectedTo data:SandownWTW .

```

E.5 The Degradation Spectrum: State Class Hierarchy

Following the pattern established in Appendix D, we define a hierarchy of `ClassOfState` definitions for the WTW. Each level corresponds to an operational condition with its own set of dispositions and dependencies.

E.5.1 State classes for the WTW

```
# --- WTW state classes: type-level ---

ex:WTWState a ies:ClassOfState ;
  rdfs:label "Water Treatment Works State"@en-GB ;
  rdfs:comment "The class of all states of a water treatment
    works."@en-GB ;
  rdfs:subClassOf ies:State .

ex:OperationalWTWState a ies:ClassOfState ;
  rdfs:label "Operational Water Treatment Works State"@en-GB ;
  rdfs:comment "A state of a WTW in which it can pump at least some
    water — i.e. at least one power source is available and the
    pumping system is at least partially functional."@en-GB ;
  rdfs:subClassOf ex:WTWState .

ex:FullyOperationalWTWState a ies:ClassOfState ;
  rdfs:label "Fully Operational Water Treatment Works State"@en-GB ;
  rdfs:comment "A state of a WTW in which mains power is available and
    the facility operates at full design capacity."@en-GB ;
  rdfs:subClassOf ex:OperationalWTWState .

ex:DegradedWTWState a ies:ClassOfState ;
  rdfs:label "Degraded Water Treatment Works State"@en-GB ;
  rdfs:comment "A state of a WTW in which mains power is unavailable
    and the facility operates on backup generator power at reduced
    capacity."@en-GB ;
  rdfs:subClassOf ex:OperationalWTWState .

ex:NonOperationalWTWState a ies:ClassOfState ;
  rdfs:label "Non-Operational Water Treatment Works State"@en-GB ;
  rdfs:comment "A state of a WTW in which no power source is available
    and no water can be pumped."@en-GB ;
  rdfs:subClassOf ex:WTWState .
```

The subclass hierarchy encodes the partial order, mirroring the Eurofighter pattern from Appendix D:

- Every `FullyOperationalWTWState` is an `OperationalWTWState` (mains power implies at least some power).
- Every `DegradedWTWState` is also an `OperationalWTWState` (backup power implies at least some power).
- `FullyOperationalWTWState` and `DegradedWTWState` are *not* in a subclass relationship with each other (a degraded state is not a fully operational state, nor vice versa).
- `NonOperationalWTWState` is *not* a subclass of `OperationalWTWState`.

E.5.2 State classes for the Sandown WTW (individual level)

```
# --- Sandown WTW state classes: individual-level ---

ex:SandownWTWState a ies:ClassOfState ;
    rdfs:label "Sandown WTW State"@en-GB ;
    rdfs:subClassOf ex:WTWState ;
    ies:isAllStatesOf data:SandownWTW .

ex:OperationalStateOfSandownWTW a ies:ClassOfState ;
    rdfs:label "Operational State of Sandown WTW"@en-GB ;
    rdfs:subClassOf ex:SandownWTWState ,
        ex:OperationalWTWState .

ex:FullyOperationalStateOfSandownWTW a ies:ClassOfState ;
    rdfs:label "Fully Operational State of Sandown WTW"@en-GB ;
    rdfs:subClassOf ex:OperationalStateOfSandownWTW ,
        ex:FullyOperationalWTWState .

ex:DegradedStateOfSandownWTW a ies:ClassOfState ;
    rdfs:label "Degraded State of Sandown WTW"@en-GB ;
    rdfs:subClassOf ex:OperationalStateOfSandownWTW ,
        ex:DegradedWTWState .

ex:NonOperationalStateOfSandownWTW a ies:ClassOfState ;
    rdfs:label "Non-Operational State of Sandown WTW"@en-GB ;
    rdfs:subClassOf ex:SandownWTWState ,
        ex:NonOperationalWTWState .
```

E.5.3 State classes for the sub-station

The sub-station has a simpler binary spectrum — operational or non-operational — since its role in this scenario is as a dependency, not as the primary subject of degradation analysis.

```
# --- Sub-station state classes: type-level ---

ex:SubStationState a ies:ClassOfState ;
    rdfs:label "Electrical Sub-Station State"@en-GB ;
    rdfs:subClassOf ies:State .

ex:OperationalSubStationState a ies:ClassOfState ;
    rdfs:label "Operational Sub-Station State"@en-GB ;
    rdfs:comment "A state of a sub-station in which it supplies
        electrical power to its connected consumers."@en-GB ;
    rdfs:subClassOf ex:SubStationState .

ex:NonOperationalSubStationState a ies:ClassOfState ;
    rdfs:label "Non-Operational Sub-Station State"@en-GB ;
    rdfs:comment "A state of a sub-station in which it is unable to
        supply electrical power."@en-GB ;
    rdfs:subClassOf ex:SubStationState .

# --- Sandown Sub-Station state classes: individual-level ---

ex:SandownSubStationState a ies:ClassOfState ;
    rdfs:label "Sandown Sub-Station State"@en-GB ;
```

```

    rdfs:subClassOf ex:SubStationState ;
    ies:isAllStatesOf data:SandownSubStation .

ex:OperationalStateOfSandownSubStation a ies:ClassOfState ;
    rdfs:label "Operational State of Sandown Sub-Station"@en-GB ;
    rdfs:subClassOf ex:SandownSubStationState ,
        ex:OperationalSubStationState .

ex:NonOperationalStateOfSandownSubStation a ies:ClassOfState ;
    rdfs:label "Non-Operational State of Sandown Sub-Station"@en-GB ;
    rdfs:subClassOf ex:SandownSubStationState ,
        ex:NonOperationalSubStationState .

```

E.5.4 State classes for the backup generator

```

# --- Backup generator state classes: type-level ---

ex:BackupGeneratorState a ies:ClassOfState ;
    rdfs:label "Backup Generator State"@en-GB ;
    rdfs:subClassOf ies:State .

ex:OperationalBackupGeneratorState a ies:ClassOfState ;
    rdfs:label "Operational Backup Generator State"@en-GB ;
    rdfs:comment "A state of a backup generator in which it is running
        and providing emergency electrical power."@en-GB ;
    rdfs:subClassOf ex:BackupGeneratorState .

# --- Sandown backup generator state classes: individual-level ---

ex:SandownBackupGeneratorState a ies:ClassOfState ;
    rdfs:label "Sandown WTW Backup Generator State"@en-GB ;
    rdfs:subClassOf ex:BackupGeneratorState ;
    ies:isAllStatesOf data:SandownBackupGenerator .

ex:OperationalStateOfSandownBackupGenerator a ies:ClassOfState ;
    rdfs:label "Operational State of Sandown WTW Backup Generator"@en-GB ;
    rdfs:subClassOf ex:SandownBackupGeneratorState ,
        ex:OperationalBackupGeneratorState .

```

E.6 Disposition Assertions

E.6.1 Class-level dispositions (§3.2)

All water treatment works have the disposition to pump water. All electrical sub-stations have the disposition to supply power.

```

# Type-level dispositions
ex:OperationalWTWState
    ies:eachHasDisposition ex:PumpWaterReducedCapacity .

ex:FullyOperationalWTWState
    ies:eachHasDisposition ex:PumpWaterFullCapacity .

ex:OperationalSubStationState
    ies:eachHasDisposition ex:SupplyElectricalPower .

```

```
ex:OperationalBackupGeneratorState
  ies:eachHasDisposition ex:ProvideEmergencyPower .
```

Note the placement of disposition assertions in the hierarchy. The reduced-capacity pumping disposition is asserted on `OperationalWTWState` — the parent of both the fully operational and degraded states — because any operational WTW can pump at least at reduced capacity. The full-capacity disposition is asserted only on `FullyOperationalWTWState`, because only a fully powered WTW can achieve full capacity. This mirrors the Eurofighter pattern from Appendix C, where the degraded-flight disposition was asserted on the operational parent and the Mach 2 disposition only on the fully operational subclass.

E.6.2 Individual-level dispositions (§3.3)

The Sandown WTW has been individually assessed as having both full and reduced pumping capability:

```
data:SandownWTW ies:hasDisposition ex:PumpWaterFullCapacity .
data:SandownWTW ies:hasDisposition ex:PumpWaterReducedCapacity .

data:SandownSubStation ies:hasDisposition ex:SupplyElectricalPower .

data:SandownBackupGenerator ies:hasDisposition ex:ProvideEmergencyPower .
```

These are whole-life disposition assertions: the Sandown WTW *can* pump water at full capacity and *can* pump at reduced capacity. Which it is actually doing at any moment is determined by which `ClassOfState` its current temporal state is a member of.

E.6.3 Disposition inheritance through the subclass hierarchy

Through the subclass chain, each level of the spectrum inherits dispositions from its ancestors:

- A member of `FullyOperationalStateOfSandownWTW` inherits `eachHasDisposition ex:PumpWaterReducedCapacity` (from `OperationalWTWState`) and has `eachHasDisposition ex:PumpWaterFullCapacity` (from `FullyOperationalWTWState`). It can pump at both full and reduced capacity.
- A member of `DegradedStateOfSandownWTW` inherits `eachHasDisposition ex:PumpWaterReducedCapacity` (from `OperationalWTWState`) but does *not* inherit `PumpWaterFullCapacity`, because `DegradedWTWState` is not a subclass of `FullyOperationalWTWState`. It can pump only at reduced capacity.
- A member of `NonOperationalStateOfSandownWTW` inherits no pumping dispositions — `NonOperationalWTWState` is not a subclass of `OperationalWTWState`.

This is the same inheritance mechanism demonstrated for the Eurofighter in §B.3.6: the disposition set narrows as we move down the spectrum.

E.7 Dependency Assertions

This is the central contribution of this appendix. The Sandown WTW has two kinds of dependency: a cross-entity dependency on the external sub-station, and a whole-part dependency on the internal backup generator. These dependencies operate at different levels of the degradation spectrum.

E.7.1 Cross-entity dependency: full capacity requires mains power (§3.6)

The WTW's full operational capability depends on the Sandown Sub-Station being in an operational state. The sub-station is not part of the WTW — it is a separate entity, part of the IoW electricity distribution network. This is a cross-entity dependency, structurally parallel to the Eurofighter–Voyager dependency in §5.2.5 and the evacuation route–ferry terminal dependency in §D.9.

```
# Type-level: every fully operational WTW state requires a co-occurring
# operational sub-station state
ex:FullyOperationalWTWState
    ies:eachRequires ex:OperationalSubStationState .

# Individual-level: the Sandown WTW's fully operational state requires
# the Sandown Sub-Station to be operational
ex:FullyOperationalStateOfSandownWTW
    ies:eachRequires ex:OperationalStateOfSandownSubStation .
```

The `eachRequires` assertion reads: "Every member of `FullyOperationalStateOfSandownWTW` requires the temporal co-occurrence of at least one member of `OperationalStateOfSandownSubStation`." If the Sandown Sub-Station is not in an operational state, then no fully operational state of the Sandown WTW can exist during that period. The WTW may still be operational — but only in a degraded mode.

E.7.2 Whole-part dependency: degraded mode requires the backup generator (§3.5)

When mains power is lost, the WTW's ability to continue operating (at reduced capacity) depends on the backup generator — which is a part of the WTW — being in an operational state. This is a whole-part dependency, structurally parallel to the Eurofighter's dependency on its propulsion system in §5.2.5.

```
# Type-level: every degraded WTW state requires an operational backup
# generator as a part
ex:DegradedWTWState
    ies:eachRequiresPart ex:OperationalBackupGeneratorState .

# Individual-level: the Sandown WTW's degraded state requires the
# Sandown backup generator to be operational
ex:DegradedStateOfSandownWTW
    ies:eachRequiresPart ex:OperationalStateOfSandownBackupGenerator .
```

The `eachRequiresPart` assertion reads: "Every member of `DegradedStateOfSandownWTW` requires, as a mereological part, at least one member of `OperationalStateOfSandownBackupGenerator`." The backup generator is a part of the WTW (via `ies:isPartOf`), so the generator's operational state is a temporal part of the WTW's degraded state — the mereological relationship holds both structurally and temporally.

E.7.3 The cascading dependency

Taken together, these assertions encode a cascading dependency — the pattern identified and deferred as a planned annex in §7.4:

1. **Sandown WTW fully operational** → requires (cross-entity) Sandown Sub-Station operational.
2. **Sandown Sub-Station fails** → WTW can no longer be in a fully operational state.
3. **Sandown WTW degrades to backup power** → requires (whole-part) backup generator operational.
4. **If backup generator also fails** → WTW enters non-operational state — no pumping at all.

The cascade is not modelled as a single chain of properties. Instead, it emerges naturally from the combination of the state class hierarchy, the disposition assertions at each level, and the different dependency properties (`eachRequires` for the cross-entity link, `eachRequiresPart` for the whole-part link) at each level. Each level of the spectrum has its own set of dependencies, and the state transition from one level to another is determined by which dependencies are satisfied.

This is a key observation: *cascading dependencies do not require new ontological machinery*. The six proposed properties, combined with the `ClassOfState` hierarchy, are sufficient to represent arbitrarily complex dependency cascades. The cascade is an emergent property of the compositional modelling, not a dedicated construct.

E.8 The Actuality: Sub-Station Failure Event

On 15 January 2026 at 14:30 UTC, the Sandown Sub-Station suffers a failure. The WTW's backup generator activates automatically, and the WTW transitions from a fully operational state to a degraded (backup power) state. The mains power is restored at 18:45 UTC the same day, and the WTW returns to full operation.

E.8.1 The event

```
data:SandownSubStationFailure20260115 a ies:Event ;
    rdfs:label "Sandown Sub-Station failure, 15 January 2026"@en-GB ;
    rdfs:comment "Failure of the Sandown Sub-Station causing loss of
        mains power to the Sandown Water Treatment Works."@en-GB .

data:SandownSubStation ies:isParticipantIn
    data:SandownSubStationFailure20260115 .
```

E.8.2 States of the sub-station

Before the failure, the sub-station is operational. During the failure, it is non-operational. After repair, it returns to an operational state.

```
# --- Sub-station: operational state before the failure ---

data:SandownSubStation-Operational-Pre20260115
  a ex:OperationalStateOfSandownSubStation ;
  ies:isStateOf data:SandownSubStation ;
  ies:hasDisposition ex:SupplyElectricalPower ;
  rdfs:label "Sandown Sub-Station operational (before failure)"@en-GB .

data:SubStationOperationalEnd a ies:BoundingState ;
  ies:isEndOf data:SandownSubStation-Operational-Pre20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T14:30:00Z"^^xsd:dateTime ] .

# --- Sub-station: non-operational state during the failure ---

data:SandownSubStation-Failed-20260115
  a ex:NonOperationalStateOfSandownSubStation ;
  ies:isStateOf data:SandownSubStation ;
  rdfs:label "Sandown Sub-Station failed, 15 January 2026"@en-GB .

data:SubStationFailureStart a ies:BoundingState ;
  ies:isStartOf data:SandownSubStation-Failed-20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T14:30:00Z"^^xsd:dateTime ] .

data:SubStationFailureEnd a ies:BoundingState ;
  ies:isEndOf data:SandownSubStation-Failed-20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T18:45:00Z"^^xsd:dateTime ] .

# --- Sub-station: operational state after repair ---

data:SandownSubStation-Operational-Post20260115
  a ex:OperationalStateOfSandownSubStation ;
  ies:isStateOf data:SandownSubStation ;
  ies:hasDisposition ex:SupplyElectricalPower ;
  rdfs:label "Sandown Sub-Station operational (after repair)"@en-GB .

data:SubStationRestoredStart a ies:BoundingState ;
  ies:isStartOf data:SandownSubStation-Operational-Post20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T18:45:00Z"^^xsd:dateTime ] .
```

E.8.3 States of the WTW

The WTW transitions through three states: fully operational (before the failure), degraded (during the failure, running on backup power), and fully operational again (after mains power is restored).

```

# --- WTW: fully operational state before the failure ---

data:SandownWTW-FullyOp-Pre20260115
  a ex:FullyOperationalStateOfSandownWTW ;
  ies:isStateOf data:SandownWTW ;
  ies:hasDisposition ex:PumpWaterFullCapacity ;
  rdfs:label "Sandown WTW fully operational (before failure)"@en-GB .

data:WTWFullyOpEnd a ies:BoundingState ;
  ies:isEndOf data:SandownWTW-FullyOp-Pre20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T14:30:00Z"^^xsd:dateTime ] .

# --- The satisfied cross-entity dependency: the sub-station's operational
#       state is the external dependency that sustains the WTW's full operation
---

data:SandownSubStation-Operational-Pre20260115
  ies:isParticipantIn data:SandownSubStationFailure20260115 .
# --- WTW: degraded state during the failure ---

data:SandownWTW-Degraded-20260115
  a ex:DegradedStateOfSandownWTW ;
  ies:isStateOf data:SandownWTW ;
  ies:hasDisposition ex:PumpWaterReducedCapacity ;
  ies:requiresPart ex:OperationalStateOfSandownBackupGenerator ;
  rdfs:label "Sandown WTW degraded – backup power, 15 January
    2026"@en-GB .

data:WTWDegradedStart a ies:BoundingState ;
  ies:isStartOf data:SandownWTW-Degraded-20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T14:30:00Z"^^xsd:dateTime ] .

data:WTWDegradedEnd a ies:BoundingState ;
  ies:isEndOf data:SandownWTW-Degraded-20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T18:45:00Z"^^xsd:dateTime ] .

# --- The backup generator's operational state – a part of the WTW's
#       degraded state, satisfying the whole-part dependency ---

data:SandownBackupGenerator-Running-20260115
  a ex:OperationalStateOfSandownBackupGenerator ;
  ies:isStateOf data:SandownBackupGenerator ;
  ies:hasDisposition ex:ProvideEmergencyPower ;
  ies:isPartOf data:SandownWTW-Degraded-20260115 ;
  rdfs:label "Sandown WTW backup generator running, 15 January
    2026"@en-GB .

data:GeneratorStart a ies:BoundingState ;
  ies:isStartOf data:SandownBackupGenerator-Running-20260115 ;
  ies:inPeriod [ a ies:ParticularPeriod ;
    rdfs:label "2026-01-15T14:30:00Z"^^xsd:dateTime ] .

data:GeneratorStop a ies:BoundingState ;
  ies:isEndOf data:SandownBackupGenerator-Running-20260115 ;

```

```

    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2026-01-15T18:45:00Z"^^xsd:dateTime ] .
# --- WTW: fully operational state after restoration ---

data:SandownWTW-FullyOp-Post20260115
    a ex:FullyOperationalStateOfSandownWTW ;
    ies:isStateOf data:SandownWTW ;
    ies:hasDisposition ex:PumpWaterFullCapacity ;
    rdfs:label "Sandown WTW fully operational (after restoration)"@en-GB .

data:WTWRestoredStart a ies:BoundingState ;
    ies:isStartOf data:SandownWTW-FullyOp-Post20260115 ;
    ies:inPeriod [ a ies:ParticularPeriod ;
        rdfs:label "2026-01-15T18:45:00Z"^^xsd:dateTime ] .

```

E.8.4 Structural comparison with Appendix C

Compare the WTW's degraded state data with the Eurofighter's limp-mode data from §C.3.7. The structure is identical:

Eurofighter (§C.3.7)	Sandown WTW (§E.8.3)
data:FighterXYZ-123-LimpMode-20240717	data:SandownWTW-Degraded-20260115
a ex:DegradedStateOfFighterXYZ-123	a ex:DegradedStateOfSandownWTW
ies:hasDisposition ex:FlyDegraded	ies:hasDisposition ex:PumpWaterReducedCapacity
ies:requiresPart ... (propulsion system state)	ies:requiresPart ... (backup generator state)
Part: ies:isPartOf fighter's degraded state	Part: ies:isPartOf WTW's degraded state

The only structural addition in this appendix is the *cross-entity* dependency (§E.7.1) — the WTW's fully operational state depending on the sub-station — which uses `eachRequires` rather than `eachRequiresPart`. This exercises a dependency type that does not appear in Appendix C's single-asset scenario.

E.9 Queries

The following SPARQL queries illustrate how the recorded data can answer operational questions about the WTW's status and dependencies.

Query 1: What are the Sandown WTW's current dispositions?

Given a point in time, retrieve the WTW's active state and its available dispositions:

```

PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX ex: <http://example.org/iow-water#>
PREFIX data: <http://example.org/iow-water/data#>

SELECT ?state ?stateLabel ?disposition ?dispLabel

```

```

WHERE {
    ?state ies:isStateOf data:SandownWTW ;
        rdfs:label ?stateLabel ;
        ies:hasDisposition ?disposition .
    ?disposition rdfs:label ?dispLabel .
}

```

During the failure (14:30–18:45 on 15 January 2026), this returns the degraded state with `PumpWaterReducedCapacity`. Before and after, it returns the fully operational state with `PumpWaterFullCapacity`.

Query 2: What external dependencies does the Sandown WTW's full operation have?

```

PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX ex: <http://example.org/iow-water#>

SELECT ?dependency ?depLabel
WHERE {
    ex:FullyOperationalStateOfSandownWTW
        ies:eachRequires ?dependency .
    ?dependency rdfs:label ?depLabel .
}

```

This returns `OperationalStateOfSandownSubStation` — the cross-entity dependency that, when unsatisfied, causes the WTW to degrade.

Query 3: What internal dependencies does the Sandown WTW's degraded operation have?

```

PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX ex: <http://example.org/iow-water#>

SELECT ?partDependency ?partDepLabel
WHERE {
    ex:DegradedStateOfSandownWTW
        ies:eachRequiresPart ?partDependency .
    ?partDependency rdfs:label ?partDepLabel .
}

```

This returns `OperationalStateOfSandownBackupGenerator` — the whole-part dependency that, if also unsatisfied, would push the WTW into a non-operational state.

Query 4: Which facilities lost capability due to the sub-station failure?

This query finds all entities whose fully operational state depends on the failed sub-station:

```

PREFIX ies: <http://informationexchangestandard.org/ont/ies/common/>
PREFIX ex: <http://example.org/iow-water#>
PREFIX data: <http://example.org/iow-water/data#>

SELECT ?entityStateClass ?label
WHERE {
    ?entityStateClass ies:eachRequires

```

```

    ex:OperationalStateOfSandownSubStation .
    ?entityStateClass rdfs:label ?label .
}

```

In this simple example, only the Sandown WTW's fully operational state is returned. In a richer dataset — where multiple facilities depend on the same sub-station — the query would reveal the full impact footprint of the failure, directly supporting emergency planning and impact assessment.

E.10 Summary

This appendix demonstrates three things:

1. **Cascading infrastructure dependencies require no new ontological machinery.** The combination of `eachRequires` (for the cross-entity dependency on the external sub-station) and `eachRequiresPart` (for the whole-part dependency on the internal backup generator) is sufficient to model the cascade from full capability through degraded operation to non-operational status. The cascade emerges from the interaction of the state class hierarchy and the dependency properties — it is a compositional consequence, not a dedicated construct.
2. **The degradation spectrum pattern (Appendix B) applies to infrastructure facilities as naturally as to military assets.** The WTW's three-level spectrum (fully operational, degraded, non-operational) follows exactly the same structural pattern as the Eurofighter's spectrum: distinct `ClassOfState` definitions at each level, subclass relationships encoding the partial order, and disposition assertions scoped to each level. The only difference is the domain vocabulary.
3. **Cross-entity and whole-part dependencies can coexist at different levels of the same spectrum.** The WTW's full capability depends on an external entity (cross-entity, via `eachRequires`), while its degraded capability depends on an internal part (whole-part, via `eachRequiresPart`). The disposition pattern's property vocabulary — which distinguishes these two kinds of dependency at the semantic level — enables a consuming system to distinguish external supply failures from internal component failures, supporting different response strategies for each.

Together with Appendix B (degradation and enhancement within a single asset) and Appendix C (linear infrastructure dependencies along a corridor), this appendix completes the set of deferred annex scenarios identified in §7.4, demonstrating that the six proposed properties handle cascading infrastructure dependencies without extension.